Physics-Informed Neural Networks

Lecture Notes: Foundations, Training, Applications, and Outlook

PINNs Course

September 29, 2026

Contents

1 Why Neural Networks for PDEs? 1

1.1 Introduction and learning goals . . . 1

1.2 Historical background . . . 2

1.2.1 The classical numerical PDE legacy . . . 2

1.2.2 Early neural approaches to differential equations . . . 2

1.2.3 Dissanayake and Phan-Thien (1994) . . . 3

1.2.4 Lagaris, Likas and Fotiadis (1998) . . . 3

1.2.5 Why modern PINNs became practical . . . 4

1.2.6 The 2019 PINN framework . . . 4

1.3 PDEs and boundary value problems . . . 5

1.3.1 What is a PDE? . . . 5

1.3.2 Strong and weak formulations . . . 6

1.3.3 Examples: Poisson and heat equations . . . 6

1.4 Classical numerical solvers . . . 6

1.4.1 Finite difference methods . . . 6

1.4.2 Finite element methods . . . 7

1.4.3 Accuracy and mathematical theory . . . 7

1.4.4 Where classical methods can become difficult . . . 7

1.5 Approximating PDE solutions as functions . . . 8

1.5.1 The solution itself is a function . . . 8

1.5.2 Parameterized approximation is not unique to neural networks . . . 8

1.5.3 Constraints become loss terms . . . 8

1.6 What is a neural network really? . . . 9

1.6.1 From linear regression to nonlinear feature learning . . . 9

1.6.2 The single neuron . . . 9

1.6.3 Multilayer perceptrons . . . 10

1.6.4 Training as optimization . . . 10

1.7 The core PINN formulation . . . 10
1.7.1 Neural solution ansatz . . . 10
1.7.2 Automatic differentiation . . . 11
1.7.3 PDE residual . . . 11
1.7.4 Example: Poisson equation . . . 11
1.8 Where PINNs can be attractive . . . 12
1.8.1 Sparse or noisy observations . . . 12
1.8.2 Unknown coefficients and inverse problems . . . 12
1.8.3 Difficult meshes and repeated queries . . . 12
1.9 Where PINNs can struggle . . . 13
1.9.1 High-frequency and oscillatory solutions . . . 13
1.9.2 Shocks and sharp layers . . . 13
1.9.3 Multiscale and stiff problems . . . 13
1.9.4 Loss balancing and optimization instability . . . 13
1.10 Scientific reproducibility . . . 13
1.11 Takeaway . . . 14
Further reading . . . 14

2 Demystifying Neural Networks 16
2.1 Overview and learning goals . . . 16
2.2 Historical background . . . 16
2.3 A single neuron . . . 17
2.3.1 Affine map plus nonlinearity . . . 17
2.3.2 Why nonlinearity is essential . . . 17
2.4 From linear regression to multilayer perceptrons . . . 18
2.4.1 Linear and nonlinear regression . . . 18
2.4.2 Layer notation . . . 18
2.4.3 Depth and width . . . 18
2.5 Activation functions . . . 19
2.5.1 ReLU . . . 19
2.5.2 Hyperbolic tangent . . . 19
2.5.3 Other activations . . . 20
2.6 Loss functions and empirical risk . . . 20
2.7 Gradient descent . . . 21
2.8 Stochastic gradient descent . . . 21
2.8.1 Why stochastic gradients? . . . 21

2.8.2 Mini-batches, epochs, and shuffling . . . 22
2.8.3 Geometric intuition for noisy updates . . . 22
2.8.4 SGD pseudocode . . . 23
2.9 Momentum . . . 23
2.10 The Adam algorithm . . . 23
2.11 Backpropagation . . . 24
2.11.1 Computational graphs and the chain rule . . . 24
2.11.2 One-layer derivation . . . 25
2.12 Automatic differentiation and the link to PINNs . . . 25
2.13 Training diagnostics . . . 25
2.14 Summary . . . 26
Selected references . . . 26

3 Automatic Differentiation for Physics 28
3.1 Introduction and learning goals . . . 28
3.2 Why derivatives matter in scientific machine learning . . . 29
3.2.1 Training derivatives and physics derivatives . . . 29
3.3 Three approaches to differentiation . . . 29
3.3.1 Symbolic differentiation . . . 29
3.3.2 Numerical differentiation . . . 30
3.3.3 Automatic differentiation . . . 30
3.4 Computational graphs and the chain rule . . . 31
3.4.1 A simple graph . . . 31
3.4.2 Elementary derivative rules . . . 32
3.4.3 Neural networks as large computation graphs . . . 32
3.5 Forward-mode automatic differentiation . . . 32
3.5.1 Tangent propagation . . . 32
3.5.2 Dual-number interpretation . . . 33
3.6 Reverse-mode automatic differentiation and backpropagation . . . 33
3.6.1 Adjoint propagation . . . 33
3.6.2 Why reverse mode dominates neural-network training . . . 34
3.6.3 Forward versus reverse mode . . . 34
3.7 Jacobians, gradients, Hessians, and PDE derivatives . . . 35
3.7.1 Jacobians . . . 35
3.7.2 Gradients and Hessians . . . 35
3.7.3 Input derivatives versus parameter derivatives . . . 35

3.7.4 First derivatives: transport equation . . . 36
3.7.5 Second derivatives . . . 36
3.7.6 Mixed derivatives and smoothness . . . 36
3.8 PDE examples . . . 36
3.8.1 Poisson equation . . . 36
3.8.2 Heat equation . . . 37
3.8.3 Burgers' equation . . . 37
3.8.4 Inverse coefficient problem . . . 38
3.9 Software implementation . . . 38
3.9.1 PyTorch-style input derivatives . . . 38
3.9.2 Heat-equation residual . . . 39
3.9.3 Frameworks . . . 39
3.9.4 Batching and vectorization . . . 40
3.10 Practical issues and limitations . . . 40
3.10.1 Memory cost of reverse mode . . . 40
3.10.2 Differentiability pitfalls . . . 40
3.10.3 Higher-order derivatives and approximation quality . . . 40
3.10.4 Stiffness and scaling . . . 41
3.10.5 Finite differences still have a role . . . 41
3.11 A unified view of PINN differentiation . . . 41
3.12 Summary . . . 42
3.13 Discussion questions and exercises . . . 42

4 From PDEs to Loss Functions 45
4.1 Introduction and learning goals . . . 45
4.2 From PDEs to optimization . . . 45
4.2.1 Historical perspective: neural trial functions . . . 45
4.2.2 The neural-network ansatz . . . 46
4.2.3 Classical workflow versus PINN workflow . . . 46
4.2.4 Anatomy of a PINN loss . . . 46
4.3 Collocation points and residuals . . . 47
4.3.1 Interior, boundary, initial and data points . . . 47
4.3.2 Strong-form residuals . . . 47
4.3.3 Automatic differentiation . . . 48
4.3.4 Residual losses as empirical norms . . . 48
4.3.5 Strong form versus weak form . . . 49

4.4 Worked example: the Poisson equation . . . 49
4.4.1 Problem statement . . . 49
4.4.2 Manufactured solution . . . 50
4.4.3 Constructing the residual . . . 50
4.4.4 What the optimizer actually sees . . . 50
4.5 Boundary conditions: soft and hard enforcement . . . 51
4.5.1 Soft enforcement . . . 51
4.5.2 Hard enforcement . . . 51
4.5.3 Neumann and Robin conditions . . . 51
4.6 Adding observations and inverse information . . . 52
4.7 Loss weights, scaling, and nondimensionalization . . . 52
4.7.1 Why weights matter . . . 52
4.7.2 Units and scales . . . 52
4.8 Implementation pattern . . . 52
4.9 Training diagnostics and failure modes . . . 53
4.9.1 Monitor individual components . . . 53
4.9.2 Small training loss does not imply small solution error . . . 53
4.9.3 Typical failure modes . . . 54
4.10 Adaptive collocation strategies . . . 54
4.11 Extensions . . . 54
4.11.1 Heat equation . . . 54
4.11.2 Burgers' equation . . . 55
4.11.3 Inverse coefficient problems . . . 55
4.11.4 Wave equation exercise . . . 55
4.12 Connection with finite element residuals . . . 56
4.13 Computational cost . . . 56
4.14 Reproducibility and scientific reporting . . . 56
4.15 Summary . . . 57

5 Time-Dependent Problems 59
5.1 Introduction and learning objectives . . . 59
5.2 Evolution equations and spacetime domains . . . 59
5.2.1 PINN viewpoint . . . 60
5.3 Sampling the spacetime domain . . . 60
5.4 Initial and boundary conditions . . . 61
5.4.1 Why the initial condition matters . . . 61

5.4.2 Boundary conditions . . . 61
5.4.3 Hard and soft enforcement . . . 61
5.5 Worked example: the heat equation . . . 62
5.5.1 Exact benchmark solution . . . 62
5.5.2 PINN residual and loss . . . 63
5.5.3 Manufactured solutions . . . 63
5.5.4 Conceptual implementation . . . 63
5.6 Classical time-marching methods . . . 64
5.6.1 Method of lines . . . 64
5.6.2 Explicit Euler . . . 64
5.6.3 Backward Euler . . . 64
5.6.4 Crank–Nicolson . . . 65
5.7 Global-in-time PINNs . . . 65
5.8 Long-time dynamics and causal training . . . 65
5.8.1 Causal weighting . . . 65
5.8.2 Time-domain decomposition . . . 66
5.8.3 Curriculum and windowing . . . 66
5.8.4 Adaptive sampling in time . . . 66
5.9 Data-assisted PINNs and inverse problems . . . 66
5.10 Other time-dependent PDEs . . . 67
5.10.1 Wave equation . . . 67
5.10.2 Reaction–diffusion equations . . . 67
5.10.3 Burgers' equation . . . 68
5.11 Practical checklist . . . 68
5.12 Summary . . . 68

6 Nonlinear PDEs and Training Challenges 71
6.1 Overview and learning objectives . . . 71
6.2 Why nonlinear PDEs are different . . . 72
6.2.1 From linear models to nonlinear physics . . . 72
6.3 The viscous Burgers equation . . . 73
6.3.1 Equation and interpretation . . . 73
6.3.2 A schematic illustration . . . 73
6.4 PINN formulation for Burgers' equation . . . 74
6.4.1 Neural-network approximation and residual . . . 74
6.4.2 Loss components . . . 74

6.4.3 A generalized nonlinear residual . . . 75
6.5 What can go wrong? . . . 75
6.5.1 Slow convergence and plateaus . . . 75
6.5.2 A smooth but wrong solution . . . 76
6.5.3 Objective imbalance . . . 76
6.5.4 Poor generalization between collocation points . . . 76
6.6 Why the optimization problem is difficult . . . 77
6.6.1 Gradient descent and competing directions . . . 77
6.6.2 Training stiffness . . . 77
6.6.3 Neural tangent kernel intuition . . . 77
6.6.4 Differential operators and conditioning . . . 77
6.6.5 Useful diagnostics . . . 78
6.7 Smoothness and spectral bias . . . 78
6.7.1 Low frequencies are often learned first . . . 78
6.7.2 Activation functions . . . 79
6.7.3 Fourier features . . . 79
6.7.4 Sinusoidal networks . . . 79
6.7.5 Domain decomposition . . . 79
6.8 Sampling collocation points . . . 80
6.8.1 Why sampling matters . . . 80
6.8.2 Uniform sampling . . . 80
6.8.3 Random and quasi-random sampling . . . 80
6.8.4 Residual-based adaptive refinement . . . 80
6.8.5 Boundary and initial sampling . . . 81
6.9 Advanced strategies . . . 81
6.9.1 Loss balancing . . . 81
6.9.2 Curriculum and continuation . . . 81
6.9.3 Causal and time-marching PINNs . . . 81
6.9.4 Weak and conservative formulations . . . 82
6.10 A practical workflow for nonlinear PINNs . . . 82
6.11 Summary . . . 82
6.12 Exercises . . . 83

7 PINNs versus Classical Numerical Methods 85
7.1 Motivation: choosing the right computational tool . . . 85
7.2 Two computational philosophies . . . 85

7.2.1 The PINN workflow . . . 85
7.2.2 The classical numerical workflow . . . 86
7.3 A review of classical numerical methods . . . 86
7.3.1 Finite difference methods . . . 86
7.3.2 Finite element methods . . . 86
7.3.3 Finite volume methods . . . 87
7.3.4 Spectral methods . . . 87
7.3.5 Why classical methods remain the default in many settings . . . 87
7.4 Accuracy: what should actually be compared? . . . 88
7.4.1 Classical error decomposition . . . 88
7.4.2 A useful PINN error picture . . . 88
7.5 Computational cost: offline versus online . . . 89
7.6 Mesh-based and mesh-free viewpoints . . . 89
7.6.1 What a mesh provides . . . 89
7.6.2 What “mesh-free” means for PINNs . . . 90
7.6.3 High-dimensional settings . . . 90
7.7 When PINNs are attractive . . . 90
7.7.1 Inverse problems . . . 90
7.7.2 Sparse measurements and data assimilation . . . 91
7.7.3 Digital twins and real-time use . . . 91
7.7.4 Parameterized problems and uncertainty quantification . . . 92
7.8 When PINNs struggle . . . 92
7.8.1 High-frequency solutions . . . 92
7.8.2 Multiscale problems . . . 92
7.8.3 Sharp layers and shocks . . . 93
7.8.4 Stiffness and optimization . . . 93
7.8.5 Very high accuracy requirements . . . 93
7.8.6 Large industrial simulations . . . 93
7.9 Hybrid and advanced viewpoints . . . 93
7.9.1 Weak-form and variational PINNs . . . 93
7.9.2 Operator learning . . . 94
7.10 Benchmarking and reproducibility . . . 94
7.11 A practical decision framework . . . 95
7.12 Worked discussion examples . . . 96
7.13 Open research questions . . . 97

7.14 Summary . . . 97

8 Applications and Outlook 100

8.1 From PINN methodology to scientific applications . . . 100
8.1.1 Physics, data, and learning . . . 100
8.1.2 What PINNs add beyond curve fitting . . . 101
8.2 Forward and inverse problems . . . 101
8.2.1 The distinction . . . 101
8.2.2 Example: unknown diffusion coefficient . . . 102
8.2.3 Physics as regularization . . . 103
8.2.4 What quantities may be inferred? . . . 103
8.2.5 Scalar parameter identification: Burgers' equation . . . 103
8.2.6 Functional coefficient identification . . . 103
8.2.7 Identifiability and sensor placement . . . 104
8.3 Data-assisted PINNs . . . 104
8.3.1 Combining physics and observations . . . 104
8.3.2 Sparse measurements . . . 104
8.3.3 Noisy measurements . . . 104
8.3.4 Loss balancing . . . 105
8.3.5 Validation data . . . 105
8.3.6 Purely data-driven learning versus data-assisted PINNs . . . 106
8.4 Representative applications . . . 106
8.4.1 Diffusion and heat transfer . . . 106
8.4.2 Inverse heat conduction . . . 107
8.4.3 Pollutant transport . . . 107
8.4.4 Fluid dynamics . . . 107
8.4.5 Pressure recovery from velocity measurements . . . 107
8.4.6 Reaction–diffusion systems . . . 107
8.4.7 Digital-twin viewpoint . . . 108
8.5 Why realistic applications are difficult . . . 108
8.6 Advanced PINN methods . . . 109
8.6.1 Residual-based adaptive sampling . . . 109
8.6.2 Improved representations and Fourier features . . . 109
8.6.3 Domain decomposition . . . 110
8.6.4 Neural operators . . . 110
8.6.5 Hybrid classical–learning methods . . . 110

8.6.6 Software ecosystems . . . 111
8.7 Uncertainty, validation, and reproducibility . . . 111
8.7.1 Why uncertainty matters . . . 111
8.7.2 Scientific reproducibility . . . 111
8.7.3 Fair benchmarking . . . 112
8.7.4 A validation hierarchy . . . 112
8.7.5 Responsible scientific claims . . . 112
8.8 Summary . . . 112
Exercises . . . 113

Chapter 1

Why Neural Networks for PDEs?

1.1 Introduction and learning goals

Physics-informed neural networks (PINNs) sit at the intersection of several mature areas of applied mathematics and scientific computing. They combine numerical approximation of differential equations, flexible function approximation by neural networks, automatic differentiation, and optimization. For this reason, it is useful to begin a course on PINNs not with machine learning formalism, but with the broader numerical and historical context in which the method belongs.

The aim of this lecture is to answer a basic question:

> Why might it be useful to replace a mesh-based discrete approximation of a PDE solution by a trainable continuous function?

By the end of the lecture, the reader should be able to:

- recall the structure of PDE boundary value and initial value problems;
- explain how classical methods such as finite differences and finite elements convert a PDE into a finite-dimensional algebraic problem;
- describe the historical path from early neural-network approaches for differential equations to modern PINNs;
- interpret a neural network as a flexible parameterized function rather than as a mysterious black box;
- explain how a PDE, boundary conditions, and initial conditions can be transformed into loss terms;
- recognize situations in which PINNs are promising, as well as important situations in which classical numerical methods remain preferable.

The key conceptual message is that a PINN is not simply a neural network applied to a PDE. It is a particular way of reformulating a differential equation as a constrained function approximation problem. Three long-standing ideas are brought together:

1. approximate an unknown solution by a finite-dimensional family of functions;

2. measure the violation of the governing equations and data;

3. choose parameters by solving an optimization problem.

The novelty is not that any one of these ideas is new. Rather, modern PINNs became practical when they were combined with contemporary neural-network architectures, automatic differentiation, modern optimization algorithms, GPUs, and widely available machine-learning software.

1.2 Historical background

1.2.1 The classical numerical PDE legacy

For many decades, the numerical solution of partial differential equations has been dominated by finite-dimensional discretization methods. The most prominent families are:

- finite difference methods (FDM),
- finite element methods (FEM),
- finite volume methods (FVM),
- spectral and pseudospectral methods.

Although these methods are mathematically different, they share a common philosophy. One starts from a continuous problem, such as

$$\mathcal{N}[u] = f \qquad \text{in } \Omega,$$

together with boundary or initial conditions, and replaces the infinite-dimensional unknown function u by finitely many degrees of freedom. The result is typically a linear or nonlinear algebraic system. Symbolically,

$$\text{continuous PDE} \quad \longrightarrow \quad \text{finite-dimensional algebraic system.}$$

The success of these methods is difficult to overstate. They are supported by rigorous approximation theory, stability analysis, error estimates, mature software, adaptive algorithms, and decades of experience in engineering and science. Any discussion of PINNs should therefore begin from the position that classical numerical methods are not obsolete competitors. They are the benchmark against which new methods must be judged.

1.2.2 Early neural approaches to differential equations

The use of neural computation for differential equations predates the modern term "physics-informed neural network." One early example is the work of Lee and Kang in 1990, who studied neural algorithms for differential equations by writing finite-difference equations in a minimization form. Their approach already reflected a theme that remains central today: a differential equation can be transformed into a residual-based optimization problem, and a neural-style computational system can be used to reduce that residual.

A second important ingredient is the approximation capability of feed-forward neural networks. In broad terms, a one-hidden-layer network has the form

$$N(x;\theta) = \sum_{j=1}^{m} a_j \, \sigma(w_j^\top x + b_j),$$

where the parameters

$$\theta = \{a_j, w_j, b_j\}_{j=1}^{m}$$

are learned. Universal approximation results established that, for suitable activation functions and sufficiently many hidden units, such networks can approximate very broad classes of continuous functions on compact sets.

This immediately suggests a question that is now natural but was historically significant:

> If a neural network can approximate a general function, can it also approximate the unknown solution of a differential equation?

1.2.3 Dissanayake and Phan-Thien (1994)

In 1994, Dissanayake and Phan-Thien proposed a neural-network-based method for the solution of differential equations. The method used a neural network as a universal approximator, enforced the differential equation at collocation points, and minimized a residual-based objective. The examples included both linear and nonlinear differential equations.

Conceptually, this is already very close to the modern strong-form PINN framework:

1. choose a parameterized neural approximation to the unknown solution;
2. compute derivatives of that approximation;
3. evaluate the governing equation at selected points;
4. minimize the discrepancy.

The computational environment of the 1990s, however, was very different from the present one. Modern automatic differentiation systems, GPU acceleration, and mature optimization libraries were not available in the form now taken for granted.

1.2.4 Lagaris, Likas and Fotiadis (1998)

Another influential step appeared in the 1998 work of Lagaris, Likas and Fotiadis. Their approach introduced trial functions that satisfied boundary or initial conditions by construction. A typical ansatz takes the form

$$u_{\text{trial}}(x;\theta) = A(x) + B(x)N(x;\theta),$$

where A is selected to satisfy the prescribed boundary data and B is chosen so that $B = 0$ on the relevant boundary. Consequently, the neural correction cannot violate the boundary condition.

This distinction is important because it anticipates a concept that appears repeatedly in modern PINN research: *hard* versus *soft* constraint enforcement. A hard constraint is satisfied exactly by the architecture or trial function. A soft constraint is imposed through a penalty term in the loss.

Remark 1.1. The distinction between hard and soft enforcement is not merely cosmetic. Hard constraints can reduce the burden on the optimizer, but constructing suitable trial functions may be difficult for complicated geometries or mixed boundary conditions. Soft constraints are more flexible but introduce penalty weights and possible loss-balancing difficulties.

1.2.5 Why modern PINNs became practical

Many of the conceptual ingredients of PINNs therefore existed well before 2010. What changed was the computational ecosystem. Several developments came together:

- deeper neural networks and improved parameter initialization;
- large-scale gradient-based optimization methods;
- GPU acceleration;
- automatic differentiation software;
- open-source machine-learning frameworks;
- rapidly growing interest in scientific machine learning.

Automatic differentiation is especially important. For a PDE problem, one may need derivatives such as

$$u_t, \qquad u_x, \qquad u_{xx}, \qquad \nabla u, \qquad \Delta u.$$

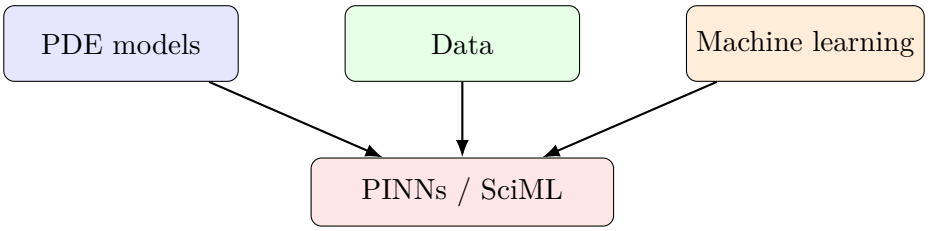
If the approximation is a neural network $\widehat{u}(x, t; \theta)$ composed of differentiable elementary operations, these derivatives can be obtained by systematically applying the chain rule to the computational graph. One does not need to derive a new finite-difference stencil for the network output, and one does not need to manipulate symbolic expressions by hand.

1.2.6 The 2019 PINN framework

Raissi, Perdikaris and Karniadakis popularized the term *physics-informed neural networks* and presented a general framework for forward and inverse problems involving nonlinear PDEs. Their formulation brought together:

- a neural-network representation of the unknown field;
- automatic differentiation for spatial and temporal derivatives;
- PDE residuals incorporated into the loss;
- boundary, initial, and observational data terms;
- optimization over the network parameters.

The resulting method is naturally situated within the broader area of *scientific machine learning* (SciML), which seeks to combine mathematical models, data, and learning algorithms.



The central scientific-machine-learning viewpoint is therefore not to discard physical models in favor of data, but to combine them. When observations are sparse, the equations can act as structure or regularization. When parts of the model are unknown, data can help identify them.

1.3 PDEs and boundary value problems

1.3.1 What is a PDE?

A partial differential equation relates an unknown function to its partial derivatives. A standard example is the Poisson equation

$$-\Delta u(x) = f(x), \qquad x \in \Omega \subset \mathbb{R}^d,$$

where the Laplacian is

$$\Delta u = \frac{\partial^2 u}{\partial x_1^2} + \cdots + \frac{\partial^2 u}{\partial x_d^2}.$$

The unknown is not a finite vector but a function

$$u : \Omega \to \mathbb{R}.$$

To determine it, we must specify the domain Ω, the forcing term f, and suitable boundary conditions.

For example, a Dirichlet problem for Poisson's equation is

$$\begin{aligned} -\Delta u &= f \quad \text{in } \Omega, \\ u &= g \quad \text{on } \partial\Omega. \end{aligned}$$

The boundary condition encodes information about the behavior of the field on the boundary of the physical region.

Time-dependent equations also require initial data. A typical evolution problem is

$$u_t - \Delta u = f \qquad \text{in } \Omega \times (0, T],$$

with

$$u(x, 0) = u_0(x).$$

Here, the initial condition anchors the dynamics at time $t = 0$.

1.3.2 Strong and weak formulations

The same PDE may be formulated in different mathematical ways. In a *strong formulation*, the differential equation is required to hold pointwise:

$$\mathcal{N}[u](x) = f(x), \qquad x \in \Omega.$$

This requires sufficient differentiability of u for all derivatives in $\mathcal{N}$ to make sense pointwise.

In a *weak formulation*, the equation is tested against a class of test functions. One seeks u satisfying

$$a(u, v) = \ell(v)$$

for all admissible test functions v. Weak formulations often reduce the differentiability required of the solution and are the natural basis of the finite element method.

This distinction matters for PINNs. Most introductory PINNs enforce the strong-form residual at collocation points. However, weak-form and variational neural methods also exist. The choice of formulation affects smoothness requirements, numerical stability, and the kind of derivatives that must be computed.

1.3.3 Examples: Poisson and heat equations

The Poisson equation

$$-\Delta u = f$$

models a range of steady-state phenomena, including electrostatic potential, steady heat conduction, and simplified equilibrium models in mechanics and fluid flow.

A basic time-dependent example is the heat equation

$$u_t - \alpha \Delta u = 0, \qquad x \in \Omega,$$

with an initial condition

$$u(x, 0) = u_0(x).$$

Classical numerical methods usually advance the solution through a sequence of discrete time levels. A PINN can instead represent the full space–time function $u(x, t)$ with a single network. This is one of the first conceptual differences students encounter when comparing PINNs with standard time-stepping algorithms.

1.4 Classical numerical solvers

1.4.1 Finite difference methods

Finite difference methods approximate derivatives by combinations of nearby values on a grid. For example, in one dimension,

$$u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}.$$

When applied at each interior grid point, a differential equation is converted into a finite system of algebraic equations for the nodal values.

Finite differences are particularly natural on rectangular or structured domains. Their strengths include simplicity, sparse matrix structure, and clear consistency and stability analysis. Under appropriate assumptions, decreasing the mesh spacing h improves the approximation.

1.4.2 Finite element methods

Finite element methods instead approximate the solution by a finite-dimensional expansion

$$u_h(x) = \sum_{j=1}^{N} U_j \phi_j(x),$$

where the basis functions ϕ_j are associated with a mesh. The coefficients U_j become the unknown degrees of freedom. The method is typically derived from a weak formulation of the PDE.

Finite elements are especially powerful on irregular geometries. They support local mesh refinement, unstructured meshes, high-order approximation, and rigorous a priori and a posteriori error analysis.

1.4.3 Accuracy and mathematical theory

Classical numerical analysis asks a fundamental question: how close is the computed approximation to the exact solution? A representative error estimate has the form

$$\|u - u_h\| \leq Ch^p \|u\|_{H^s(\Omega)},$$

where p depends on the approximation order and the norm reflects the regularity of the exact solution.

Such estimates provide more than a numerical value; they explain how the error changes under refinement. For many established PDE discretizations, stability, convergence, and conditioning are well understood. This theoretical maturity is one of the strongest advantages of classical methods.

1.4.4 Where classical methods can become difficult

Despite their success, there are settings in which standard numerical workflows become cumbersome or expensive. Examples include:

- very high-dimensional domains;
- repeated solves for many parameter values;
- inverse problems with unknown coefficients or source terms;
- sparse or noisy observational data;
- moving geometries that require repeated remeshing;
- problems in which simulation must be coupled with learning, estimation, or control.

Consider, for example,

$$-\nabla \cdot (k(x)\nabla u(x)) = f(x),$$

where $k(x)$ is unknown and only a small number of measurements of u are available. This is no longer a standard forward problem. We must infer an unknown coefficient from incomplete data while maintaining consistency with the governing PDE. Problems of this type motivate methods that can combine data and physics within the same optimization framework.

1.5 Approximating PDE solutions as functions

1.5.1 The solution itself is a function

A PDE solution is fundamentally a function. For a steady problem,

$$u : \Omega \to \mathbb{R},$$

whereas for a time-dependent problem,

$$u : \Omega \times [0, T] \to \mathbb{R}.$$

Classical mesh-based methods usually store finitely many coefficients or nodal values from which an approximate function is reconstructed. A neural network provides a different kind of finite-dimensional representation:

$$u(x) \approx \widehat{u}(x; \theta),$$

where θ is a finite parameter vector.

The important shift in viewpoint is that the approximation is a continuous, explicitly evaluable function. Once the network parameters are fixed, the model can be evaluated at any input point without interpolation from a mesh.

1.5.2 Parameterized approximation is not unique to neural networks

The idea of parameterized approximation is classical. One may write

$$u_m(x) = \sum_{j=1}^{m} c_j \phi_j(x),$$

where the basis functions ϕ_j may be polynomials, Fourier modes, finite element basis functions, radial basis functions, wavelets, or other functions.

In a standard linear approximation method, the basis functions are fixed in advance and only the coefficients c_j are determined. In a neural network, the features themselves depend on trainable parameters. For a one-hidden-layer network,

$$\widehat{u}(x; \theta) = \sum_{j=1}^{m} a_j \sigma(w_j^\top x + b_j).$$

Both the outer coefficients a_j and the feature parameters (w_j, b_j) are learned.

This gives a useful interpretation of neural networks for PDEs: rather than selecting all basis functions beforehand, we allow the representation to adapt during training to the combined demands of data and physics.

1.5.3 Constraints become loss terms

Suppose a PDE problem is written abstractly as

$$\mathcal{N}[u] = f \qquad \text{in } \Omega,$$

together with

$$\mathcal{B}[u] = g \qquad \text{on } \partial\Omega.$$

A PINN replaces the exact unknown u by a neural approximation $\widehat{u}(\cdot;\theta)$ and measures the violation of these conditions. A basic objective is

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathrm{PDE}}(\theta) + \lambda_b \mathcal{L}_{\mathrm{BC}}(\theta),$$

where $\lambda_b > 0$ is a penalty weight.

For strong-form collocation, choose interior points

$$\{x_i\}_{i=1}^{N_f} \subset \Omega$$

and define

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| \mathcal{N}[\widehat{u}](x_i) - f(x_i) \right|^2 .$$

Boundary conditions can be treated similarly.

Remark 1.2. PINNs are often described as mesh-free. This means that they do not require a connected mesh of elements in the usual finite-element or finite-volume sense. It does *not* mean that no spatial or space–time sampling is required. The method still depends on collocation points, and the choice of those points can strongly influence training.

1.6 What is a neural network really?

1.6.1 From linear regression to nonlinear feature learning

A useful way to demystify neural networks is to begin with linear regression. A linear model has the form

$$\hat{y} = w^\top x + b.$$

It is easy to understand and optimize, but it can only represent affine relationships in the original input coordinates.

A standard way to obtain nonlinearity is to first transform the input through features $\phi_j(x)$:

$$\hat{y} = \sum_{j=1}^{m} a_j \phi_j(x).$$

If the features are sufficiently rich, complicated functions can be approximated. Neural networks take this idea one step further by learning many of the features rather than prescribing them all in advance.

1.6.2 The single neuron

A neuron computes an affine map followed by a nonlinear activation:

$$z = w^\top x + b, \qquad a = \sigma(z).$$

The weight vector w and bias b determine an affine coordinate, while the activation function σ creates nonlinear behavior. Common activations include sigmoid, hyperbolic tangent, rectified linear unit (ReLU), and sinusoidal functions.

For PDE applications, smoothness of the activation may matter. If the PDE residual contains second or higher derivatives, the network must possess the corresponding differentiability. This is one reason that smooth activations such as tanh are frequently used in introductory PINN implementations.

1.6.3 Multilayer perceptrons

A multilayer perceptron (MLP) is obtained by composing affine maps and nonlinearities:

$$x^{(1)} = \sigma(W^{(1)}x + b^{(1)}),$$

$$x^{(2)} = \sigma(W^{(2)}x^{(1)} + b^{(2)}),$$

and so on, until the final output layer

$$\hat{y} = W^{(L)}x^{(L-1)} + b^{(L)}.$$

Depth allows complicated functions to be represented as compositions of simpler transformations. The nonlinearity is essential. If every layer were linear, then

$$W_3(W_2(W_1x)) = (W_3W_2W_1)x,$$

so many stacked layers would collapse to a single linear map.

1.6.4 Training as optimization

The network parameters are chosen by minimizing a loss function:

$$\min_{\theta} \mathcal{L}(\theta).$$

A basic gradient-descent iteration is

$$\theta_{k+1} = \theta_k - \eta\nabla_\theta\mathcal{L}(\theta_k),$$

where $\eta > 0$ is the learning rate.

In ordinary supervised learning, the loss typically measures disagreement between model predictions and labeled data. In PINNs, the loss can include data mismatch, PDE residuals, boundary conditions, initial conditions, conservation laws, or other physical constraints. This is the key transition from generic neural-network training to physics-informed learning.

1.7 The core PINN formulation

1.7.1 Neural solution ansatz

For a time-dependent PDE, a PINN uses a neural approximation

$$u(x,t) \approx \widehat{u}(x,t;\theta).$$

The network input is not an image or a categorical label. It is the physical coordinate, for example

$$(x,t) \longmapsto \widehat{u}(x,t;\theta).$$

Thus, the network can be viewed as a continuous surrogate for the physical field.

1.7.2 Automatic differentiation

Because $\widehat{u}$ is built from differentiable operations, derivatives required by the governing equation can be computed by automatic differentiation:

$$\partial_t \widehat{u}, \qquad \nabla \widehat{u}, \qquad \Delta \widehat{u}.$$

This creates the computational bridge between neural-network function approximation and differential equations. The same network provides both the field value and the derivatives entering the physical model.

1.7.3 PDE residual

For an equation

$$\mathcal{N}[u](x) = f(x),$$

define the neural residual

$$R(x;\theta) = \mathcal{N}[\widehat{u}](x) - f(x).$$

If $R(x;\theta) = 0$ everywhere and all auxiliary conditions are satisfied, then $\widehat{u}$ is an exact solution of the PDE. In practice, the residual is evaluated at finitely many collocation points and minimized in a least-squares sense.

Boundary and initial conditions are treated in the same way. For Dirichlet boundary data,

$$u = g \qquad \text{on } \partial\Omega,$$

a standard penalty is

$$\mathcal{L}_{\mathrm{BC}}(\theta) = \frac{1}{N_b} \sum_{j=1}^{N_b} \left| \widehat{u}(s_j;\theta) - g(s_j) \right|^2 .$$

For an initial condition $u(x,0) = u_0(x)$,

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \frac{1}{N_0} \sum_{j=1}^{N_0} \left| \widehat{u}(x_j,0;\theta) - u_0(x_j) \right|^2 .$$

1.7.4 Example: Poisson equation

Consider

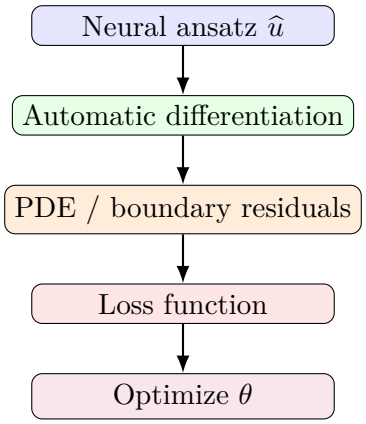
$$-\Delta u = f \qquad \text{in } \Omega, \qquad u = g \qquad \text{on } \partial\Omega.$$

A standard strong-form PINN loss is

$$\begin{aligned} \mathcal{L}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| -\Delta \widehat{u}(x_i;\theta) - f(x_i) \right|^2 \\ + \lambda_b \frac{1}{N_b} \sum_{j=1}^{N_b} \left| \widehat{u}(s_j;\theta) - g(s_j) \right|^2 . \end{aligned}$$

The Laplacian of the neural approximation is computed by automatic differentiation.

The overall workflow is therefore:



In compact language, PINNs turn PDE solving into constrained function learning.

1.8 Where PINNs can be attractive

PINNs are especially attractive when the computational task involves more than a standard forward solve. Typical situations include the following.

1.8.1 Sparse or noisy observations

Suppose only a small number of measurements of a physical field are available. A purely data-driven method may not have enough information to reconstruct the full field. If the governing PDE is known, its residual can provide additional structure between observations. In this sense, the PDE acts as a physics-based regularizer.

1.8.2 Unknown coefficients and inverse problems

If a coefficient, source term, boundary input, or material property is unknown, the unknown quantity can sometimes be represented by additional trainable parameters or an additional neural network. The optimization then attempts to fit both the observational data and the governing physics.

This is one of the settings in which PINNs can be conceptually appealing: forward solution and parameter estimation can be embedded in one differentiable computational graph.

1.8.3 Difficult meshes and repeated queries

PINNs do not require a connected computational mesh, so they may be attractive when mesh generation is cumbersome or when the geometry changes repeatedly. They also produce a continuous surrogate that can be evaluated cheaply once trained. This can be useful when the solution must be queried many times after the expensive training phase.

However, these possible advantages must always be weighed against training cost and accuracy.

1.9 Where PINNs can struggle

PINNs are not a universal replacement for established numerical solvers. Several classes of problems are known to be difficult in the basic PINN formulation.

1.9.1 High-frequency and oscillatory solutions

Standard neural networks often learn low-frequency components more easily than high-frequency components. Consequently, rapidly oscillatory solutions, wave propagation, and highly resolved Helmholtz-type problems can be challenging.

1.9.2 Shocks and sharp layers

Strong-form PINNs based on smooth neural networks may have difficulty representing shocks, discontinuities, or very thin boundary and internal layers. The optimizer can settle on an overly smooth approximation that reduces the average residual without resolving the sharp feature accurately.

1.9.3 Multiscale and stiff problems

When several widely separated scales are present, the different parts of the solution may be learned at very different rates. Stiff systems and long-time integration can also lead to severe optimization difficulties.

1.9.4 Loss balancing and optimization instability

A typical PINN objective contains several terms:

$$\mathcal{L} = \mathcal{L}_{\mathrm{PDE}} + \lambda_b \mathcal{L}_{\mathrm{BC}} + \lambda_0 \mathcal{L}_{\mathrm{IC}} + \lambda_d \mathcal{L}_{\mathrm{data}}.$$

The magnitudes and gradients of these terms can differ greatly. Poor weighting can cause the optimizer to satisfy one constraint while neglecting another. Thus, even when the neural network has enough expressive power, the optimization problem may be the real bottleneck.

Remark 1.3. For a standard forward PDE problem on a moderate-dimensional physical domain, a well-designed FEM, FDM, FVM, or spectral solver may be considerably more accurate and efficient than a PINN. The appropriate question is therefore not "Are PINNs better than classical solvers?" but rather "For which problem structures does a physics-informed learning formulation provide a useful advantage?"

1.10 Scientific reproducibility

PINN results can depend strongly on implementation choices. Important sources of variability include:

- random initialization of the neural network;
- the number and distribution of collocation points;

- optimizer choice and learning-rate schedule;
- network depth and width;
- activation functions;
- normalization and nondimensionalization;
- penalty weights in the loss;
- stopping criteria and training budget.

For this reason, reproducibility is an essential part of scientifically credible PINN work. At minimum, studies should report the architecture, activation functions, random seeds when applicable, training-point generation procedure, optimizer settings, learning rates, loss weights, normalization, stopping criteria, error metrics, and benchmark details. Code and data should be made available whenever possible.

The need for reproducibility also reinforces the importance of fair comparison with classical methods. A PINN should not be judged only by whether it produces a visually plausible field. It should be compared using quantitative errors, computational cost, robustness across random seeds, and clearly defined test problems.

1.11 Takeaway

The historical development of PINNs shows that they are best understood as a convergence of several older ideas rather than as a completely isolated invention. Numerical analysts have long approximated unknown PDE solutions in finite-dimensional spaces. Approximation theorists have long studied expressive families of functions. Optimization has long been used to determine unknown coefficients. What modern PINNs add is a particularly flexible neural parameterization together with automatic differentiation and machine-learning optimization infrastructure.

The central PINN idea can be summarized in one sentence:

> Represent the unknown physical field by a neural network and train the network so that its predictions satisfy observations and the governing differential equations as closely as possible.

This idea is simple, but its practical behavior can be complicated. PINNs are promising when data, physics, and inference need to be combined in one framework, especially for inverse problems or sparse-data settings. At the same time, optimization, sampling, accuracy, multiscale behavior, and reproducibility remain major challenges.

These observations motivate the rest of the course. The next lectures will demystify neural networks, introduce automatic differentiation, derive PINN loss functions in detail, study time-dependent and nonlinear PDEs, and examine both the practical strengths and the limitations of physics-informed learning.

Bibliography

[1] H. Lee and I. S. Kang, "Neural algorithms for differential equations," *Journal of Computational Physics*, 1990.

[2] M. W. M. G. Dissanayake and N. Phan-Thien, "Neural-network approximation for differential equations," *Communications in Numerical Methods in Engineering*, 1994.

[3] I. E. Lagaris, A. Likas, and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," *IEEE Transactions on Neural Networks*, 1998.

[4] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 2019.

Chapter 2

Demystifying Neural Networks

2.1 Overview and learning goals

The purpose of this lecture is to remove some of the mystery surrounding neural networks and to present them as familiar mathematical objects. A neural network is a parameterized map

$$f_\theta : \mathbb{R}^d \to \mathbb{R}^m,$$

where the parameter vector θ collects all weights and biases. Training consists of choosing θ so that f_θ approximately solves a prescribed approximation problem. In standard supervised learning, the objective is to match observed data. In physics-informed neural networks (PINNs), the objective also contains differential-equation residuals, boundary conditions, and initial conditions.

The main goals are to understand:

- a single neuron as an affine map followed by a nonlinearity;
- how neurons are assembled into multilayer perceptrons (MLPs);
- the role of activation functions such as ReLU and tanh;
- how loss functions convert approximation into optimization;
- gradient descent, stochastic gradient descent (SGD), momentum, and Adam;
- backpropagation as an efficient implementation of the chain rule;
- why these ideas are the computational foundation of PINNs.

2.2 Historical background

The idea of representing complicated input–output relations by networks of simple computational units is much older than modern deep learning. McCulloch and Pitts proposed an early mathematical model of simplified neural computation in 1943 [1]. Rosenblatt's perceptron subsequently introduced a trainable linear-threshold model and helped establish the idea that weights could be adjusted from data [2].

A major difficulty with early neural networks was training multiple layers. Once a model is formed by composing many functions, computing the derivative of a final loss with respect to

parameters in early layers requires repeated applications of the chain rule. The backpropagation algorithm organizes these calculations efficiently. Its role in training multilayer networks was popularized by Rumelhart, Hinton, and Williams [3].

The modern success of deep learning was enabled by several developments acting together: large datasets, faster hardware, improved initialization, effective activation functions, scalable optimization algorithms, and automatic differentiation software. In scientific machine learning, the same ingredients make it possible to train neural networks as flexible approximations of physical fields.

From the viewpoint of PDEs, one particularly important conceptual step is to think of the neural network simply as a function approximation space. Universal approximation results such as those of Cybenko [4] and Hornik [5] helped establish that sufficiently wide feedforward networks can approximate broad classes of functions. Such theorems concern approximation capability, however; they do not guarantee that a practical optimizer will find the best parameters. This distinction between *representation* and *optimization* is central throughout the course.

2.3 A single neuron

2.3.1 Affine map plus nonlinearity

The basic computational unit of a feedforward neural network is a neuron. Given an input $x \in \mathbb{R}^d$, a neuron first forms an affine combination

$$z = w^\top x + b,$$

where $w \in \mathbb{R}^d$ is a weight vector and $b \in \mathbb{R}$ is a bias. The scalar z is called the *pre-activation*. A nonlinear activation function σ is then applied:

$$a = \sigma(z) = \sigma(w^\top x + b).$$

The affine part is exactly the same mathematical object used in linear regression. Geometrically, the equation

$$w^\top x + b = 0$$

defines a hyperplane. The vector w is normal to that hyperplane and the bias b shifts it. One can therefore think of a neuron as a soft detector of whether an input lies in a certain geometric region.

The bias is important because it allows the activation threshold to move away from the origin. Without a bias, every affine map is forced to pass through the origin, which unnecessarily restricts the model.

2.3.2 Why nonlinearity is essential

If every activation is the identity map, then any composition of layers remains affine. For example,

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

Thus depth alone does not create a richer function class. The nonlinear activation is what prevents a deep network from collapsing to a single affine transformation.

A neuron may therefore be viewed as producing a learned nonlinear feature

$$x \mapsto \sigma(w^\top x + b).$$

A layer contains many such features evaluated in parallel.

2.4 From linear regression to multilayer perceptrons

2.4.1 Linear and nonlinear regression

Linear regression has the form

$$\widehat{y} = w^\top x + b.$$

It is simple and interpretable, and many associated optimization problems are convex. Its limitation is that the dependence on x is linear.

A classical nonlinear regression model often uses prescribed basis functions,

$$\widehat{y}(x) = \sum_{j=1}^{m} a_j \phi_j(x).$$

A one-hidden-layer neural network has a similar form,

$$\widehat{y}(x) = \sum_{j=1}^{m} a_j \sigma(w_j^\top x + b_j) + c,$$

but now the features themselves are learned because the parameters w_j and b_j are trainable. This is one of the simplest ways to see why neural networks are flexible function approximators.

2.4.2 Layer notation

For a multilayer perceptron, let $x^{(0)} = x$. Layer ℓ computes

$$z^{(\ell)} = W^{(\ell)} x^{(\ell-1)} + b^{(\ell)}, \qquad x^{(\ell)} = \sigma\left(z^{(\ell)}\right).$$

If layer $\ell - 1$ has width $n_{\ell-1}$ and layer ℓ has width n_ℓ, then

$$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}, \qquad b^{(\ell)} \in \mathbb{R}^{n_\ell}.$$

The activation is applied componentwise.

An L-layer network can be written as the composition

$$f_\theta = f_L \circ f_{L-1} \circ \cdots \circ f_1.$$

The forward pass is simply the evaluation of this composition from input to output.

2.4.3 Depth and width

Two basic architecture choices are *width*, the number of neurons in a hidden layer, and *depth*, the number of hidden layers. Width provides more learned features in parallel. Depth allows these features to be repeatedly transformed and combined.

For PDE problems the output layer is usually a regression layer. A scalar PDE solution might be approximated by

$$u(x,t) \approx u_\theta(x,t),$$

while a coupled system may use several output neurons representing several physical fields.

It is helpful to formalize the approximation viewpoint by defining the neural-network class

$$\mathcal{F} = \{f_\theta : \theta \in \Theta\}.$$

Training chooses a particular member of this class by solving

$$\min_{\theta\in\Theta} \mathcal{L}(\theta).$$

The design of $\mathcal{L}$ determines what the network is asked to learn.

2.5 Activation functions

Activation functions determine the nonlinear behaviour and regularity of the network. They influence approximation power, gradient propagation, and the smoothness of the resulting function. These issues are especially important for PINNs because the PDE loss may involve derivatives of the network with respect to its inputs.

2.5.1 ReLU

The rectified linear unit is

$$\mathrm{ReLU}(z) = \max(0, z).$$

It is inexpensive and is widely used in conventional deep learning. Its derivative is

$$\mathrm{ReLU}'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0, \end{cases}$$

with the derivative at $z = 0$ treated by a convention in software implementations.

The function is piecewise linear and therefore not twice differentiable. For PDEs containing second derivatives, this can be a disadvantage: a ReLU network is piecewise linear as a function of its inputs, so second derivatives vanish almost everywhere and are undefined at the kinks.

2.5.2 Hyperbolic tangent

The hyperbolic tangent is

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

It is smooth, bounded between -1 and 1, and centered around zero. Its derivative is

$$\frac{\mathrm{d}}{\mathrm{d}z}\tanh(z) = 1 - \tanh^2(z).$$

Because tanh is smooth, it has been a common choice in many PINN implementations. The main drawback is saturation: when $|z|$ is large, $\tanh(z)$ is close to ± 1 and its derivative becomes small.

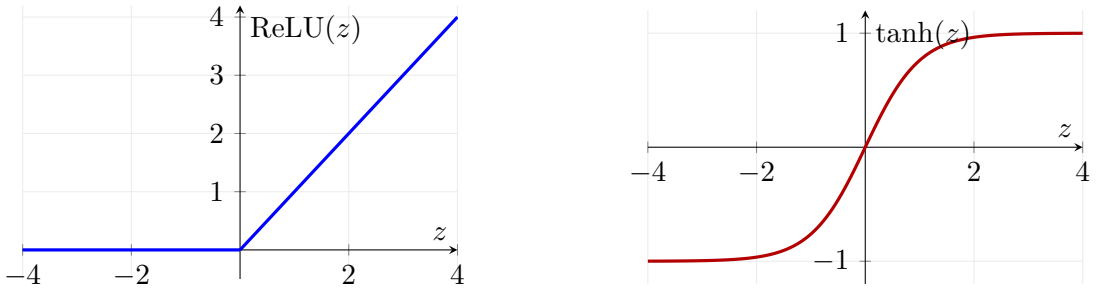


Figure 2.1: Two important activation functions. ReLU is piecewise linear, whereas tanh is smooth and saturates at ± 1.

2.5.3 Other activations

The logistic sigmoid,

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

is smooth and maps $\mathbb{R}$ into $(0, 1)$. It is important historically and remains useful for probabilistic outputs. Like tanh, however, it saturates for large $|z|$.

Another smooth choice is softplus,

$$\text{softplus}(z) = \log(1 + e^{z}),$$

whose derivative is the sigmoid. Sine activations are also used in scientific machine learning because they naturally represent oscillatory functions. For PINNs, the activation choice should be made with the required PDE derivatives in mind.

2.6 Loss functions and empirical risk

Suppose a supervised dataset is given by

$$\{(x_i, y_i)\}_{i=1}^{N}.$$

The network predicts $\widehat{y}_i = f_\theta(x_i)$. A loss function measures the discrepancy between predictions and targets.

For regression, the mean squared error is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} |f_\theta(x_i) - y_i|^2 .$$

This is an example of empirical risk minimization,

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i), y_i).$$

The sample points determine where the model is tested during training, while the loss function determines which types of error are penalized.

This viewpoint transfers directly to PINNs. Instead of only data mismatch, one may include a PDE residual loss, boundary loss, initial-condition loss, and possibly observation loss. Each term is usually estimated using finite sets of sample points.

2.7 Gradient descent

The network parameters are chosen by numerical optimization. In full-batch gradient descent,

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}(\theta_k),$$

where $\eta > 0$ is the learning rate.

The geometric interpretation is simple. Think of $\mathcal{L}(\theta)$ as a terrain whose height is the loss value. The gradient points in the direction of steepest local increase, so the negative gradient points locally downhill. Since the gradient is a local quantity, it does not reveal the global structure of a non-convex loss landscape.

The learning rate controls how far the optimizer moves in each iteration. If it is too small, progress can be extremely slow. If it is too large, the iterates may oscillate or diverge. Learning-rate schedules reduce the step size as training proceeds.

2.8 Stochastic gradient descent

2.8.1 Why stochastic gradients?

Suppose the objective is a finite sum

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell_i(\theta).$$

The exact gradient is

$$\nabla \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla \ell_i(\theta).$$

When N is very large, evaluating every term at every iteration can be expensive. Stochastic-gradient methods replace the full gradient by an estimate computed from only part of the data. The roots of stochastic approximation go back to Robbins and Monro [6]; modern large-scale neural-network optimization is strongly based on this idea [9].

At iteration k, choose a mini-batch $B_k \subset \{1, \ldots, N\}$. Define

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla \ell_i(\theta_k).$$

The SGD update is

$$\theta_{k+1} = \theta_k - \eta_k g_k.$$

If the mini-batch is sampled uniformly, then under the usual sampling assumptions g_k is an unbiased estimate of the full gradient:

$$\mathbb{E}[g_k \mid \theta_k] = \nabla \mathcal{L}(\theta_k).$$

The price paid for lower computational cost is variance: different mini-batches produce different gradient directions.

2.8.2 Mini-batches, epochs, and shuffling

Three terms are useful in practice:

- **Batch size**: the number $|B_k|$ of samples used in one update.
- **Iteration**: one parameter update.
- **Epoch**: enough iterations that the algorithm has processed approximately one full pass through the training set.

A common implementation shuffles the training data at the start of each epoch and splits the shuffled indices into mini-batches. Small mini-batches provide inexpensive but noisy gradients. Larger mini-batches reduce the noise but increase the cost of each step and can require different learning-rate choices.

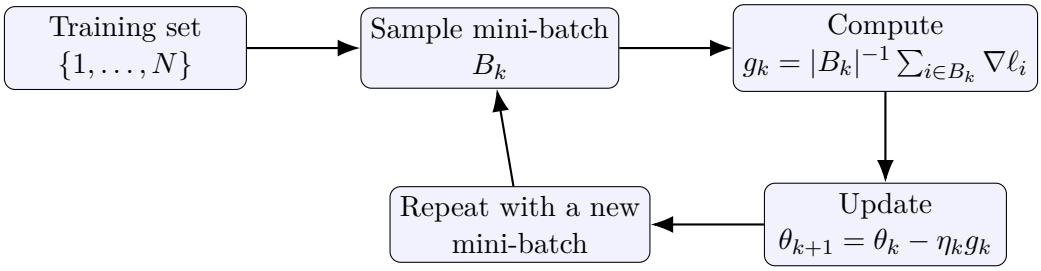


Figure 2.2: Schematic of mini-batch stochastic gradient descent.

2.8.3 Geometric intuition for noisy updates

Because the mini-batch gradient is only an estimate, SGD does not follow the smooth path of full-batch gradient descent. Its trajectory fluctuates around the underlying descent direction. This is not merely a defect. The noise can help the iterate move through flat regions or avoid becoming trapped in some shallow structures of the loss landscape. On the other hand, too much noise can make training unstable.

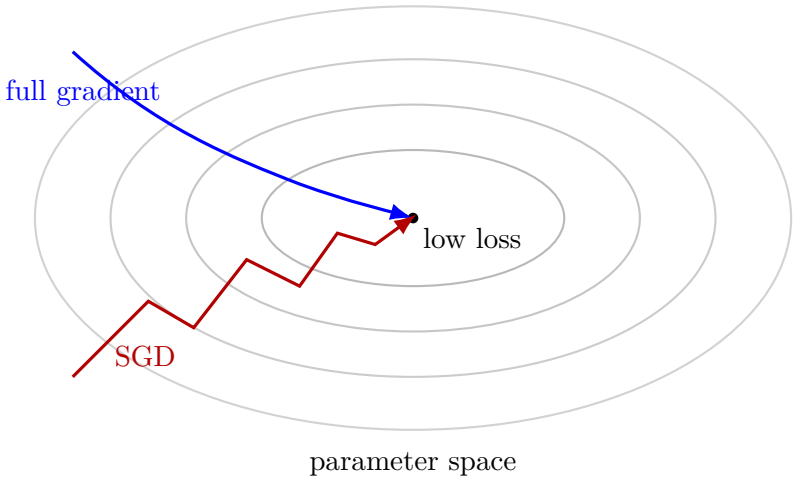


Figure 2.3: A schematic comparison of a smooth full-gradient path and a noisy SGD path. The drawing is conceptual rather than a literal optimizer trace.

2.8.4 SGD pseudocode

A simple mini-batch SGD procedure is:

1. Choose initial parameters θ_0 and a learning-rate schedule η_k.
2. For $k = 0, 1, 2, \ldots$:
   (a) sample a mini-batch B_k;
   (b) evaluate the network and the mini-batch loss;
   (c) use backpropagation to compute g_k;
   (d) update $\theta_{k+1} = \theta_k - \eta_k g_k$.

For PINNs, mini-batches may contain interior collocation points, boundary points, initial-condition points, and/or observation data. One can sample the different categories separately because they often correspond to distinct terms in the loss.

2.9 Momentum

Plain SGD can oscillate when the loss landscape is narrow in one direction and shallow in another. Momentum smooths the search direction by accumulating a running average of gradients. A simple form is

$$v_{k+1} = \beta v_k + g_k, \qquad \theta_{k+1} = \theta_k - \eta v_{k+1},$$

where $\beta \in [0, 1)$ controls how strongly past gradients influence the current step.

If gradients repeatedly point in a similar direction, momentum accelerates movement along that direction. If a gradient component alternates in sign from step to step, the averaging tends to damp that oscillation. This is the intuition behind many modern optimizers.

2.10 The Adam algorithm

Adam combines momentum-like averaging with adaptive scaling of each parameter [7]. Let

$$g_k = \nabla_\theta \mathcal{L}(\theta_k)$$

denote the current stochastic gradient. Adam maintains two exponential moving averages:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k,$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2)(g_k \odot g_k),$$

where $\odot$ denotes componentwise multiplication.

The first quantity m_k tracks the mean gradient direction. The second quantity v_k tracks the scale of squared gradients. Since both moving averages start at zero, they are biased toward zero during the first iterations. Adam therefore uses bias-corrected estimates

$$\widehat{m}_k = \frac{m_k}{1 - \beta_1^k}, \qquad \widehat{v}_k = \frac{v_k}{1 - \beta_2^k}.$$

The parameter update is

$$\theta_{k+1} = \theta_k - \eta \frac{\widehat{m}_k}{\sqrt{\widehat{v}_k} + \varepsilon},$$

with all division and square roots applied componentwise.

Typical default values are

$$\beta_1 = 0.9, \qquad \beta_2 = 0.999, \qquad \varepsilon = 10^{-8}.$$

These defaults are useful starting points, but they are not universally optimal. Adam is popular because it often makes rapid initial progress without extensive tuning. In PINN practice, Adam is frequently used for an initial phase of training, sometimes followed by a deterministic quasi-Newton method such as L-BFGS for further refinement. The optimizer cannot, however, repair every problem: poor sampling, badly scaled loss terms, unsuitable activations, or an inadequate network representation can still dominate the error.

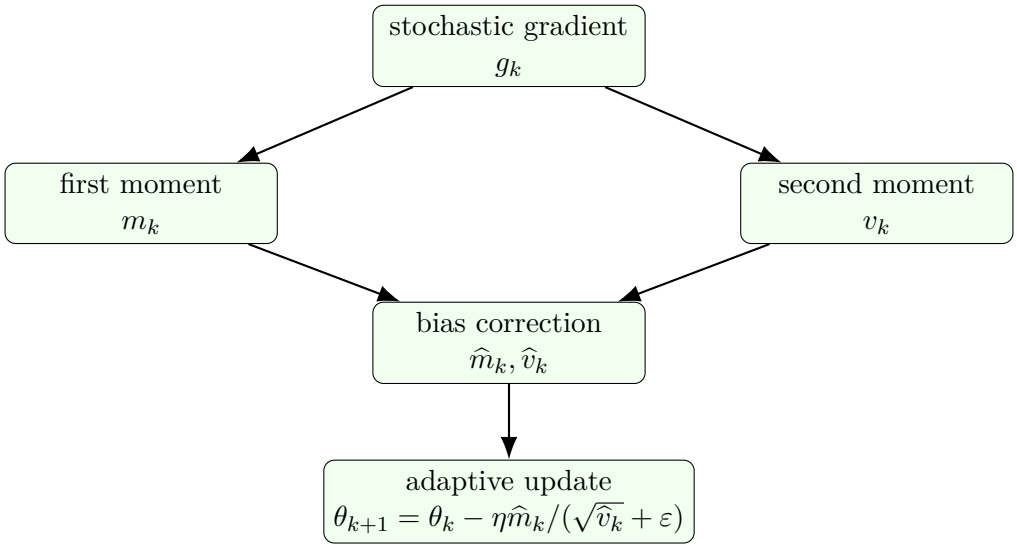


Figure 2.4: Conceptual structure of Adam: momentum-like first moments plus adaptive second-moment scaling.

2.11 Backpropagation

2.11.1 Computational graphs and the chain rule

A neural network can be represented as a computational graph. Intermediate quantities such as pre-activations and activations are nodes, and elementary operations form the edges. For a simple composition

$$y = f(g(x)),$$

the chain rule gives

$$\frac{\mathrm{d}y}{\mathrm{d}x} = f'(g(x))g'(x).$$

A deep network applies this idea repeatedly through many layers.

Backpropagation is an efficient reverse-mode procedure for evaluating these derivatives. The forward pass computes and stores intermediate values. The backward pass propagates sensitivities from the scalar loss back toward earlier layers.

2.11.2 One-layer derivation

Consider

$$z = Wx + b, \qquad a = \sigma(z), \qquad \mathcal{L} = \mathcal{L}(a).$$

Define the sensitivity

$$\delta = \frac{\partial \mathcal{L}}{\partial z}.$$

Then the gradients are

$$\frac{\partial \mathcal{L}}{\partial W} = \delta x^\top, \qquad \frac{\partial \mathcal{L}}{\partial b} = \delta, \qquad \frac{\partial \mathcal{L}}{\partial x} = W^\top \delta.$$

For a multilayer network, the sensitivities satisfy the recurrence

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^\top \delta^{(\ell+1)}\right) \odot \sigma'(z^{(\ell)}),$$

and therefore

$$\nabla_{W^{(\ell)}} \mathcal{L} = \delta^{(\ell)} (x^{(\ell-1)})^\top, \qquad \nabla_{b^{(\ell)}} \mathcal{L} = \delta^{(\ell)}.$$

The important conceptual point is that backpropagation reuses local derivative information instead of recomputing the same chain-rule products independently for every parameter.

2.12 Automatic differentiation and the link to PINNs

Backpropagation can be viewed as a special case of reverse-mode automatic differentiation. Modern machine-learning systems build a computational graph while evaluating the forward pass and then differentiate that graph automatically.

For ordinary supervised learning, the main goal is to compute derivatives of a scalar loss with respect to many model parameters. PINNs additionally differentiate the network output with respect to its *inputs*. For example, if $u_\theta(x, t)$ approximates the solution of a PDE, automatic differentiation can provide

$$\partial_t u_\theta, \qquad \partial_x u_\theta, \qquad \partial_{xx} u_\theta.$$

These derivatives are inserted into the PDE residual. Training then requires differentiating the resulting residual loss with respect to the parameters θ. This nested use of differentiation is the computational bridge between neural networks and differential equations.

2.13 Training diagnostics

A single scalar training loss is rarely enough to diagnose a neural-network calculation. Useful quantities to monitor include:

- the total loss as a function of iteration or epoch;
- the separate components of the loss;
- norms of parameter gradients;
- validation or test error when independent data are available;

- for PINNs, the PDE residual, boundary-condition error, initial-condition error, and any data mismatch separately.

If the loss is nearly constant, the learning rate may be too small, the gradients may be vanishing, or the problem may be poorly scaled. If the loss oscillates or explodes, the learning rate may be too large or the gradients may be unstable. If one loss component decreases while another remains large, the weighting of the objectives may require attention.

These diagnostics are especially important for PINNs because their optimization problem combines several objectives with potentially very different physical units and gradient magnitudes.

2.14 Summary

The essential ideas of the lecture can be summarized as follows.

1. A neuron is an affine map followed by a nonlinear activation.
2. An MLP is a composition of layers and therefore a parameterized nonlinear function.
3. Activation functions determine both nonlinear representation and smoothness properties.
4. Training means minimizing a loss over all network parameters.
5. Full-batch gradient descent uses the exact empirical gradient; SGD uses a mini-batch estimate.
6. Stochasticity lowers the cost per update but introduces gradient noise.
7. Momentum averages gradient directions, while Adam combines first-moment averaging with second-moment adaptive scaling.
8. Backpropagation efficiently applies the chain rule through the computational graph.
9. Automatic differentiation extends these ideas and makes it possible to compute PDE derivatives of neural-network approximations.

These ingredients—function approximation, differentiation, and optimization—form the basic computational toolkit required for physics-informed neural networks.

Bibliography

[1] W. S. McCulloch and W. Pitts. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5:115–133, 1943.

[2] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[4] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.

[5] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[6] H. Robbins and S. Monro. A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[7] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015.

[8] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[9] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.

[10] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Chapter 3

Automatic Differentiation for Physics

3.1 Introduction and learning goals

Physics-informed neural networks (PINNs) sit at the intersection of numerical analysis, differential equations, optimization, and machine learning. One of the main technical ideas that makes modern PINNs practical is *automatic differentiation* (AD). AD allows us to differentiate a computer program accurately and efficiently, without manually deriving long symbolic expressions and without introducing the truncation error of finite-difference formulas.

In a PINN, differentiation is required in two conceptually distinct places. First, a differential equation contains derivatives of the unknown field with respect to the physical coordinates. For example, if a neural network $u_\theta(x,t)$ is used to approximate a solution of a PDE, then the PDE may require quantities such as

$$\partial_t u_\theta, \qquad \partial_x u_\theta, \qquad \partial_{xx} u_\theta, \qquad \nabla u_\theta, \qquad \Delta u_\theta.$$

Second, training the neural network requires derivatives of a loss function with respect to the neural-network parameters,

$$\nabla_\theta \mathcal{L}(\theta).$$

Thus, PINNs require differentiation with respect to both *inputs* and *parameters*. AD provides a unified computational mechanism for both tasks.

The main goals of this lecture are to:

- distinguish symbolic, numerical, and automatic differentiation;
- understand computation graphs and the chain rule;
- compare forward-mode and reverse-mode automatic differentiation;
- explain the connection between reverse-mode AD and backpropagation;
- compute first- and second-order derivatives needed in PDE residuals;
- illustrate AD on Poisson, heat, Burgers, and inverse-coefficient problems;
- understand practical issues involving memory, smoothness, and higher derivatives.

3.2 Why derivatives matter in scientific machine learning

3.2.1 Training derivatives and physics derivatives

Suppose a neural network u_θ is used as a parameterized approximation of an unknown solution. There are two different derivative calculations:

$$\begin{aligned} &\text{training derivatives:} && \nabla_\theta \mathcal{L}(\theta), \\ &\text{physics derivatives:} && \partial_t u_\theta,\ \nabla u_\theta,\ \Delta u_\theta, \dots. \end{aligned}$$

The first tells an optimizer how to modify the network parameters. The second tells us whether the network output satisfies the governing differential equation.

For example, consider the one-dimensional heat equation

$$u_t - \alpha u_{xx} = 0.$$

If we approximate u by u_θ, then the PDE residual is

$$\mathcal{R}(x, t; \theta) = \partial_t u_\theta(x, t) - \alpha \partial_{xx} u_\theta(x, t).$$

At interior collocation points (x_i, t_i), a common loss is

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| \mathcal{R}(x_i, t_i; \theta) \right|^2.$$

Training then needs

$$\nabla_\theta \mathcal{L}_{\text{PDE}}(\theta).$$

Consequently, the computation differentiates u_θ with respect to (x, t) to form the residual, and differentiates the residual loss with respect to θ to train the network.

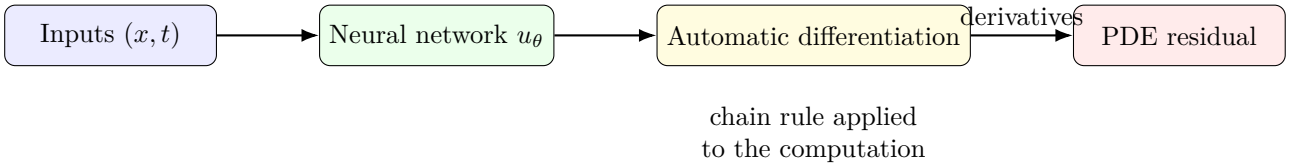


Figure 3.1: A conceptual view of the role of automatic differentiation in a PINN.

3.3 Three approaches to differentiation

3.3.1 Symbolic differentiation

Symbolic differentiation manipulates algebraic expressions. If

$$f(x) = \sin(x^2),$$

then symbolic calculus produces

$$f'(x) = 2x \cos(x^2).$$

The output is a new symbolic formula. This is ideal for short mathematical expressions and for hand calculations. However, large neural networks are deeply nested compositions of affine

transformations and nonlinear activations. Expanding their derivatives symbolically causes *expression swell*: intermediate formulas become very large even when the final numerical derivative could be evaluated efficiently.

For example, consider a simple nested scalar model

$$f(x) = \sigma(a_3\sigma(a_2\sigma(a_1x + b_1) + b_2) + b_3).$$

Writing the full derivative explicitly is possible, but it quickly becomes cumbersome. In a modern network, there may be millions of operations. What we really want is not a giant formula but an efficient numerical evaluation of the derivative at the current input and current parameter values.

3.3.2 Numerical differentiation

Numerical differentiation approximates derivatives from nearby function values. The forward-difference formula is

$$f'(x) \approx \frac{f(x+h) - f(x)}{h},$$

whereas the central-difference formula is

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

These formulas are simple and remain extremely useful in numerical analysis. Nevertheless, the choice of h involves a classical trade-off. If h is too large, truncation error dominates. If h is too small, subtractive cancellation and floating-point round-off become significant.

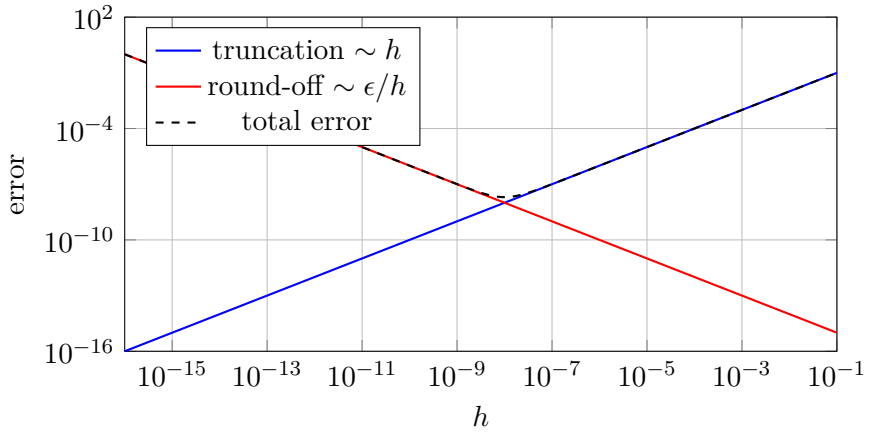


Figure 3.2: Qualitative finite-difference error balance. Decreasing h initially reduces truncation error, but eventually round-off error dominates.

For PINNs, finite differences have additional disadvantages. A separate perturbation may be needed in every coordinate direction, repeated network evaluations are required, and second derivatives need nested difference formulas. These costs become inconvenient when a PDE residual must be evaluated at thousands of collocation points during every optimization step.

3.3.3 Automatic differentiation

Automatic differentiation takes a different viewpoint. A computer program is decomposed into elementary operations such as addition, multiplication, sine, exponential functions, matrix

multiplication, and activation functions. The derivative of each elementary operation is known. The chain rule then combines these local derivative rules in the same order as the computation.

AD therefore differentiates the *program trace*. It does not approximate derivatives by perturbing inputs, and it need not create a giant symbolic expression. Up to floating-point arithmetic, it evaluates the exact derivative of the implemented program.

Table 3.1 summarizes the three approaches.

	Symbolic	Numerical	Automatic
Object	Formula	Function values	Program trace
Accuracy	Exact formula	Approximate	Machine precision
Main cost issue	Expression swell	Repeated evaluations	Mode and memory dependent
Typical use	Closed forms	Gradient checks, classical discretizations	Neural networks and PDE residuals
Main limitation	Formula complexity	Step-size sensitivity	Memory and implementation details

Table 3.1: Comparison of symbolic, numerical, and automatic differentiation.

3.4 Computational graphs and the chain rule

3.4.1 A simple graph

Consider

$$f(x) = (x^2 + 1)\sin x.$$

We can write the computation as

$$a = x^2, \qquad b = a + 1, \qquad c = \sin x, \qquad f = bc.$$

This creates a directed acyclic computational graph.

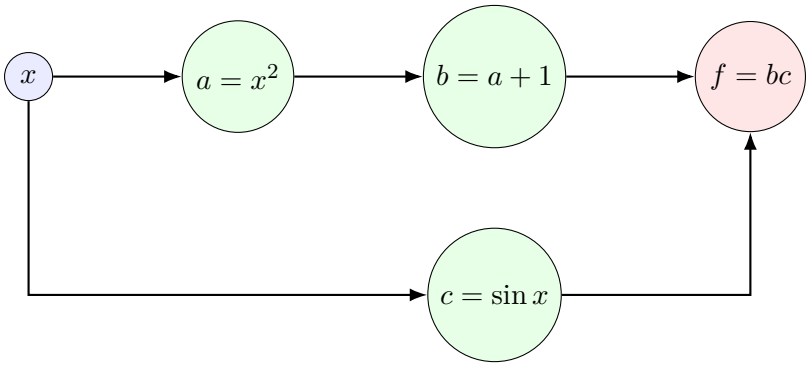


Figure 3.3: Computational graph for $f(x) = (x^2 + 1)\sin x$.

The forward evaluation stores both values and enough information to apply local derivatives. The relevant local rules are

$$\frac{\partial a}{\partial x} = 2x, \qquad \frac{\partial b}{\partial a} = 1, \qquad \frac{\partial c}{\partial x} = \cos x,$$

and

$$\frac{\partial f}{\partial b} = c, \qquad \frac{\partial f}{\partial c} = b.$$

There are two directed paths from x to f. Applying the chain rule along each path and summing gives

$$\begin{aligned}\frac{\mathrm{d}f}{\mathrm{d}x} &= \frac{\partial f}{\partial b}\frac{\partial b}{\partial a}\frac{\partial a}{\partial x} + \frac{\partial f}{\partial c}\frac{\partial c}{\partial x} \\ &= c(1)(2x) + b\cos x \\ &= 2x\sin x + (x^2+1)\cos x.\end{aligned}$$

The important point is that AD obtains this value using only local derivative rules and graph connectivity.

3.4.2 Elementary derivative rules

An AD system contains derivative rules for elementary operations. Examples include

$$\begin{aligned} z = x + y: &\quad z_x = 1, \quad z_y = 1, \\ z = xy: &\quad z_x = y, \quad z_y = x, \\ z = x/y: &\quad z_x = 1/y, \quad z_y = -x/y^2, \\ z = \sin x: &\quad z_x = \cos x, \\ z = \exp x: &\quad z_x = \exp x, \\ z = \sigma(x): &\quad z_x = \sigma'(x). \end{aligned}$$

Matrix operations are treated similarly, using the corresponding vector- or matrix-valued derivative rules.

3.4.3 Neural networks as large computation graphs

A one-hidden-layer network can be written as

$$z = Wx + b, \qquad a = \sigma(z), \qquad \widehat{y} = v^T a + c.$$

Figure 3.4 shows the corresponding graph.

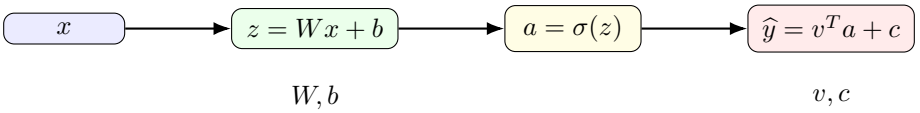


Figure 3.4: A one-layer neural network as a computation graph.

Backpropagation is reverse-mode automatic differentiation applied to a network and a scalar loss.

3.5 Forward-mode automatic differentiation

3.5.1 Tangent propagation

Forward mode augments every intermediate variable v with a tangent value

$$\dot{v} = \frac{\mathrm{d}v}{\mathrm{d}s},$$

where s is the chosen input direction. Each elementary operation propagates both its ordinary value and its tangent.

For multiplication $z = xy$,

$$\dot{z} = \dot{x}\, y + x\, \dot{y}.$$

For $z = \sin x$,

$$\dot{z} = \cos(x)\dot{x}.$$

Thus one forward pass computes a directional derivative through the entire program.

For our example $f(x) = (x^2 + 1)\sin x$, set $\dot{x} = 1$. Then

$$\begin{aligned} a &= x^2, & \dot{a} &= 2x, \\ b &= a + 1, & \dot{b} &= \dot{a}, \\ c &= \sin x, & \dot{c} &= \cos x, \\ f &= bc, & \dot{f} &= \dot{b}\, c + b\, \dot{c}. \end{aligned}$$

Hence

$$\dot{f} = 2x \sin x + (x^2 + 1)\cos x = f'(x).$$

3.5.2 Dual-number interpretation

Forward mode can be understood elegantly using dual numbers. Introduce a formal symbol ε satisfying $\varepsilon^2 = 0$. Then

$$x + \varepsilon\dot{x}$$

contains both the value x and the tangent $\dot{x}$. For a smooth scalar function,

$$f(x + \varepsilon\dot{x}) = f(x) + \varepsilon f'(x)\dot{x},$$

because all terms containing ε^2 or higher vanish. The coefficient of ε is therefore the directional derivative.

This perspective explains why forward-mode AD is exact for the sequence of elementary operations: it is not using a limiting process or a small perturbation parameter.

3.6 Reverse-mode automatic differentiation and backpropagation

3.6.1 Adjoint propagation

Reverse mode starts from an output y and propagates sensitivities backward. For each intermediate variable v, define the adjoint

$$\bar{v} = \frac{\partial y}{\partial v}.$$

Suppose $z = xy$. If $\bar{z}$ is known, then the chain rule gives the contributions

$$\bar{x} \mathrel{+}= \bar{z}\, y, \qquad \bar{y} \mathrel{+}= \bar{z}\, x.$$

If $z = \sin x$, then

$$\bar{x} \mathrel{+}= \bar{z} \cos x.$$

For $f = (x^2 + 1)\sin x$, initialize $\bar{f} = 1$. The reverse sweep yields

$$\bar{b} \mathrel{+}= \bar{f}c, \qquad \bar{c} \mathrel{+}= \bar{f}b,$$

then

$$\bar{a} \mathrel{+}= \bar{b},$$

and finally

$$\bar{x} \mathrel{+}= \bar{a}(2x) + \bar{c}\cos x.$$

Therefore

$$\bar{x} = 2x\sin x + (x^2 + 1)\cos x.$$

3.6.2 Why reverse mode dominates neural-network training

Suppose

$$F : \mathbb{R}^n \to \mathbb{R}^m.$$

Forward mode naturally propagates one input direction at a time, while reverse mode naturally propagates one output sensitivity at a time. Consequently:

- forward mode is attractive when n is small relative to m;
- reverse mode is attractive when m is small relative to n.

A neural-network loss usually has millions of parameters and one scalar output,

$$\mathcal{L} : \mathbb{R}^p \to \mathbb{R}.$$

This is exactly the situation in which reverse mode is effective. Backpropagation is therefore not a separate differentiation principle; it is the specialization of reverse-mode AD to layered neural-network computations.

A useful cost principle is that the reverse-mode gradient of a scalar output can often be computed at a cost comparable to a small constant multiple of the original forward evaluation. The trade-off is memory: reverse mode usually needs access to intermediate quantities from the forward pass.

3.6.3 Forward versus reverse mode

Task	Preferred mode	Reason
One directional derivative	Forward	One tangent pass
Gradient of scalar loss	Reverse	One backward pass
Full Jacobian when $m \ll n$	Reverse	Efficient row accumulation
Full Jacobian when $n \ll m$	Forward	Efficient column accumulation
Hessian-vector product	Mixed	Often forward-over-reverse
Low-dimensional PDE input derivative	Either	Framework and residual structure matter

Table 3.2: Typical mode choices in automatic differentiation.

3.7 Jacobians, gradients, Hessians, and PDE derivatives

3.7.1 Jacobians

For

$$F(x) = \begin{pmatrix} F_1(x) \\ \vdots \\ F_m(x) \end{pmatrix}, \qquad x \in \mathbb{R}^n,$$

the Jacobian is

$$J_F(x) = \left(\frac{\partial F_i}{\partial x_j} \right)_{i=1,\dots,m}^{j=1,\dots,n}.$$

Jacobians are needed for vector-valued fields and systems of PDEs. For example, if a network predicts a velocity field $\boldsymbol{u} = (u_1, u_2)$, then the velocity gradient contains all first derivatives $\partial_{x_j} u_i$.

3.7.2 Gradients and Hessians

For a scalar function $f : \mathbb{R}^d \to \mathbb{R}$,

$$\nabla f(x) = \begin{pmatrix} \partial_{x_1} f \\ \vdots \\ \partial_{x_d} f \end{pmatrix}.$$

The Hessian is

$$H_f(x) = \nabla^2 f(x) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j} \right)_{i,j=1}^{d}.$$

The Laplacian is the trace of the Hessian,

$$\Delta f = \operatorname{tr}(H_f) = \sum_{i=1}^{d} \partial_{x_i x_i} f.$$

This immediately explains why a strong-form PINN for Poisson's equation needs second-order AD.

3.7.3 Input derivatives versus parameter derivatives

The function $u_\theta(x, t)$ depends on both physical coordinates and model parameters. We must distinguish

$$\partial_x u_\theta, \quad \partial_t u_\theta, \quad \partial_{xx} u_\theta$$

from

$$\nabla_\theta \mathcal{L}(\theta).$$

The first group enters the PDE residual; the second group drives optimization. Modern AD systems can differentiate with respect to either set of variables, provided those variables participate in the computation graph.

3.7.4 First derivatives: transport equation

Consider the transport equation

$$u_t + cu_x = 0.$$

For a network $u_\theta(x,t)$, the residual is

$$\mathcal{R}(x,t;\theta) = \partial_t u_\theta + c\partial_x u_\theta.$$

Only first derivatives with respect to the inputs are needed.

3.7.5 Second derivatives

Second-order PDEs require nested differentiation. Examples include

$$-\Delta u = f, \qquad u_t - \alpha\Delta u = 0, \qquad u_{tt} - c^2 u_{xx} = 0.$$

AD evaluates, for example,

$$\partial_{xx} u_\theta = \partial_x(\partial_x u_\theta).$$

This is mathematically straightforward but computationally more expensive because the derivative graph itself must remain differentiable.

3.7.6 Mixed derivatives and smoothness

When u is sufficiently smooth, Clairaut's theorem gives

$$\partial_{xt} u = \partial_{tx} u.$$

AD can compute either order, but the existence and quality of those derivatives depend on the regularity of the represented function. This leads to an important design issue in PINNs: the activation function should be compatible with the derivative order required by the PDE.

For example, ReLU is continuous and piecewise linear. Its first derivative is piecewise constant and its classical second derivative is zero away from the kink and undefined at the kink. Consequently, a strong-form PINN for a second-order PDE often uses smooth activations such as tanh, sine, or other differentiable nonlinearities. This does not mean ReLU is never usable in scientific machine learning; rather, the residual formulation and required regularity must be considered carefully.

3.8 PDE examples

3.8.1 Poisson equation

Consider

$$-\Delta u(x) = f(x), \qquad x \in \Omega,$$

with Dirichlet condition

$$u = g \qquad \text{on } \partial\Omega.$$

Let $u_\theta(x)$ be a neural approximation. The strong-form residual is

$$\mathcal{R}_f(x;\theta) = -\Delta u_\theta(x) - f(x).$$

At collocation points $\{x_i\}_{i=1}^{N_f}$,

$$\mathcal{L}_f(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{R}_f(x_i; \theta)|^2.$$

AD supplies the spatial second derivatives that make up the Laplacian.

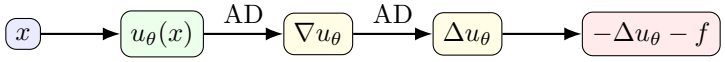


Figure 3.5: Computation of the Poisson residual using repeated automatic differentiation.

3.8.2 Heat equation

For the one-dimensional heat equation

$$u_t - \alpha u_{xx} = 0, \qquad (x, t) \in (0, 1) \times (0, T),$$

we represent the spacetime solution by $u_\theta(x, t)$. The residual is

$$\mathcal{R}(x, t; \theta) = \partial_t u_\theta(x, t) - \alpha \partial_{xx} u_\theta(x, t).$$

A typical training loss is

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_b \mathcal{L}_{\text{BC}}(\theta) + \lambda_0 \mathcal{L}_{\text{IC}}(\theta).$$

The key conceptual point is that time is treated simply as another differentiable input coordinate. Unlike a classical time-stepping scheme, a global-in-time PINN represents a function over the entire spacetime cylinder.

3.8.3 Burgers' equation

The viscous Burgers equation is

$$u_t + u u_x = \nu u_{xx}.$$

The PINN residual is

$$\mathcal{R}(x, t; \theta) = \partial_t u_\theta + u_\theta \partial_x u_\theta - \nu \partial_{xx} u_\theta.$$

The nonlinear term does not require any special differentiation rule beyond the ordinary product structure already available in the computation graph. AD differentiates the network; algebraic operations combine the derivatives and field values into the residual. During training, the residual loss is then differentiated with respect to θ.

This example makes the nested nature of PINN differentiation especially clear:

1. differentiate u_θ with respect to x and t;
2. construct the nonlinear residual;
3. square and average the residual over collocation points;
4. differentiate the resulting scalar loss with respect to θ.

3.8.4 Inverse coefficient problem

Suppose

$$-\nabla \cdot (k(x)\nabla u(x)) = f(x),$$

where both u and the coefficient k are unknown. One possible approach is to introduce two networks,

$$u_\theta(x), \qquad k_\phi(x).$$

Then

$$\mathcal{R}(x;\theta,\phi) = -\nabla \cdot (k_\phi(x)\nabla u_\theta(x)) - f(x).$$

AD computes the gradient of u_θ and the divergence of the product $k_\phi \nabla u_\theta$. Data misfit terms can be added for observed values of u, k, or related quantities. This is one reason PINNs and differentiable programming are attractive for inverse problems: physical constraints and unknown model components can be placed in one differentiable graph.

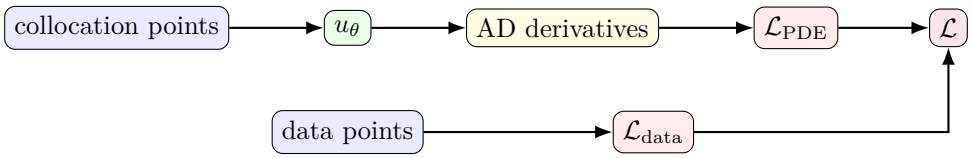


Figure 3.6: Physics and data can be combined within one differentiable objective.

3.9 Software implementation

3.9.1 PyTorch-style input derivatives

A practical implementation must tell the framework that the input coordinates participate in differentiation. In PyTorch-style pseudocode:

```
# x and t are coordinate tensors
X = torch.cat([x, t], dim=1)
X.requires_grad_(True)

u = model(X)

# First derivatives of u with respect to X
# create_graph=True keeps this derivative differentiable
grad_u = torch.autograd.grad(
    u, X,
    grad_outputs=torch.ones_like(u),
    create_graph=True
)[0]

u_x = grad_u[:, 0:1]
u_t = grad_u[:, 1:2]

# Second derivative u_xx
grad_ux = torch.autograd.grad(
    u_x, X,
    grad_outputs=torch.ones_like(u_x),
    create_graph=True
)[0]
```

```
u_xx = grad_ux[:, 0:1]
```

The option `create_graph=True` is essential when a derivative will itself be differentiated. Without it, the framework may discard the graph needed to form u_{xx} or to backpropagate through the residual.

3.9.2 Heat-equation residual

A residual function can then be written as follows.

```
def heat_residual(model, x, t, alpha):
    X = torch.cat([x, t], dim=1)
    X.requires_grad_(True)
    u = model(X)

    grad_u = torch.autograd.grad(
        u, X,
        grad_outputs=torch.ones_like(u),
        create_graph=True
    )[0]

    u_x = grad_u[:, 0:1]
    u_t = grad_u[:, 1:2]

    grad_ux = torch.autograd.grad(
        u_x, X,
        grad_outputs=torch.ones_like(u_x),
        create_graph=True
    )[0]
    u_xx = grad_ux[:, 0:1]

    return u_t - alpha * u_xx
```

The returned residual is a differentiable tensor. A mean-square residual loss can be formed and differentiated with respect to the network parameters in the usual way.

3.9.3 Frameworks

Several modern frameworks provide AD:

- **PyTorch**: dynamic computation graphs and reverse-mode differentiation through `autograd`;
- **TensorFlow**: differentiation through gradient tapes;
- **JAX**: composable transformations such as `grad`, `jacfwd`, `jacrev`, `jvp`, `vjp`, and `vmap`;
- **Julia ecosystems**: differentiable-programming tools such as Zygote and Flux.

The software interfaces differ, but the underlying mathematical principle is the chain rule on a computational graph.

3.9.4 Batching and vectorization

PINNs evaluate residuals at many points,

$$\{(x_i, t_i)\}_{i=1}^{N_f}.$$

It is usually inefficient to loop over points in a high-level language. Instead, all collocation points in a batch are passed through the network simultaneously. Vectorized operations allow efficient use of GPUs and other accelerators.

Batch size is nevertheless constrained by memory. Second derivatives require retaining more graph information than ordinary supervised learning. Hence a batch size that is comfortable for standard regression may be too large for a second-order PINN.

3.10 Practical issues and limitations

3.10.1 Memory cost of reverse mode

Reverse-mode AD generally stores intermediate activations from the forward pass because they are needed during the backward sweep. If a PINN requires higher-order derivatives, the derivative operations themselves must remain part of a differentiable graph. This can substantially increase memory usage.

Common remedies include:

- smaller collocation batches;
- gradient checkpointing, which recomputes selected activations instead of storing all of them;
- mixed forward/reverse-mode strategies for Hessian-related quantities;
- reformulating the PDE as a first-order system to reduce the derivative order;
- weak or variational residuals, which may require fewer derivatives of the network output.

3.10.2 Differentiability pitfalls

AD differentiates the program that was actually written. It cannot repair a mathematically unsuitable representation. Non-smooth functions, clipping, discontinuous branching, or other operations may produce undefined, zero, or misleading gradients.

This observation matters for boundary-condition constructions and for custom layers. If a quantity is inserted inside a PDE residual, every operation on the path from input to residual should be compatible with the derivative order being requested.

3.10.3 Higher-order derivatives and approximation quality

An important distinction is that AD can compute the derivative of the *learned function* accurately, but the learned derivative need not approximate the derivative of the true solution accurately. A network may fit u well in value while producing poor u_{xx}. Therefore, derivative

accuracy is part of the approximation and optimization problem, not merely an implementation detail.

For example, suppose

$$\|u - u_\theta\|_{L^2}$$

is small. This alone does not imply that

$$\|u_{xx} - (u_\theta)_{xx}\|_{L^2}$$

is small. Strong-form PINNs directly penalize derivative-based residuals precisely because value accuracy alone is insufficient for satisfying a differential operator.

3.10.4 Stiffness and scaling

AD does not remove the numerical conditioning of the underlying learning problem. If a PDE is stiff or contains terms with very different scales, then the residual can have highly unbalanced derivatives with respect to the parameters. This can make gradient-based optimization difficult even though each derivative is computed correctly.

Consequently, nondimensionalization, input scaling, loss weighting, adaptive sampling, and suitable optimizers remain important. AD is a differentiation engine, not a cure for poor conditioning.

3.10.5 Finite differences still have a role

Despite the advantages of AD, numerical differentiation remains useful for verification. A standard debugging technique is to compare an AD gradient with a finite-difference approximation on a small test problem. If

$$g_{\mathrm{AD}}$$

is the automatic derivative and

$$g_{\mathrm{FD}}(h)$$

is a finite-difference estimate, agreement over a sensible range of h gives confidence that a custom residual or derivative rule has been implemented correctly.

Finite differences are also necessary for black-box components that cannot be differentiated by the AD framework, unless a custom derivative or an alternative differentiable surrogate is supplied.

3.11 A unified view of PINN differentiation

It is useful to summarize the full nested computation. Consider a PDE operator $\mathcal{N}$ and a network u_θ. At collocation points x_i:

1. evaluate $u_\theta(x_i)$;
2. differentiate u_θ with respect to coordinates to obtain the derivatives required by $\mathcal{N}$;
3. construct the residual
$$\mathcal{R}_i(\theta) = \mathcal{N}[u_\theta](x_i) - f(x_i);$$

4. define the residual loss
$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{N_f} \sum_i |\mathcal{R}_i(\theta)|^2;$$

5. differentiate the scalar loss with respect to θ;

6. update θ using a gradient-based optimizer.

Thus automatic differentiation is used both *inside* the physical loss and *outside* it during optimization. This is the computational bridge between calculus and machine learning that underpins the standard strong-form PINN methodology.

3.12 Summary

The central ideas of this lecture can be summarized as follows.

- Symbolic differentiation manipulates formulas; numerical differentiation approximates derivatives from nearby function values; automatic differentiation differentiates the executed computation.
- AD works by decomposing a program into elementary operations and systematically applying the chain rule.
- Forward mode propagates tangent information from inputs to outputs and is efficient for small input dimension or directional derivatives.
- Reverse mode propagates adjoint sensitivities from outputs back to inputs and is especially effective for gradients of scalar losses with many parameters.
- Backpropagation is reverse-mode AD applied to neural networks.
- PINNs use AD to compute physical derivatives with respect to coordinates and training gradients with respect to parameters.
- Second derivatives, memory usage, activation smoothness, and derivative quality are important practical issues.
- AD evaluates derivatives of the represented function accurately, but it does not guarantee that the represented function or its derivatives approximate the true PDE solution accurately.

3.13 Discussion questions and exercises

1. For $f(x) = e^{\sin(x^2)}$, write a computational graph and identify the local derivative at every node. Compute $f'(x)$ using the chain rule.

2. Let
$$F(x_1, x_2) = \begin{pmatrix} x_1 x_2 \\ \sin x_1 + x_2^2 \end{pmatrix}.$$
Compute the Jacobian analytically. Discuss whether forward or reverse mode would be preferable if F were replaced by a mapping $\mathbb{R}^2 \to \mathbb{R}^{1000}$.

3. For the wave equation
$$u_{tt} - c^2 u_{xx} = 0,$$
write the strong-form PINN residual and list all automatic derivatives that must be computed.

4. Consider Poisson's equation in d dimensions. Express the residual in terms of the Hessian of u_θ. Which entries of the Hessian are actually needed?

5. Explain why a network with ReLU activation is potentially problematic for a classical strong-form PINN approximation of Poisson's equation.

6. Implement a small scalar function in a framework of your choice and compare its AD derivative with a central finite-difference derivative over several values of h.

7. In an inverse problem with two networks u_θ and k_ϕ, explain which derivatives are taken with respect to physical coordinates and which are taken with respect to trainable parameters.

Bibliography

[1] L. B. Rall, *Automatic Differentiation: Techniques and Applications*, Springer, 1981.

[2] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM, 2008.

[3] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, 323:533–536, 1986.

[5] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378:686–707, 2019.

[6] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems*, 2019.

[7] J. Bradbury et al., "JAX: composable transformations of Python+NumPy programs," 2018.

Chapter 4

From PDEs to Loss Functions

4.1 Introduction and learning goals

The central idea of a physics-informed neural network (PINN) is simple to state but important to understand carefully. A classical numerical method typically starts by discretizing the domain, deriving a finite-dimensional algebraic problem, and then solving for nodal or modal coefficients. A PINN starts from a different computational object: a neural network that represents the unknown solution as a global parameterized function. The PDE, boundary conditions, initial conditions, and any available observations are then converted into differentiable residuals and loss terms. Training consists of minimizing the resulting loss with respect to the network parameters.

The purpose of this lecture is therefore to explain the translation

$$\boxed{\text{PDE problem} \quad \longrightarrow \quad \text{differentiable optimization problem.}}$$

By the end of the lecture, one should be able to

- describe the neural-network ansatz $\widehat{u}(x;\theta)$;
- distinguish interior, boundary, initial, and data collocation points;
- construct a strong-form PDE residual;
- derive the PINN loss for the Poisson equation;
- explain soft and hard enforcement of boundary conditions;
- explain why scaling, loss weighting, and sampling influence training;
- extend the same construction to time-dependent, nonlinear, and inverse problems.

4.2 From PDEs to optimization

4.2.1 Historical perspective: neural trial functions

Using neural networks as trial functions for differential equations predates the modern terminology of "physics-informed neural networks." A representative early idea is to write a trial solution in the form

$$u_T(x;\theta) = A(x) + B(x)N(x;\theta), \tag{4.1}$$

where $N(x;\theta)$ is a trainable neural network, $A(x)$ is chosen so that the boundary or initial data are already satisfied, and $B(x)$ is chosen to vanish on the constrained part of the boundary. In this way, the admissibility conditions are built directly into the approximation. This strategy appears prominently in the neural differential-equation work of Lagaris, Likas and Fotiadis [1].

Modern PINNs use the same underlying philosophy, but automatic differentiation, modern optimizers, GPU computing, and flexible neural architectures make it possible to build the entire differential equation into a training objective. The modern framework was popularized by Raissi, Perdikaris and Karniadakis [2] and has since developed into a broad area of physics-informed machine learning [3].

4.2.2 The neural-network ansatz

Consider a boundary value problem

$$\mathcal{N}[u](x) = f(x), \qquad x \in \Omega, \tag{4.2}$$

with boundary condition

$$\mathcal{B}[u](x) = g(x), \qquad x \in \partial\Omega. \tag{4.3}$$

A PINN approximates u by a neural network

$$u(x) \approx \widehat{u}(x;\theta), \tag{4.4}$$

where θ contains all weights and biases. For a time-dependent problem,

$$u(x,t) \approx \widehat{u}(x,t;\theta).$$

The coordinates are therefore the network inputs. In supervised learning one often thinks of pairs $x_i \mapsto y_i$. In a PINN, one instead has

$$x \mapsto \widehat{u}(x;\theta), \qquad \text{or} \qquad (x,t) \mapsto \widehat{u}(x,t;\theta).$$

The important distinction is that target values do not need to be known at every point. The governing equation itself provides information through the residual.

4.2.3 Classical workflow versus PINN workflow

The two computational philosophies can be summarized schematically as follows.

The distinction in Figure 4.1 is conceptual rather than absolute. Both methods must still approximate a continuous mathematical problem by finite computation. A PINN is often described as mesh-free, but it is not sampling-free: it still evaluates the residual at finitely many points.

4.2.4 Anatomy of a PINN loss

A typical loss is a weighted sum of several contributions:

$$\mathcal{L}(\theta) = \lambda_f \mathcal{L}_f(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_d \mathcal{L}_d(\theta). \tag{4.5}$$

Here $\mathcal{L}_f$ measures the PDE residual in the interior, $\mathcal{L}_b$ measures boundary or initial-condition mismatch, and $\mathcal{L}_d$ measures mismatch with observations. The nonnegative parameters λ_f, λ_b, and λ_d determine how strongly the corresponding objectives are weighted.

The role of these weights should not be underestimated. They are part of the numerical method: they alter the geometry of the optimization problem and therefore influence what the optimizer learns first and how the competing objectives are balanced.

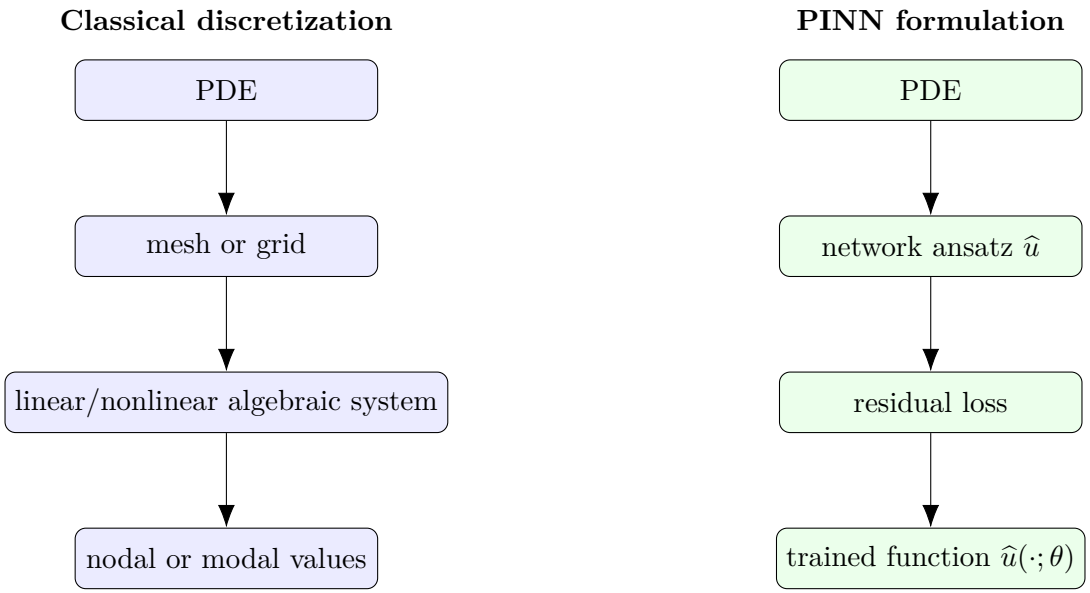


Figure 4.1: Two different computational routes to approximate the same mathematical solution.

4.3 Collocation points and residuals

4.3.1 Interior, boundary, initial and data points

PINNs enforce the PDE at sampled locations called collocation points. For a spatial domain Ω, define the interior set

$$X_f = \{x_i^f\}_{i=1}^{N_f} \subset \Omega$$

and a boundary set

$$X_b = \{x_j^b\}_{j=1}^{N_b} \subset \partial\Omega.$$

If measurements are available, one may also use

$$X_d = \{x_k^d\}_{k=1}^{N_d}.$$

For an evolution problem on $\Omega_T = \Omega \times (0, T]$, interior points take the form (x_i, t_i), initial points lie on $t = 0$, and boundary points lie on $\partial\Omega \times [0, T]$.

Common sampling choices include uniform grids, independent random points, Latin hypercube sampling, low-discrepancy sequences, and adaptive strategies. The sampling rule determines where the physics is explicitly checked and therefore plays a role analogous to resolution in classical discretization.

4.3.2 Strong-form residuals

Suppose the PDE is written in strong form as

$$\mathcal{N}[u](x) = f(x), \qquad x \in \Omega.$$

Given a neural approximation $\widehat{u}$, define the interior residual

$$\mathcal{R}_f(x;\theta) = \mathcal{N}[\widehat{u}](x;\theta) - f(x). \tag{4.6}$$

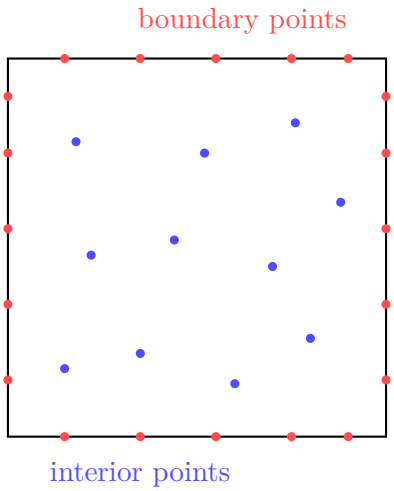


Figure 4.2: Schematic collocation points in a square domain. Blue points are used for the PDE residual; red points are used for boundary conditions.

Similarly, for a boundary operator $\mathcal{B}$,

$$\mathcal{R}_b(x;\theta) = \mathcal{B}[\widehat{u}](x;\theta) - g(x). \tag{4.7}$$

The goal is to make these residuals small at the sampled points.

Typical strong-form examples are

$$-\Delta u = f, \qquad u_t - \alpha u_{xx} = 0, \qquad u_t + uu_x - \nu u_{xx} = 0.$$

The first is elliptic, the second is parabolic, and the third is nonlinear. The loss-construction principle is the same in all three cases.

4.3.3 Automatic differentiation

The derivatives appearing in the residual are obtained by automatic differentiation. Because a neural network is a computational graph composed of elementary differentiable operations, one can compute

$$\partial_{x_i}\widehat{u}, \qquad \partial_{x_i x_j}\widehat{u}, \qquad \partial_t \widehat{u}, \qquad \Delta\widehat{u}$$

without constructing finite-difference stencils or manipulating a symbolic formula. Automatic differentiation should be distinguished from both symbolic differentiation and numerical finite differences; see Baydin et al. [3] for a detailed survey.

For higher-order PDEs, activation smoothness matters. For example, if second derivatives are required, an activation such as tanh is often preferred over piecewise-linear ReLU in standard strong-form PINNs because the second derivative of ReLU vanishes almost everywhere and is undefined at its kink.

4.3.4 Residual losses as empirical norms

The continuous squared L^2 residual is

$$\|\mathcal{R}_f\|^2_{L^2(\Omega)} = \int_\Omega |\mathcal{R}_f(x;\theta)|^2 \, dx.$$

If x_i are sampled approximately uniformly, the integral may be approximated by Monte Carlo or quadrature-like sampling:

$$\int_\Omega |\mathcal{R}_f(x;\theta)|^2\,dx \approx \frac{|\Omega|}{N_f}\sum_{i=1}^{N_f}|\mathcal{R}_f(x_i;\theta)|^2.$$

Up to an irrelevant constant factor, this motivates the standard mean-squared residual loss

$$\mathcal{L}_f(\theta) = \frac{1}{N_f}\sum_{i=1}^{N_f}|\mathcal{R}_f(x_i;\theta)|^2. \tag{4.8}$$

This viewpoint is useful because it connects PINNs with least-squares formulations: the network is trained to minimize an empirical norm of equation violation.

4.3.5 Strong form versus weak form

Strong-form PINNs are the most direct construction: the PDE is evaluated pointwise. This is attractive for introductory purposes because it mirrors the differential equation exactly. However, it also requires enough regularity from the neural ansatz to evaluate the derivatives appearing in the PDE.

Weak-form approaches instead multiply the equation by test functions and integrate. For example, Poisson's equation

$$-\Delta u = f$$

with suitable boundary conditions can be rewritten as

$$\int_\Omega \nabla u\cdot\nabla v\,dx = \int_\Omega fv\,dx$$

for test functions v. Such formulations can reduce derivative order and connect more directly with finite element theory. They also introduce quadrature and choices of test functions. For smooth PDEs and simple domains, the strong form is often the natural starting point; for nonsmooth solutions, conservation laws, or low-regularity settings, weak or conservative variants may be more appropriate.

4.4 Worked example: the Poisson equation

4.4.1 Problem statement

Consider the Dirichlet problem

$$\begin{aligned} -\Delta u(x) &= f(x), \quad x\in\Omega,\\ u(x) &= g(x), \quad x\in\partial\Omega. \end{aligned} \tag{4.9}$$

This is one of the cleanest PDEs for introducing the PINN construction because the residual is simple while still requiring second derivatives.

4.4.2 Manufactured solution

Let $\Omega = (0,1)^2$ and choose the exact solution

$$u(x,y) = \sin(\pi x)\sin(\pi y). \tag{4.10}$$

Differentiating gives

$$u_{xx} = -\pi^2 \sin(\pi x)\sin(\pi y), \qquad u_{yy} = -\pi^2 \sin(\pi x)\sin(\pi y),$$

so

$$\Delta u = -2\pi^2 \sin(\pi x)\sin(\pi y).$$

Therefore the forcing term must be

$$f(x,y) = 2\pi^2 \sin(\pi x)\sin(\pi y), \tag{4.11}$$

and the Dirichlet data are homogeneous because $u = 0$ on the boundary of the unit square.

Manufactured solutions are especially useful for teaching and verification. One chooses u first, derives f, and then tests whether the numerical method can reproduce the known solution. This gives an immediate way to measure error and identify implementation mistakes.

4.4.3 Constructing the residual

Let $\widehat{u}(x,y;\theta)$ be the network approximation. The PDE residual is

$$\mathcal{R}_f(x,y;\theta) = -\left(\frac{\partial^2 \widehat{u}}{\partial x^2} + \frac{\partial^2 \widehat{u}}{\partial y^2}\right) - f(x,y). \tag{4.12}$$

The boundary residual for Dirichlet data is

$$\mathcal{R}_b(x,y;\theta) = \widehat{u}(x,y;\theta) - g(x,y), \qquad (x,y) \in \partial\Omega. \tag{4.13}$$

For the manufactured solution, $g = 0$, so $\mathcal{R}_b = \widehat{u}$ on the boundary.

With interior points $\{(x_i^f, y_i^f)\}_{i=1}^{N_f}$ and boundary points $\{(x_j^b, y_j^b)\}_{j=1}^{N_b}$, define

$$\mathcal{L}_f(\theta) = \frac{1}{N_f}\sum_{i=1}^{N_f} \left|\mathcal{R}_f(x_i^f, y_i^f;\theta)\right|^2, \tag{4.14}$$

$$\mathcal{L}_b(\theta) = \frac{1}{N_b}\sum_{j=1}^{N_b} \left|\mathcal{R}_b(x_j^b, y_j^b;\theta)\right|^2. \tag{4.15}$$

The total loss is

$$\boxed{\mathcal{L}(\theta) = \mathcal{L}_f(\theta) + \lambda_b \mathcal{L}_b(\theta).} \tag{4.16}$$

4.4.4 What the optimizer actually sees

The optimizer does not directly understand Poisson's equation, elliptic regularity, or the maximum principle. It sees only a scalar objective and its gradient with respect to parameters. The computational pipeline is illustrated in Figure 4.3.

This observation explains why a mathematically correct loss does not automatically imply an easy optimization problem. The quality of the solution depends both on approximation capability and on whether gradient-based training can locate useful parameters.

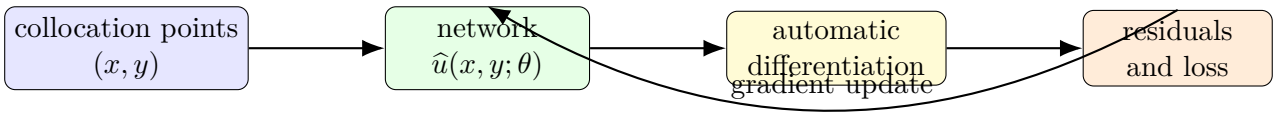


Figure 4.3: Computational pipeline for PINN training.

4.5 Boundary conditions: soft and hard enforcement

4.5.1 Soft enforcement

Soft enforcement introduces a penalty term into the objective. For Dirichlet conditions,

$$\mathcal{L} = \mathcal{L}_f + \lambda_b \mathcal{L}_b.$$

This is simple and flexible. It also allows different types of constraints and data to be combined in a common framework. However, the boundary condition is not satisfied exactly: it is only encouraged through optimization.

4.5.2 Hard enforcement

Hard enforcement builds the boundary condition into the trial function. For example, in one dimension consider

$$-u''(x) = f(x), \qquad x \in (0, 1),$$

with

$$u(0) = u(1) = 0.$$

Choose

$$\widehat{u}(x; \theta) = x(1 - x)N(x; \theta). \tag{4.17}$$

Then $\widehat{u}(0; \theta) = \widehat{u}(1; \theta) = 0$ for every value of θ. The boundary constraints disappear from the optimization problem because they are already satisfied by construction.

Hard constraints can be extremely useful, but they require an ansatz adapted to the geometry and the boundary data. On complicated domains this construction may be nontrivial, whereas soft enforcement is usually straightforward.

4.5.3 Neumann and Robin conditions

For a Neumann boundary condition

$$\frac{\partial u}{\partial n} = q,$$

the boundary residual is

$$\mathcal{R}_N(x; \theta) = \nabla \widehat{u}(x; \theta) \cdot n(x) - q(x). \tag{4.18}$$

For a Robin condition

$$\alpha u + \beta \frac{\partial u}{\partial n} = r,$$

use

$$\mathcal{R}_R(x; \theta) = \alpha \widehat{u}(x; \theta) + \beta \nabla \widehat{u}(x; \theta) \cdot n(x) - r(x). \tag{4.19}$$

Thus the same residual philosophy handles different boundary operators; only the definition of the boundary residual changes.

4.6 Adding observations and inverse information

If observations are available at locations x_k^d, add a data term

$$\mathcal{L}_d(\theta) = \frac{1}{N_d} \sum_{k=1}^{N_d} |\widehat{u}(x_k^d; \theta) - u_k^d|^2. \tag{4.20}$$

The total loss may then take the form

$$\mathcal{L} = \lambda_f \mathcal{L}_f + \lambda_b \mathcal{L}_b + \lambda_d \mathcal{L}_d. \tag{4.21}$$

This is one of the main reasons PINNs are attractive in scientific machine learning: they provide a common framework for combining governing equations with sparse measurements. The physics acts as a structural prior or regularizer, while the data help determine quantities that are not fully constrained by the PDE alone.

4.7 Loss weights, scaling, and nondimensionalization

4.7.1 Why weights matter

Consider

$$\mathcal{L} = \mathcal{L}_f + \lambda_b \mathcal{L}_b.$$

If λ_b is too small, the optimizer may reduce the interior PDE residual while leaving a significant boundary mismatch. If λ_b is too large, it may fit the boundary accurately while making slow progress in the interior. Therefore λ_b is not merely a cosmetic tuning parameter: it influences the balance of gradient signals during training.

4.7.2 Units and scales

Loss components may have different physical units. For example,

$$\mathcal{R}_f = -\Delta \widehat{u} - f$$

has the units of a forcing term, while

$$\mathcal{R}_b = \widehat{u} - g$$

has the units of the solution. If these quantities differ by orders of magnitude, one term may dominate the optimization purely because of units rather than physical importance.

For this reason, nondimensionalization and normalization are often valuable. One can rescale coordinates, state variables, coefficients, or residuals so that the important terms have comparable numerical magnitudes. This is closely related to good conditioning in classical numerical analysis.

4.8 Implementation pattern

The mathematics of a PINN translates almost directly into code. A simplified training loop has the following structure.

Listing 4.1: Schematic PINN training loop for a Poisson-type problem.

```
for epoch in range(num_epochs):
    xf = sample_interior_points(Nf)
    xb = sample_boundary_points(Nb)

    u = net(xf)
    ux = grad(u, xf)
    uxx = grad(ux, xf)

    Rf = -laplacian(u, xf) - f(xf)
    Rb = net(xb) - g(xb)

    loss_f = mean(Rf**2)
    loss_b = mean(Rb**2)
    loss = loss_f + lambda_b * loss_b

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

In practice one must take care that the computational graph used for the first derivative is retained when second derivatives are required. Framework-specific details differ, but the mathematical sequence is always the same: evaluate the network, differentiate it, construct residuals, form the loss, differentiate the loss with respect to parameters, and update the parameters.

4.9 Training diagnostics and failure modes

4.9.1 Monitor individual components

A single total loss value can hide important information. It is good practice to monitor

- the PDE residual loss $\mathcal{L}_f$;
- the boundary loss $\mathcal{L}_b$;
- any data loss $\mathcal{L}_d$;
- validation error when an exact or reference solution is available;
- the spatial distribution of the residual.

The diagnostic question is not simply “Is the loss going down?” but rather “Which part of the physical problem is the network failing to satisfy?”

4.9.2 Small training loss does not imply small solution error

Because the residual is evaluated at finitely many points,

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{R}_f(x_i; \theta)|^2,$$

a network may fit the residual well at sampled points while violating the PDE between them. Similarly, a low PDE residual does not by itself guarantee accurate boundary values unless the boundary term is also controlled. This is why validation points, residual maps, and comparison with reference solutions are important.

4.9.3 Typical failure modes

Common difficulties include

- insufficient or poorly distributed collocation points;
- unbalanced loss components;
- poorly scaled variables and coefficients;
- incompatible or noisy data;
- optimization plateaus or unstable gradients;
- activation functions that are not smooth enough for the required derivatives;
- spectral bias, leading to overly smooth approximations of sharp or oscillatory features.

These are not merely implementation annoyances. They are central numerical issues that motivate many advanced PINN methods.

4.10 Adaptive collocation strategies

One simple refinement is residual-based adaptive sampling. Start with an initial set of collocation points, train the PINN, evaluate the residual on a larger set of candidate points, add points where the residual is large, and continue training. Symbolically:

1. train on $X_f^{(0)}$;
2. evaluate $|\mathcal{R}_f|$ on candidate points;
3. select points with large residual;
4. augment X_f and continue optimization.

The intuition is the same as adaptive refinement in classical numerical analysis: spend computational effort where the current approximation is least satisfactory.

4.11 Extensions

4.11.1 Heat equation

For the one-dimensional heat equation

$$u_t - \alpha u_{xx} = 0, \tag{4.22}$$

use the network $\widehat{u}(x,t;\theta)$ and define

$$\mathcal{R}_f(x,t;\theta) = \partial_t \widehat{u}(x,t;\theta) - \alpha \partial_{xx}\widehat{u}(x,t;\theta). \tag{4.23}$$

The loss must include interior, boundary, and initial-condition terms:

$$\mathcal{L} = \mathcal{L}_f + \lambda_b \mathcal{L}_b + \lambda_0 \mathcal{L}_0.$$

This is the direct time-dependent analogue of the Poisson construction.

4.11.2 Burgers' equation

For viscous Burgers' equation

$$u_t + u u_x - \nu u_{xx} = 0, \tag{4.24}$$

the PINN residual is

$$\mathcal{R}_f(x,t;\theta) = \partial_t \widehat{u} + \widehat{u}\,\partial_x \widehat{u} - \nu \partial_{xx}\widehat{u}. \tag{4.25}$$

The nonlinear product $\widehat{u}\,\partial_x\widehat{u}$ couples the network output and its derivative. This makes the optimization landscape more complicated than in linear elliptic or parabolic examples, and it provides a useful benchmark for studying training difficulties.

4.11.3 Inverse coefficient problems

Consider

$$-\nabla \cdot (k(x)\nabla u(x)) = f(x), \tag{4.26}$$

where $k(x)$ is unknown. One option is to approximate both fields with neural networks,

$$u(x) \approx \widehat{u}(x;\theta_u), \qquad k(x) \approx \widehat{k}(x;\theta_k).$$

The residual becomes

$$\mathcal{R}_f(x;\theta_u,\theta_k) = -\nabla \cdot \left(\widehat{k}(x;\theta_k)\nabla \widehat{u}(x;\theta_u)\right) - f(x). \tag{4.27}$$

Measurements of u can then be included through a data loss. Training attempts to infer both the state and the coefficient. This is one of the settings in which PINNs are especially attractive because the forward model and inverse inference can be coupled in a single optimization problem.

4.11.4 Wave equation exercise

For

$$u_{tt} - c^2 u_{xx} = 0, \qquad x \in (0,1), \quad t \in (0,T),$$

with

$$u(0,t) = u(1,t) = 0, \qquad u(x,0) = u_0(x), \qquad u_t(x,0) = v_0(x),$$

a natural network is $\widehat{u}(x,t;\theta)$ and the PDE residual is

$$\mathcal{R}_f = \partial_{tt}\widehat{u} - c^2 \partial_{xx}\widehat{u}.$$

The loss contains four pieces: a PDE residual term, a boundary term, an initial displacement term, and an initial velocity term. Writing these explicitly is a useful exercise because it reinforces the general recipe.

4.12 Connection with finite element residuals

Finite elements and PINNs can both be interpreted through residuals, but they measure residuals differently. A conforming finite element method for Poisson's equation seeks u_h satisfying

$$\int_\Omega \nabla u_h \cdot \nabla v_h \, dx = \int_\Omega f v_h \, dx$$

for all discrete test functions v_h. A strong-form PINN instead minimizes pointwise residual samples

$$\sum_i | - \Delta \widehat{u}(x_i) - f(x_i)|^2.$$

Thus FEM enforces a variational statement against test functions, while a standard PINN enforces pointwise differential residuals. This comparison helps place PINNs within the broader numerical-analysis landscape rather than treating them as an unrelated machine-learning construction.

4.13 Computational cost

PINNs are not automatically cheaper than classical solvers. Higher-order automatic differentiation through a deep network can be expensive in both time and memory. Cost depends on

- the number of collocation points;
- network depth and width;
- the order of derivatives in the PDE;
- whether Hessian-type information is required;
- optimizer choice and batch size.

For a single well-posed forward problem in low spatial dimension, a mature finite difference, finite element, or spectral solver may be far faster and more accurate. PINNs become especially interesting when one wants to combine physics and data, infer unknown coefficients, avoid repeated meshing, or build a reusable surrogate.

4.14 Reproducibility and scientific reporting

PINN results can be sensitive to details that are sometimes omitted from published descriptions. A reproducible computational study should report at least

- network depth, width, activation functions, and parameter count;
- optimizer, learning rate, number of iterations, and any learning-rate schedule;
- number and distribution of interior, boundary, initial, and data points;
- loss weights and any adaptive weighting rule;
- variable scaling or nondimensionalization;

- random seeds;
- validation metrics and reference solutions;
- computing hardware and software versions when relevant.

A visually plausible solution is not enough. Scientific use of PINNs requires quantitative error analysis, diagnostics, and enough implementation detail that another researcher can reproduce the experiment.

4.15 Summary

The essential construction of a PINN can be summarized in five steps:

1. represent the unknown solution by a neural network $\widehat{u}$;
2. sample points in the interior and on constrained sets;
3. use automatic differentiation to construct PDE and boundary residuals;
4. turn those residuals into differentiable loss terms;
5. optimize the network parameters.

The simplicity of this recipe should not hide the important numerical issues underneath it. Sampling, scaling, loss weighting, activation smoothness, and optimization quality can strongly affect the final solution. The Poisson equation provides a clean first example, but the same structure extends naturally to time-dependent PDEs, nonlinear equations, and inverse problems.

Exercises

1. For the one-dimensional Poisson problem $-u'' = f$ on $(0,1)$ with $u(0) = u(1) = 0$, construct both a soft-constraint PINN loss and a hard-constraint trial function.
2. For the heat equation $u_t - u_{xx} = 0$ with known initial and boundary values, write the full loss using separate weights for PDE, boundary, and initial-condition terms.
3. For the manufactured Poisson solution $u(x,y) = \sin(\pi x)\sin(\pi y)$, verify the forcing term and compute the exact L^2 norm of u on $(0,1)^2$.
4. Explain why a ReLU network is problematic for a strong-form Poisson PINN. What changes if one uses a weak formulation?
5. Design a residual-based adaptive sampling algorithm and explain how you would decide when to stop adding collocation points.
6. For the inverse problem (4.26), discuss identifiability: under what circumstances might observations of u be insufficient to determine k uniquely?

Bibliography

[1] I. E. Lagaris, A. Likas, and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378:686–707, 2019.

[3] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[4] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, 3:422–440, 2021.

Chapter 5

Time-Dependent Problems

5.1 Introduction and learning objectives

Time-dependent partial differential equations describe systems that evolve in time. Examples include heat conduction, waves and vibrations, fluid flow, transport, chemical reactions, population dynamics, and many stochastic or quantum models. The unknown is no longer only a spatial field, but a function on a *spacetime domain*,

$$u : \Omega \times [0, T] \to \mathbb{R}^m.$$

This additional time variable changes both the mathematical formulation and the numerical strategy. Classical solvers usually discretize space and then advance the solution sequentially in time. Physics-Informed Neural Networks (PINNs), by contrast, can treat time simply as another input variable and attempt to learn the solution over the whole spacetime region at once.

The goals of this lecture are to understand how PINNs are formulated for evolution equations, how initial and boundary conditions enter the loss, how the heat equation provides a canonical benchmark, and how PINNs compare with explicit and implicit time-stepping schemes. We also discuss important practical issues such as long-time dynamics, causal training, time-domain decomposition, adaptive sampling, data assimilation, and inverse parameter estimation.

5.2 Evolution equations and spacetime domains

A broad class of time-dependent PDEs can be written as

$$\partial_t u + \mathcal{N}[u] = f, \qquad (x, t) \in \Omega \times (0, T], \tag{5.1}$$

subject to an initial condition

$$u(x, 0) = u_0(x), \tag{5.2}$$

and boundary conditions

$$\mathcal{B}u(x, t) = g(x, t), \qquad (x, t) \in \partial\Omega \times (0, T]. \tag{5.3}$$

Here $\mathcal{N}$ is a spatial differential operator, possibly nonlinear, while $\mathcal{B}$ denotes a boundary operator.

For a steady problem, one seeks a field $u(x)$. For an evolution problem, the state changes with time and the initial condition selects the trajectory followed by the solution. The geometry of the computational problem is therefore naturally viewed as a spacetime cylinder.

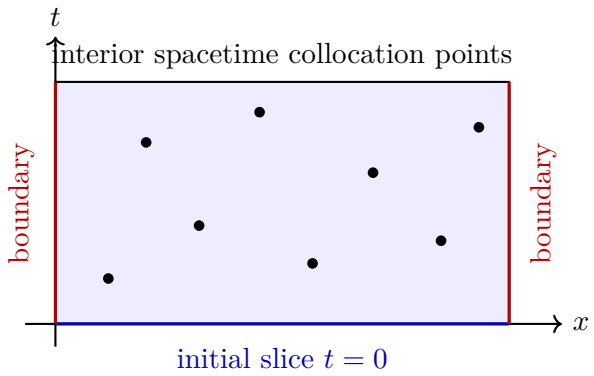


Figure 5.1: A spacetime domain for a one-dimensional time-dependent PDE.

5.2.1 PINN viewpoint

A PINN replaces the unknown function $u(x,t)$ by a neural network approximation

$$u(x,t) \approx \widehat{u}(x,t;\theta), \tag{5.4}$$

where θ denotes all trainable weights and biases. The coordinates (x,t) are network inputs. Automatic differentiation is then used to compute $\partial_t \widehat{u}$, $\nabla_x \widehat{u}$, $\Delta \widehat{u}$, and any other derivatives required by the PDE.

For example, for the heat equation the residual is

$$\mathcal{R}(x,t;\theta) = \partial_t \widehat{u}(x,t;\theta) - \alpha \partial_{xx} \widehat{u}(x,t;\theta). \tag{5.5}$$

This residual is itself differentiable with respect to θ, which allows it to be incorporated into a trainable loss function.

5.3 Sampling the spacetime domain

The PDE is continuous, but training uses finitely many points. We therefore introduce separate sets for interior, boundary, and initial data:

$$\mathcal{S}_f = \{(x_i,t_i)\}_{i=1}^{N_f} \subset \Omega \times (0,T], \tag{5.6}$$

$$\mathcal{S}_b = \{(x_j^b,t_j^b)\}_{j=1}^{N_b} \subset \partial\Omega \times (0,T], \tag{5.7}$$

$$\mathcal{S}_0 = \{(x_k^0,0)\}_{k=1}^{N_0} \subset \Omega \times \{0\}. \tag{5.8}$$

The standard time-dependent PINN loss is then

$$\mathcal{L}(\theta) = \lambda_f \mathcal{L}_{\mathrm{PDE}}(\theta) + \lambda_b \mathcal{L}_{\mathrm{BC}}(\theta) + \lambda_0 \mathcal{L}_{\mathrm{IC}}(\theta), \tag{5.9}$$

with

$$\mathcal{L}_{\mathrm{PDE}} = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{R}(x_i,t_i;\theta)|^2, \tag{5.10}$$

$$\mathcal{L}_{\mathrm{BC}} = \frac{1}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}\widehat{u}(x_j^b,t_j^b;\theta) - g(x_j^b,t_j^b)|^2, \tag{5.11}$$

$$\mathcal{L}_{\mathrm{IC}} = \frac{1}{N_0} \sum_{k=1}^{N_0} |\widehat{u}(x_k^0,0;\theta) - u_0(x_k^0)|^2. \tag{5.12}$$

Common sampling strategies include uniform tensor-product grids, random sampling, Latin hypercube sampling, low-discrepancy sequences, residual-based adaptive refinement, and time-windowed sampling. A useful principle is that a PINN may be *mesh-free*, but it is not *sample-free*; training quality depends strongly on the locations at which physics is enforced.

5.4 Initial and boundary conditions

5.4.1 Why the initial condition matters

For an evolution equation, the initial condition is not a secondary constraint. It selects which trajectory the system follows. A soft initial-condition penalty is

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \frac{1}{N_0} \sum_{k=1}^{N_0} |\widehat{u}(x_k, 0; \theta) - u_0(x_k)|^2. \tag{5.13}$$

If the network fits the initial state poorly, this error may contaminate later times. This observation motivates giving the initial-condition term sufficient weight and, in long-time problems, sometimes training the network causally from early times to later times.

5.4.2 Boundary conditions

Boundary conditions hold for every time level. Typical forms are

$$\begin{aligned} \text{Dirichlet:} &\quad u = g, \\ \text{Neumann:} &\quad \nabla u \cdot n = h, \\ \text{Robin:} &\quad au + b\nabla u \cdot n = r. \end{aligned}$$

For Neumann and Robin conditions, boundary losses themselves require derivatives of the network output, which can again be obtained by automatic differentiation.

Initial and boundary data should also be compatible at the spacetime corners. For instance, if

$$u(0, t) = u(1, t) = 0,$$

then a smooth solution should satisfy

$$u_0(0) = u_0(1) = 0.$$

If this compatibility condition fails, the exact PDE solution may contain an initial layer or have reduced regularity.

5.4.3 Hard and soft enforcement

Soft enforcement uses penalties such as (5.9). Hard enforcement instead modifies the network ansatz so that a constraint holds identically. For example,

$$\widehat{u}(x, t; \theta) = u_0(x) + tN(x, t; \theta) \tag{5.14}$$

automatically satisfies

$$\widehat{u}(x, 0; \theta) = u_0(x)$$

for all parameter values. Hard enforcement removes the need to balance an initial-condition penalty, but it can complicate the ansatz or interact awkwardly with boundary constraints.

	Soft enforcement	Hard enforcement
Mechanism	penalty term	ansatz/architecture
Exact satisfaction	usually no	yes
Flexibility	high	problem dependent
Loss weighting	required	reduced

Table 5.1: Soft versus hard enforcement of initial and boundary conditions.

5.5 Worked example: the heat equation

Consider the one-dimensional heat equation

$$u_t = \alpha u_{xx}, \qquad 0 < x < 1, \quad 0 < t \le T, \tag{5.15}$$

$$u(0,t) = 0, \qquad u(1,t) = 0, \tag{5.16}$$

$$u(x,0) = u_0(x). \tag{5.17}$$

The parameter $\alpha > 0$ is the thermal diffusivity. The term u_t describes local temporal change while u_{xx} measures spatial curvature. Diffusion acts to smooth spatial variations.

5.5.1 Exact benchmark solution

If

$$u_0(x) = \sin(\pi x),$$

then separation of variables gives the exact solution

$$u(x,t) = e^{-\alpha\pi^2 t}\sin(\pi x). \tag{5.18}$$

Indeed,

$$u_t = -\alpha\pi^2 e^{-\alpha\pi^2 t}\sin(\pi x),$$

while

$$u_{xx} = -\pi^2 e^{-\alpha\pi^2 t}\sin(\pi x),$$

so that $u_t = \alpha u_{xx}$.

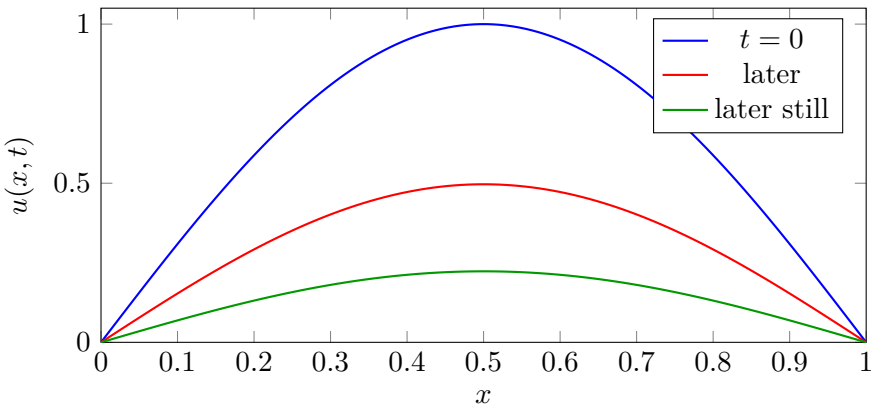


Figure 5.2: Diffusion damps the amplitude while preserving the eigenfunction shape in this simple benchmark.

5.5.2 PINN residual and loss

Let $\widehat{u}(x,t;\theta)$ denote the neural approximation. The residual is

$$\mathcal{R}(x,t;\theta) = \partial_t \widehat{u}(x,t;\theta) - \alpha \partial_{xx} \widehat{u}(x,t;\theta). \tag{5.19}$$

The PDE loss is

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| \partial_t \widehat{u}(x_i,t_i;\theta) - \alpha \partial_{xx} \widehat{u}(x_i,t_i;\theta) \right|^2. \tag{5.20}$$

For homogeneous Dirichlet data,

$$\mathcal{L}_{\mathrm{BC}}(\theta) = \frac{1}{N_b} \sum_{j} \left(|\widehat{u}(0,t_j;\theta)|^2 + |\widehat{u}(1,t_j;\theta)|^2 \right), \tag{5.21}$$

and the initial loss is

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \frac{1}{N_0} \sum_{k} |\widehat{u}(x_k,0;\theta) - u_0(x_k)|^2. \tag{5.22}$$

The total loss is a weighted sum of these terms. The weights are important because the three objectives may have very different numerical scales during training.

5.5.3 Manufactured solutions

Manufactured solutions are useful for verification. Rather than start with a forcing term, choose an exact function and derive the forcing required to make it a solution. For example, let

$$u(x,t) = e^{-t} \sin(\pi x).$$

Then

$$u_t = -e^{-t} \sin(\pi x), \qquad u_{xx} = -\pi^2 e^{-t} \sin(\pi x),$$

so the forced heat equation

$$u_t - \alpha u_{xx} = f(x,t)$$

has

$$f(x,t) = (-1 + \alpha\pi^2) e^{-t} \sin(\pi x). \tag{5.23}$$

This provides a controlled test in which the exact error can be measured.

5.5.4 Conceptual implementation

A minimal PyTorch-style implementation has the following structure:

```
# x and t require gradients
u = net(torch.cat([x, t], dim=1))
u_t = grad(u, t)
u_x = grad(u, x)
u_xx = grad(u_x, x)

residual = u_t - alpha * u_xx
loss_pde = mean(residual**2)
loss_bc = mean(net(x_boundary, t_boundary)**2)
loss_ic = mean((net(x_initial, 0) - u0(x_initial))**2)

loss = loss_pde + lambda_b*loss_bc + lambda_0*loss_ic
loss.backward()
optimizer.step()
```

The essential idea is that the differential operator is evaluated by differentiating the network with respect to its input coordinates.

5.6 Classical time-marching methods

To understand what is distinctive about a time-dependent PINN, it is useful to review the classical approach.

5.6.1 Method of lines

Classical solvers often discretize space first. The PDE is converted into a finite-dimensional ODE system

$$\frac{d\mathbf{u}}{dt} = F(\mathbf{u}, t), \tag{5.24}$$

where $\mathbf{u}(t)$ contains the spatial degrees of freedom. A time integrator then advances the approximation sequentially.

For the heat equation with a standard second-difference discretization,

$$\frac{d\mathbf{u}}{dt} = A\mathbf{u},$$

where A approximates the Laplacian.

5.6.2 Explicit Euler

Explicit Euler is

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t\, F(\mathbf{u}^n, t^n). \tag{5.25}$$

It is simple and cheap per step because no algebraic system must be solved. However, it is conditionally stable. For the heat equation with mesh size h, a typical restriction is

$$\Delta t \lesssim Ch^2. \tag{5.26}$$

Thus spatial refinement can force extremely small time steps.

5.6.3 Backward Euler

Backward Euler uses the new-time value:

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t\, F(\mathbf{u}^{n+1}, t^{n+1}). \tag{5.27}$$

For a linear diffusion equation this gives

$$(I - \Delta t A)\mathbf{u}^{n+1} = \mathbf{u}^n. \tag{5.28}$$

The method is much more stable for parabolic problems, but each step requires solving a linear system. For nonlinear problems, a nonlinear solve may be required.

5.6.4 Crank–Nicolson

Crank–Nicolson averages the old and new right-hand sides:

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \frac{\Delta t}{2}\left(F(\mathbf{u}^n, t^n) + F(\mathbf{u}^{n+1}, t^{n+1})\right). \tag{5.29}$$

For smooth linear problems it is second-order accurate in time. It is often more accurate than backward Euler but still requires implicit solves.

5.7 Global-in-time PINNs

A global PINN approximates

$$u(x,t) \approx \widehat{u}(x,t;\theta)$$

on the entire time interval. It does not produce a sequence of states through an explicit update rule. Instead, the optimizer adjusts one parameter vector so that the network approximately satisfies the PDE and data throughout spacetime.

Feature	Classical time marching	Global PINN
Representation	nodal states	continuous neural function
Time direction	sequential	simultaneous
CFL restriction	may apply	not directly
Linear solves	often required	no PDE system solve
Main cost	many time steps	nonlinear training
Dominant error	discretization/stability	optimization/generalization

Table 5.2: Conceptual comparison between time marching and global-in-time PINNs.

It is common to say that PINNs avoid CFL restrictions. This statement is true only in the narrow sense that no explicit time-stepping update is used, so there is no requirement of the form $\Delta t \le Ch^2$. However, the absence of a CFL condition does not make long-time learning easy. Optimization difficulty, loss imbalance, poor propagation of initial information, and insufficient representation of oscillations may become the dominant limitations.

5.8 Long-time dynamics and causal training

Global training over a large time interval can be difficult. A network may obtain a small average residual while remaining inaccurate in localized time regions. Errors near $t = 0$ can propagate, and highly oscillatory or chaotic systems can amplify small inaccuracies.

A useful principle is *causality*: earlier times should be learned before later times. This motivates several practical strategies.

5.8.1 Causal weighting

The time interval can be partitioned into subintervals and the loss weighted so that early-time residuals receive higher priority until they become accurate. Later intervals are then activated progressively.

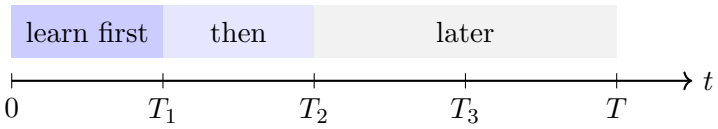


Figure 5.3: Causal or curriculum training emphasizes earlier time windows before later ones.

5.8.2 Time-domain decomposition

Instead of a single network on $[0, T]$, one can split the interval:

$$[0, T] = [T_0, T_1] \cup [T_1, T_2] \cup \cdots \cup [T_{K-1}, T_K].$$

Separate networks may be trained on the windows, with continuity constraints at the interfaces. This reduces the optimization difficulty of each subproblem. Related ideas include XPINNs, sequential PINNs, and other domain-decomposition strategies.

5.8.3 Curriculum and windowing

A simpler variant is to begin on a short interval $[0, T_1]$, then gradually enlarge the training window while reusing the previously trained network as initialization. In effect, the model learns progressively longer dynamics rather than facing the full difficulty at once.

5.8.4 Adaptive sampling in time

Residual-based adaptive sampling adds points in regions where the PDE residual is large. Sharp transients or fronts often require significantly more points than smooth time intervals.

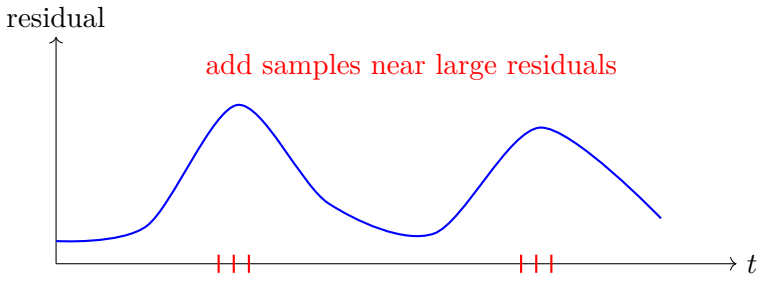


Figure 5.4: Residual-based sampling places additional training points where the current PINN violates the PDE most strongly.

5.9 Data-assisted PINNs and inverse problems

If observations are available,

$$\mathcal{D} = \{(x_i, t_i, u_i)\}_{i=1}^{N_d},$$

one may add

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |\widehat{u}(x_i, t_i; \theta) - u_i|^2. \tag{5.30}$$

The full loss becomes

$$\mathcal{L} = \lambda_f \mathcal{L}_{\text{PDE}} + \lambda_b \mathcal{L}_{\text{BC}} + \lambda_0 \mathcal{L}_{\text{IC}} + \lambda_d \mathcal{L}_{\text{data}}. \tag{5.31}$$

This is particularly useful when the data are sparse, noisy, or incomplete, because the governing equation regularizes the interpolation between observations.

A natural inverse problem is estimation of the diffusivity α in

$$u_t = \alpha u_{xx}.$$

If α is unknown, it can be treated as a trainable scalar together with the network parameters:

$$\mathcal{R}(x, t; \theta, \alpha) = \partial_t \widehat{u} - \alpha \partial_{xx} \widehat{u}.$$

One then minimizes

$$\mathcal{L}(\theta, \alpha) = \mathcal{L}_{\text{PDE}}(\theta, \alpha) + \mathcal{L}_{\text{BC}}(\theta) + \mathcal{L}_{\text{IC}}(\theta) + \mathcal{L}_{\text{data}}(\theta).$$

The data help identify the unknown parameter, while the PDE constrains the admissible solutions.

5.10 Other time-dependent PDEs

5.10.1 Wave equation

For

$$u_{tt} = c^2 u_{xx},$$

a PINN residual is

$$\mathcal{R} = \partial_{tt} \widehat{u} - c^2 \partial_{xx} \widehat{u}.$$

Because the equation is second order in time, two initial conditions are required:

$$u(x, 0) = u_0(x), \qquad u_t(x, 0) = v_0(x).$$

Wave equations are often more difficult than diffusion equations because oscillatory solutions may be poorly represented by standard neural networks and spectral bias can become important.

5.10.2 Reaction–diffusion equations

A general reaction–diffusion model has the form

$$u_t = D\Delta u + F(u).$$

The corresponding residual is

$$\mathcal{R} = \partial_t \widehat{u} - D\Delta \widehat{u} - F(\widehat{u}).$$

The diffusion term smooths the solution, while the reaction term can produce growth, decay, travelling fronts, or phase separation. Fisher–KPP and Allen–Cahn equations are common examples.

5.10.3 Burgers' equation

The viscous Burgers equation is

$$u_t + uu_x = \nu u_{xx}.$$

Its PINN residual is

$$\mathcal{R} = \partial_t \widehat{u} + \widehat{u}\,\partial_x \widehat{u} - \nu \partial_{xx} \widehat{u}.$$

For small viscosity, steep gradients or shock-like structures can develop. Such features make training more sensitive to sampling, network architecture, activation functions, and loss balancing. These issues motivate a deeper discussion of nonlinear PDEs and PINN training challenges.

5.11 Practical checklist

Before training a time-dependent PINN, it is useful to ask:

1. What is the spacetime domain $\Omega \times [0, T]$?
2. Are the initial and boundary data compatible?
3. Which first- and higher-order derivatives are needed in the residual?
4. How will interior, boundary, and initial points be sampled?
5. Have x and t been nondimensionalized or scaled to comparable ranges?
6. Are the loss terms numerically balanced?
7. Is a global-in-time model realistic, or should time be decomposed?
8. Is the solution oscillatory, stiff, multiscale, or sharply transient?
9. Are observational data available and should they enter the loss?
10. Is the goal a forward solution, parameter inference, or both?

5.12 Summary

Time-dependent PINNs approximate a continuous spacetime field $u(x, t)$ with a trainable neural function. Initial conditions, boundary conditions, and the PDE residual define the principal components of the training objective. The heat equation provides the simplest benchmark because its solution is smooth, its exact behavior is well understood, and classical time-stepping methods provide a clear comparison.

The main conceptual contrast is that classical solvers advance states sequentially in time, whereas a global PINN may learn all times simultaneously. This avoids explicit time-step stability restrictions in the usual sense, but introduces a difficult nonconvex optimization problem. For long-time or strongly transient dynamics, causal weighting, time decomposition, curriculum training, and adaptive sampling can be valuable. Time-dependent PINNs also provide a natural framework for combining PDEs with sparse measurements and for estimating unknown physical parameters.

Exercises

1. Derive the exact solution of the heat equation with initial condition $u_0(x) = \sin(2\pi x)$ and homogeneous Dirichlet boundary conditions.
2. Write the PINN residual and loss for the forced heat equation $u_t - \alpha u_{xx} = f(x,t)$ with nonzero Dirichlet data.
3. Construct a hard-constrained ansatz that satisfies both $u(x,0) = u_0(x)$ and homogeneous boundary conditions on $x = 0, 1$.
4. Derive the explicit Euler finite-difference scheme for the heat equation and identify the usual stability restriction.
5. Explain why the phrase "PINNs have no CFL condition" can be misleading.
6. For the wave equation, formulate the full PINN loss including both initial displacement and initial velocity.
7. Suppose α in the heat equation is unknown. Explain what information is needed for it to be identifiable from data.

Bibliography

[1] I. E. Lagaris, A. Likas, and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," *IEEE Transactions on Neural Networks*, 9(5), 987–1000, 1998.

[2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378, 686–707, 2019.

[3] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, 3, 422–440, 2021.

[4] A. D. Jagtap and G. E. Karniadakis, "Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations," *Communications in Computational Physics*, 28, 2002–2041, 2020.

[5] S. Wang, S. Sankaran, and P. Perdikaris, "Respecting causality is all you need for training physics-informed neural networks," arXiv preprint, 2022.

[6] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, 2nd ed., Cambridge University Press, 2022.

Chapter 6

Nonlinear PDEs and Training Challenges

6.1 Overview and learning objectives

The previous lectures developed the main ingredients of a physics-informed neural network (PINN): a neural-network approximation of an unknown field, automatic differentiation of that approximation, and a loss function obtained by penalizing violations of a differential equation together with boundary and initial conditions. For linear problems, these ideas already raise interesting questions of approximation, optimization, and sampling. For nonlinear partial differential equations (PDEs), however, the interaction among these ingredients becomes substantially more delicate.

This lecture focuses on the training difficulties that arise when PINNs are applied to nonlinear PDEs. The one-dimensional viscous Burgers equation is used as the main running example because it combines nonlinear convection and diffusion in a mathematically simple model. It is rich enough to exhibit steep gradients, competing scales, and optimization pathologies, but simple enough that every component of the PINN formulation can be written explicitly.

By the end of the lecture, the reader should be able to:

- formulate nonlinear PDE residuals for PINNs;
- derive the PINN objective for the viscous Burgers equation;
- recognize common failure modes such as slow convergence, inaccurate solutions, and imbalance among loss terms;
- explain why the optimization problem may be poorly conditioned;
- understand the role of smoothness or spectral bias in learning shocks and layers;
- explain why collocation-point placement affects the quality of a PINN approximation;
- describe several practical mitigation strategies.

Throughout the notes, the notation $\mathcal{L}$ is used for loss functions, in keeping with the revised slide deck.

6.2 Why nonlinear PDEs are different

6.2.1 From linear models to nonlinear physics

Many equations in science and engineering are nonlinear. Examples include the Navier–Stokes equations of fluid mechanics, nonlinear wave equations, reaction–diffusion systems, nonlinear elasticity, porous-medium models, and conservation laws. Nonlinearity can appear through products of the unknown and its derivatives, through nonlinear constitutive laws, or through nonlinear source terms.

A PINN approximates the solution by a parametrized function

$$u(x,t) \approx u_\theta(x,t),$$

where θ collects the weights and biases of the neural network. The PDE, initial conditions, boundary conditions, and possibly observational data are then converted into an optimization problem

$$\min_\theta \mathcal{L}(\theta).$$

The apparent simplicity of this formulation can obscure a central difficulty: a correct PDE residual does not automatically produce an easy optimization problem.

Key point

For nonlinear PDEs, the central question is often not only whether the network is expressive enough to represent the solution, but whether gradient-based training can reliably locate parameters that represent that solution.

Consider the contrast between the heat equation

$$u_t = \alpha u_{xx}$$

and Burgers' equation

$$u_t + uu_x = \nu u_{xx}.$$

The heat equation is linear in u, superposition holds, and diffusion tends to smooth the solution. Burgers' equation contains the nonlinear product uu_x, superposition fails, and steep gradients may form when the viscosity ν is small.

The nonlinear setting introduces several additional sources of difficulty:

- steep fronts, boundary layers, or internal layers;
- multiple spatial and temporal scales;
- sensitive dependence on parameters;
- nonlinear coupling among residual terms;
- optimization landscapes with poorly scaled directions.

A useful mental model is

$$\text{nonlinear PDE} \longrightarrow \text{residual } \mathcal{R}_\theta \longrightarrow \text{loss } \mathcal{L}(\theta) \longrightarrow \text{optimization}.$$

Every arrow matters. A mathematically sensible PDE may lead to a residual whose derivatives are hard to optimize; a sensible residual may be poorly sampled; and a good sample may still lead to badly balanced loss components.

6.3 The viscous Burgers equation

6.3.1 Equation and interpretation

The one-dimensional viscous Burgers equation is

$$u_t + uu_x = \nu u_{xx}, \qquad x \in [-1, 1], \quad t \in [0, T], \tag{6.1}$$

with initial and boundary data such as

$$u(x, 0) = u_0(x), \qquad u(-1, t) = g_L(t), \qquad u(1, t) = g_R(t).$$

The terms in (6.1) have clear interpretations:

- u_t describes time evolution;
- uu_x is nonlinear convection or transport;
- νu_{xx} is diffusion, with $\nu > 0$ the viscosity.

The competition between convection and diffusion controls the qualitative behavior. For relatively large ν, diffusion smooths the solution strongly. For small ν, convection becomes more important and the solution may develop a narrow transition region with a very steep slope.

A commonly used benchmark is

$$u_t + uu_x = \nu u_{xx}, \qquad x \in [-1, 1], \quad t \in [0, 1], \tag{6.2}$$

with

$$u(x, 0) = -\sin(\pi x), \qquad u(-1, t) = u(1, t) = 0.$$

For any fixed $\nu > 0$, the viscous solution remains smooth, but for small viscosity it can vary sharply in a narrow region. Such features are challenging for a globally trained multilayer perceptron because the relevant information is highly localized.

6.3.2 A schematic illustration

Figure 6.1 shows a purely schematic comparison between a diffusion-dominated profile and a steep front. It is intended to illustrate the approximation challenge rather than to depict a particular Burgers solution.

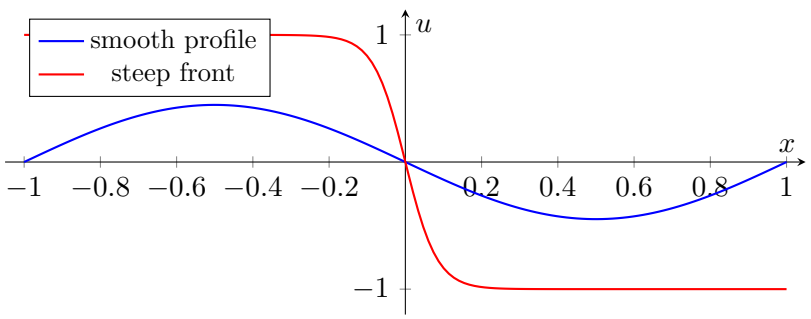


Figure 6.1: Schematic comparison of a smooth profile and a steep transition layer.

A classical finite-difference, finite-volume, or finite-element solver would normally choose a spatial grid, a time grid, a time-stepping rule, and possibly a stabilization strategy. A PINN instead tries to approximate the whole spacetime function $u(x,t)$ at once and trains against a residual. This eliminates an explicit time-stepping loop in a global-in-time formulation, but it does not eliminate numerical difficulty; the difficulty reappears in approximation, sampling, and optimization.

6.4 PINN formulation for Burgers' equation

6.4.1 Neural-network approximation and residual

Let

$$u(x,t) \approx u_\theta(x,t),$$

where the neural network takes (x,t) as input and produces a scalar output. Automatic differentiation is used to compute

$$\partial_t u_\theta, \qquad \partial_x u_\theta, \qquad \partial_{xx} u_\theta.$$

Substitution into (6.1) gives the strong-form residual

$$\mathcal{R}_f(x,t;\theta) = \partial_t u_\theta(x,t) + u_\theta(x,t)\,\partial_x u_\theta(x,t) - \nu\,\partial_{xx} u_\theta(x,t). \tag{6.3}$$

If $\mathcal{R}_f = 0$ everywhere and the initial and boundary conditions are satisfied exactly, then u_θ solves the PDE.

In practice, of course, the residual is not enforced everywhere. We introduce three finite sets of sample points:

$$\mathcal{X}_f = \{(x_i,t_i)\}_{i=1}^{N_f} \subset (-1,1)\times(0,T),$$
$$\mathcal{X}_0 = \{(x_i,0)\}_{i=1}^{N_0}, \qquad \mathcal{X}_b = \{(-1,t_j),(1,t_j)\}_{j=1}^{N_b}.$$

The first set contains interior collocation points, the second initial-condition points, and the third boundary points.

6.4.2 Loss components

The PDE residual loss is

$$\mathcal{L}_f(\theta) = \frac{1}{N_f} \sum_{(x_i,t_i)\in\mathcal{X}_f} \left|\mathcal{R}_f(x_i,t_i;\theta)\right|^2. \tag{6.4}$$

The initial-condition loss is

$$\mathcal{L}_0(\theta) = \frac{1}{N_0} \sum_{(x_i,0)\in\mathcal{X}_0} \left|u_\theta(x_i,0) - u_0(x_i)\right|^2, \tag{6.5}$$

and the boundary loss may be written as

$$\mathcal{L}_b(\theta) = \frac{1}{N_b} \sum_{(x_j,t_j)\in\mathcal{X}_b} \left|u_\theta(x_j,t_j) - g(x_j,t_j)\right|^2, \tag{6.6}$$

where g denotes the prescribed boundary data on the two endpoints.

The standard soft-constraint objective is then

$$\mathcal{L}(\theta) = \lambda_f \mathcal{L}_f(\theta) + \lambda_0 \mathcal{L}_0(\theta) + \lambda_b \mathcal{L}_b(\theta), \tag{6.7}$$

where $\lambda_f, \lambda_0, \lambda_b > 0$.

The weights in (6.7) are not merely cosmetic. They change the geometry of the optimization problem. If one term has much larger magnitude or a much larger gradient than the others, it may dominate the parameter update. Consequently, changing the loss weights can change the learned solution even when the network architecture and collocation points are unchanged.

Practical warning

A small value of the total loss $\mathcal{L}$ can hide an unacceptably large value of one component. During training, monitor $\mathcal{L}_f$, $\mathcal{L}_0$, and $\mathcal{L}_b$ separately.

The training loop can be summarized as follows:

1. evaluate $u_\theta(x,t)$ at collocation, initial, and boundary points;
2. use automatic differentiation to compute the derivatives needed by the PDE;
3. assemble $\mathcal{R}_f$;
4. compute $\mathcal{L}_f$, $\mathcal{L}_0$, $\mathcal{L}_b$, and $\mathcal{L}$;
5. differentiate $\mathcal{L}$ with respect to θ;
6. update θ using a gradient-based optimizer.

6.4.3 A generalized nonlinear residual

For the convection–diffusion–reaction equation

$$u_t + a(u)u_x = \nu u_{xx} + r(u),$$

the corresponding PINN residual is

$$\mathcal{R}_f(x,t;\theta) = \partial_t u_\theta + a(u_\theta)\partial_x u_\theta - \nu \partial_{xx} u_\theta - r(u_\theta).$$

This simple example illustrates a general rule: the residual is obtained by substituting the neural approximation into the governing equation and moving all terms to one side.

6.5 What can go wrong?

Even when the residual and loss are formulated correctly, training may fail. Failure can appear in several ways.

6.5.1 Slow convergence and plateaus

The total loss can decrease rapidly at first and then plateau. A plateau does not necessarily mean that the network lacks sufficient expressive power. It may indicate very small gradients, badly scaled gradients, competing objectives, or an unfavorable region of parameter space.

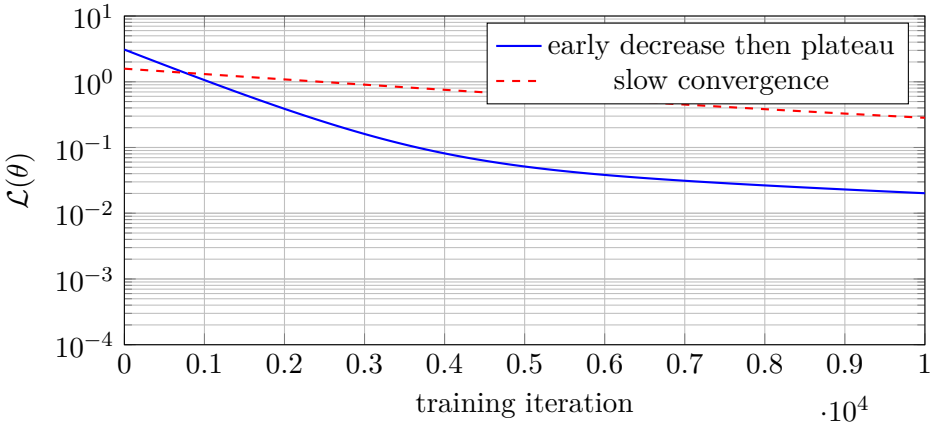


Figure 6.2: Two schematic loss histories illustrating common training behavior.

6.5.2 A smooth but wrong solution

A network can produce a visually plausible approximation that misses a physically important sharp feature. For example, it may fit most of the domain while smearing a narrow front. Because the residual loss is averaged over collocation points, a small region of large error may have limited influence unless that region is sampled adequately.

This observation is important conceptually. The empirical residual

$$\mathcal{L}_f(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{R}_f(x_i, t_i; \theta)|^2$$

is only a finite-sample approximation to a continuous requirement. A small empirical residual does not by itself guarantee a small solution error throughout the spacetime domain.

6.5.3 Objective imbalance

Suppose

$$\mathcal{L} = \lambda_f \mathcal{L}_f + \lambda_0 \mathcal{L}_0 + \lambda_b \mathcal{L}_b.$$

If $\lambda_f \mathcal{L}_f$ or its gradient dominates, the optimizer may reduce the PDE residual while fitting the initial data poorly. Conversely, if $\lambda_0 \mathcal{L}_0$ dominates, the network may reproduce the initial profile accurately while failing to propagate the correct later-time dynamics.

This phenomenon is better understood at the gradient level:

$$\nabla_\theta \mathcal{L} = \lambda_f \nabla_\theta \mathcal{L}_f + \lambda_0 \nabla_\theta \mathcal{L}_0 + \lambda_b \nabla_\theta \mathcal{L}_b. \tag{6.8}$$

The three component gradients can differ greatly in norm and may point in conflicting directions.

6.5.4 Poor generalization between collocation points

Because the PDE is enforced only at finitely many points, the network may have low residual on the training set but larger residual between those points. The risk is particularly serious for localized layers or shocks. Uniformly adding more points may help, but it can also waste computational effort in regions where the current approximation is already accurate.

6.6 Why the optimization problem is difficult

PINN training solves a high-dimensional nonconvex problem

$$\theta^\star \in \arg\min_\theta \mathcal{L}(\theta).$$

The objective depends on the neural-network parameters not only through the network output but also through first and second derivatives of that output. In a nonlinear PDE, products such as $u_\theta \partial_x u_\theta$ couple these quantities further.

6.6.1 Gradient descent and competing directions

The basic gradient-descent update is

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}(\theta_k),$$

where $\eta > 0$ is the learning rate. Combining this with (6.8) shows why loss balancing matters: the direction actually followed by the optimizer is a weighted vector sum of the gradients associated with different physical constraints.

If the vectors are badly scaled or point in incompatible directions, improving one part of the loss may temporarily worsen another. The optimizer then moves through a compromise landscape rather than minimizing each physical constraint independently.

6.6.2 Training stiffness

A useful continuous-time idealization of gradient descent is

$$\frac{\mathrm{d}\theta}{\mathrm{d}\tau} = -\nabla_\theta \mathcal{L}(\theta),$$

where τ denotes optimization time rather than physical time. Different modes of the error can decay at very different rates. Some constraints may be learned rapidly while others change only slowly. This resembles stiffness in the numerical integration of differential equations, where different dynamical scales coexist.

6.6.3 Neural tangent kernel intuition

One interpretation of PINN training comes from neural tangent kernel (NTK) analysis. In a suitable regime, the evolution of the network outputs can be approximated by a kernel system. Different modes are associated with different effective eigenvalues; modes corresponding to small eigenvalues evolve slowly. For PINNs, different loss components can therefore converge on very different time scales. This viewpoint helps explain why adaptive weighting and learning-rate strategies can improve training in some problems [5, 4].

6.6.4 Differential operators and conditioning

The Burgers residual

$$\mathcal{R}_f = \partial_t u_\theta + u_\theta \partial_x u_\theta - \nu \partial_{xx} u_\theta$$

contains both derivatives and a nonlinear product. Differentiation amplifies high-frequency components, while second derivatives can increase sensitivity to network parameters. Thus

residual minimization can be substantially more ill-conditioned than ordinary supervised data fitting.

This issue becomes more pronounced for higher-order PDEs, stiff systems, or equations with widely separated physical coefficients. Nondimensionalization and input/output scaling are therefore often important before any sophisticated optimization strategy is considered.

6.6.5 Useful diagnostics

It is rarely sufficient to inspect only the total objective. Useful diagnostics include

- the component losses $\mathcal{L}_f, \mathcal{L}_0, \mathcal{L}_b$;
- gradient norms $\|\nabla_\theta \mathcal{L}_f\|$, $\|\nabla_\theta \mathcal{L}_0\|$, and $\|\nabla_\theta \mathcal{L}_b\|$;
- the residual on an independent validation set;
- the solution error against a trusted reference solution, when available;
- sensitivity to random initialization;
- sensitivity to the collocation sample.

6.7 Smoothness and spectral bias

6.7.1 Low frequencies are often learned first

Standard multilayer perceptrons often fit smooth, low-frequency components of a target function before fine-scale or high-frequency components. This behavior is commonly called *spectral bias* [5]. A simple illustration is

$$f(x) = \sin x + 0.2 \sin(20x).$$

The low-frequency component defines the broad shape, while the small high-frequency term adds rapid oscillations. During training, the broad trend may be captured well before the fine oscillations.

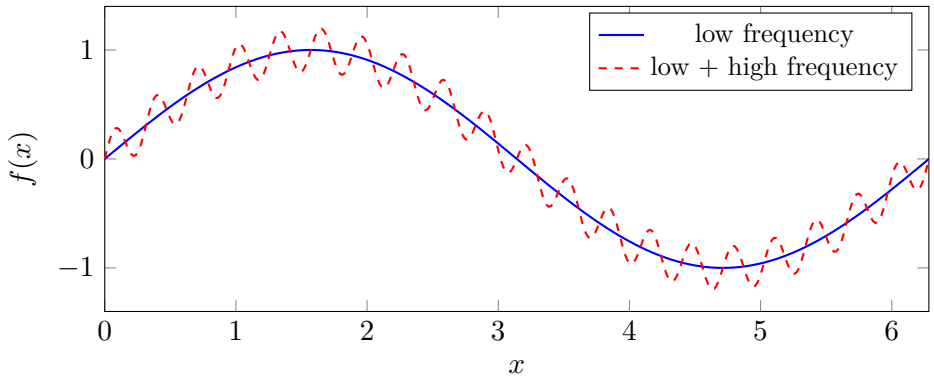


Figure 6.3: A low-frequency function and a perturbation containing a high-frequency component.

For Burgers' equation with small viscosity, a steep transition carries significant high-frequency information. If the network learns the smooth background first, the front can remain smeared

for a long time. If the residual is also undersampled in the front region, the optimizer may receive little information encouraging a sharper transition.

6.7.2 Activation functions

Activation choice matters because the PDE residual involves derivatives. ReLU networks are piecewise linear, so their classical second derivative is zero almost everywhere and undefined at kink points. This makes ReLU unattractive for straightforward strong-form PINNs for second-order PDEs. Smooth activations such as tanh are therefore common in classical PINN implementations.

Smoothness alone is not sufficient, however. A smooth activation can still exhibit spectral bias or optimization difficulty. Activation selection must be considered together with network depth, initialization, input scaling, and the character of the PDE solution.

6.7.3 Fourier features

One way to improve representation of high-frequency functions is to map the input into oscillatory features, for example

$$x \longmapsto [\sin(Bx), \cos(Bx)].$$

Fourier-feature embeddings can reduce the tendency of a standard MLP to learn only low frequencies first and have been effective for low-dimensional coordinate networks [7]. They can be useful for oscillatory waves or sharp transitions, although they do not remove optimization or sampling problems.

6.7.4 Sinusoidal networks

SIREN-type networks use sinusoidal activations of the form

$$z \longmapsto \sin(\omega_0 z).$$

Such networks are designed to represent fine-scale spatial variation and derivatives efficiently [8]. Their performance depends strongly on initialization and frequency scaling, so they should not be viewed as a universal replacement for tanh-based networks.

6.7.5 Domain decomposition

Another strategy is to replace one global network by several local networks. Suppose the spacetime domain is decomposed as

$$\Omega \times (0, T) = D_1 \cup D_2 \cup \cdots \cup D_M.$$

A separate network u_{θ_m} is trained on each subdomain, while continuity or flux conditions are imposed across interfaces. Local networks may resolve sharp features more easily because each network handles a smaller region and a narrower range of behavior. XPINNs are one example of this philosophy [7].

6.8 Sampling collocation points

6.8.1 Why sampling matters

The continuous PDE is represented during training only through the finite set

$$\mathcal{X}_f = \{(x_i, t_i)\}_{i=1}^{N_f}.$$

If an important region is undersampled, a large residual there may have almost no effect on $\mathcal{L}_f$. This is particularly relevant for shocks, narrow internal layers, or boundary layers because these features occupy a small portion of the full domain.

6.8.2 Uniform sampling

Uniform grids or approximately uniform point clouds are simple and reproducible. They are often a reasonable baseline. Their weakness is inefficiency: many points may be placed in smooth regions while too few points lie near sharp transitions.

6.8.3 Random and quasi-random sampling

Common alternatives include independent random sampling, Latin hypercube sampling, and low-discrepancy sequences such as Sobol or Halton points. Randomized designs can avoid some grid artifacts and are convenient in moderate or high input dimension. However, random sampling alone does not guarantee adequate resolution of a localized feature.

6.8.4 Residual-based adaptive refinement

A natural adaptive procedure is:

1. train an initial PINN on a baseline collocation set;
2. evaluate $|\mathcal{R}_f(x, t; \theta)|$ on a large candidate set;
3. identify points with large residual;
4. add some of those points to $\mathcal{X}_f$;
5. continue training.

Symbolically,

$$\mathcal{X}_f \leftarrow \mathcal{X}_f \cup \{\text{candidate points with large } |\mathcal{R}_f|\}.$$

The logic is similar to adaptive mesh refinement in classical numerical methods: computational effort is concentrated where the current approximation appears least accurate.

Residual-based refinement should nevertheless be interpreted carefully. A residual is not the same as the true solution error, and a residual estimator can itself be misleading if the current network is poor. It is therefore useful to combine adaptive residual sampling with independent validation and, where possible, comparison against a trusted numerical reference.

6.8.5 Boundary and initial sampling

For time-dependent nonlinear PDEs, initial and boundary data deserve separate attention. The initial condition determines the evolution, and boundary data can create or interact with layers. A weakly enforced initial condition can contaminate the entire solution. Separate validation sets should therefore be used for the interior PDE, initial condition, and boundary conditions.

6.9 Advanced strategies

6.9.1 Loss balancing

Several strategies have been proposed for balancing the components of $\mathcal{L}$:

- manual tuning of $\lambda_f, \lambda_0, \lambda_b$;
- nondimensionalization and normalization of residual terms;
- adaptive weighting based on gradient norms;
- learning-rate annealing or NTK-inspired approaches;
- self-adaptive weights associated with individual collocation points.

There is no universal rule that works for every PDE. Loss balancing remains problem dependent and is still an active research topic [4].

6.9.2 Curriculum and continuation

Continuation methods solve a sequence of related problems of increasing difficulty. For Burgers' equation, one might begin with a relatively large viscosity and then gradually reduce ν. Other possibilities include starting on a short time interval and extending it, increasing the strength of a nonlinear source term gradually, or moving from coarse to fine collocation sets.

The motivation is that the optimizer may first find a useful region of parameter space for the easier problem and then track a good solution as the problem becomes harder. Krishnapriyan et al. discuss curriculum-style training in the context of PINN failure modes [10].

6.9.3 Causal and time-marching PINNs

A global-in-time PINN attempts to represent the entire trajectory over $[0, T]$ at once. For long intervals or strongly nonlinear dynamics, this can be difficult. Alternatives include

- training on a sequence of time windows;
- using causal weights that emphasize earlier times first;
- recurrent or sequence-to-sequence formulations;
- hybrid methods that combine neural approximations with classical time integrators.

These methods reintroduce some temporal structure rather than treating all spacetime points symmetrically.

6.9.4 Weak and conservative formulations

Strong-form residuals require pointwise derivatives and can be poorly suited to solutions with discontinuities or nearly discontinuous layers. For conservation laws and shock-dominated problems, alternatives include weak-form PINNs, conservative formulations, finite-volume-inspired neural methods, and discontinuity-aware domain decomposition.

The broader lesson is that the neural-network architecture is only one design choice. The mathematical formulation of the PDE constraint should respect the underlying structure of the equation.

6.10 A practical workflow for nonlinear PINNs

For a Burgers-type problem, a reasonable workflow is:

1. begin with a moderate viscosity and a relatively short time interval;
2. use a smooth activation when second derivatives appear in the strong-form residual;
3. nondimensionalize or scale the variables if physical scales differ substantially;
4. monitor $\mathcal{L}_f$, $\mathcal{L}_0$, and $\mathcal{L}_b$ separately;
5. validate the PDE residual on an independent dense or randomly sampled set;
6. inspect the solution specifically near steep gradients and layers;
7. add adaptive collocation points in regions of large residual if necessary;
8. test more than one random seed;
9. compare with a classical numerical method or manufactured solution whenever possible;
10. record the network architecture, optimizer, learning rate, loss weights, sample sizes, random seeds, and stopping criterion.

Interpretation

The main sources of PINN error can be thought of as interacting approximation, optimization, and sampling errors. Improving one component does not automatically improve the others.

6.11 Summary

Nonlinear PDEs provide an important test of the strengths and limitations of physics-informed neural networks. Burgers' equation illustrates the main issues clearly. The PINN residual is easy to write down, yet the resulting optimization problem can still be difficult because of nonlinear coupling, steep solution features, imbalanced objectives, and finite collocation sampling.

The main conclusions are:

- a correct PINN formulation does not guarantee easy training;

- the composite loss $\mathcal{L}$ should be analyzed through its components $\mathcal{L}_f$, $\mathcal{L}_0$, and $\mathcal{L}_b$;
- small empirical loss does not by itself guarantee small continuous solution error;
- gradient imbalance and conditioning can cause very different convergence rates among constraints;
- spectral bias can make steep or oscillatory features difficult to learn;
- sampling is part of the numerical method, not a secondary implementation detail;
- adaptive sampling, better feature representations, domain decomposition, and continuation strategies can improve difficult cases;
- validation against independent points and trusted reference solutions is essential.

The guiding question for nonlinear PINNs is therefore not simply

Can the neural network represent the solution?

but also

Can the training procedure find that solution reliably?

6.12 Exercises

1. For
$$u_t + a(u)u_x = \nu u_{xx} + r(u),$$
derive the strong-form residual for a PINN u_θ. Identify which terms become difficult to approximate when u develops a steep layer.

2. Suppose $\mathcal{L}_f = 10^{-4}$, $\mathcal{L}_0 = 10^{-1}$, and $\mathcal{L}_b = 10^{-5}$. Explain why the total loss may be misleading if equal weights are used. Propose two diagnostic quantities in addition to $\mathcal{L}$.

3. Consider the benchmark Burgers problem with small viscosity. Design a residual-based adaptive collocation procedure. State how many candidate points you would evaluate, how you would select new points, and when you would stop refinement.

4. Explain why a ReLU network is problematic for a strong-form second-order PDE residual. How could a weak formulation change this consideration?

5. Consider a network that approximates $f(x) = \sin x + 0.2\sin(20x)$. Describe an experiment that would demonstrate spectral bias during training.

6. Compare a global-in-time PINN with a time-window PINN. What errors can be introduced at interfaces between time windows, and how might these be controlled?

Bibliography

[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378, 686–707, 2019.

[2] G. E. Karniadakis et al., "Physics-informed machine learning," *Nature Reviews Physics*, 3, 422–440, 2021.

[3] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney, "Characterizing possible failure modes in physics-informed neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[4] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, 43(5), A3055–A3081, 2021.

[5] S. Wang, X. Yu, and P. Perdikaris, "When and why PINNs fail to train: A neural tangent kernel perspective," arXiv:2007.14527, 2020.

[6] N. Rahaman et al., "On the spectral bias of neural networks," *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[7] M. Tancik et al., "Fourier features let networks learn high frequency functions in low dimensional domains," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] V. Sitzmann et al., "Implicit neural representations with periodic activation functions," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[9] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, "DeepXDE: A deep learning library for solving differential equations," *SIAM Review*, 63(1), 208–228, 2021.

[10] A. D. Jagtap, K. Kawaguchi, and G. E. Karniadakis, "Adaptive activation functions accelerate convergence in deep and physics-informed neural networks," *Journal of Computational Physics*, 404, 109136, 2020.

[11] A. D. Jagtap and G. E. Karniadakis, "Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework," *Communications in Computational Physics*, 28(5), 2002–2041, 2020.

Chapter 7

PINNs versus Classical Numerical Methods

7.1 Motivation: choosing the right computational tool

At this stage of a course on physics-informed neural networks, it is natural to ask a broader question than "how do we build a PINN?" The more important scientific question is:

When should we use a PINN, and when should we prefer a classical numerical method?

There is no universally best PDE solver. The answer depends on the problem being solved, the information available, the required accuracy, the computational budget, and whether the goal is a single solution or a reusable model. A fair comparison therefore has to distinguish several settings:

- a standard forward problem with fully known coefficients;
- an inverse problem with unknown parameters or fields;
- a problem with sparse or noisy measurements;
- a parameterized family of PDEs requiring many repeated evaluations;
- a real-time or digital-twin setting in which fast online evaluation matters;
- a high-accuracy engineering simulation in which rigorous error control is essential.

The main principle of this lecture is simple: *choose the method that matches the scientific question.* PINNs are not intended to replace every classical solver. Their main appeal is that they offer a flexible way to combine differential equations, boundary conditions, measurements, and unknown parameters inside one optimization problem.

7.2 Two computational philosophies

7.2.1 The PINN workflow

A PINN represents the unknown field by a trainable neural network, for example

$$u(x,t) \approx u_\theta(x,t),$$

with parameters θ. Automatic differentiation is used to evaluate the derivatives appearing in the governing equation, and the governing constraints are converted into a loss function such as

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_b \mathcal{L}_{\text{BC}}(\theta) + \lambda_d \mathcal{L}_{\text{data}}(\theta).$$

The optimizer adjusts θ so that the PDE residual, the boundary or initial conditions, and possibly observational data are all approximately satisfied [3].

Figure 7.1 summarizes this process.

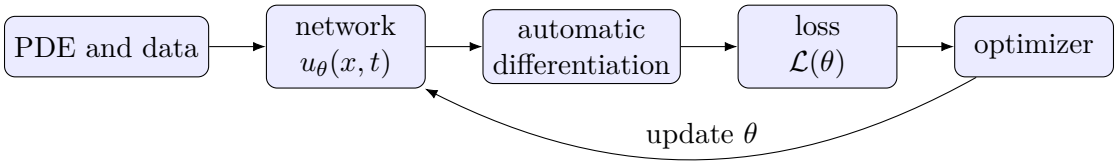


Figure 7.1: A typical PINN workflow.

7.2.2 The classical numerical workflow

Classical PDE solvers usually proceed differently. One first discretizes the spatial domain using a mesh or grid, derives a discrete operator, forms a linear or nonlinear algebraic system, and solves for nodal values or expansion coefficients. A schematic pipeline is

$$\text{domain} \longrightarrow \text{mesh/grid} \longrightarrow \text{discrete operator} \longrightarrow \text{algebraic system} \longrightarrow \text{numerical solution}.$$

The unknown is therefore usually a finite vector of coefficients rather than a neural-network parameter vector.

The difference between the two approaches is not merely one of software. Classical numerical methods are designed around numerical approximation of differential operators, weak formulations, local conservation, stability, and linear algebra. PINNs instead replace much of that pipeline by a nonlinear optimization problem over a parameterized function class.

7.3 A review of classical numerical methods

7.3.1 Finite difference methods

Finite difference methods approximate derivatives directly on a structured grid. For example,

$$\frac{\partial^2 u}{\partial x^2}(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}.$$

This construction is easy to understand and very efficient on simple geometries. On rectangular domains, finite difference stencils lead to sparse algebraic systems and highly optimized implementations. Their principal limitation is geometric flexibility: irregular boundaries, interfaces, and complicated domains usually require additional treatment [10].

7.3.2 Finite element methods

The finite element method represents the approximate solution in a finite-dimensional space,

$$\widehat{u}(x) = \sum_{j=1}^{N} U_j \phi_j(x),$$

where the basis functions ϕ_j are defined on a mesh. Rather than enforcing the PDE pointwise, FEM is built from a weak or variational formulation. This makes it particularly effective for complex geometry, heterogeneous coefficients, nontrivial boundary conditions, and local mesh refinement. It also comes with a mature mathematical theory of stability and error estimation [8, 9].

7.3.3 Finite volume methods

Finite volume methods are designed around integral conservation laws. For a control volume K, one writes a balance of the form

$$\frac{d}{dt}\int_K u\,dx + \int_{\partial K} F(u)\cdot n\,ds = \int_K s\,dx.$$

The method therefore builds local conservation into the discretization. This is particularly important for hyperbolic conservation laws and computational fluid dynamics, especially when shocks or steep fronts are present [11].

7.3.4 Spectral methods

Spectral methods use global basis functions,

$$u_N(x) = \sum_{k=0}^{N} a_k \phi_k(x),$$

with Fourier, Chebyshev, or related bases. For sufficiently smooth solutions on simple geometries, these methods can achieve very high accuracy with relatively few degrees of freedom. Their limitations include geometric flexibility and the treatment of discontinuities, where Gibbs-type oscillations may appear [12].

7.3.5 Why classical methods remain the default in many settings

Classical methods benefit from decades of theory and implementation. Among their strengths are

- rigorous convergence theory;
- a priori and a posteriori error estimates;
- sparse matrix structures;
- mature preconditioners and multigrid methods;
- local refinement and adaptive meshing;
- large-scale parallel implementations;
- industrial validation and established software ecosystems.

For a known forward PDE with a reliable existing discretization, these advantages are substantial. This is why a classical solver is usually the appropriate starting point for a routine forward simulation.

7.4 Accuracy: what should actually be compared?

Accuracy is not a single number. Depending on the application, one may care about

- pointwise error $|u(x) - \hat{u}(x)|$;
- an L^2 error $\|u - \hat{u}\|_{L^2(\Omega)}$;
- an energy or H^1 error;
- a residual norm $\|\mathcal{N}[\hat{u}] - f\|$;
- an engineering quantity of interest such as drag, lift, flux, or averaged temperature;
- error at sensor locations or future times.

An important warning is that a small PINN training loss does not by itself imply a small solution error. The loss measures satisfaction of the selected residuals at the selected samples with the selected weights. The relationship between that objective and the desired solution norm depends on the PDE, the sampling, the function class, and the stability of the underlying problem.

7.4.1 Classical error decomposition

A simplified way to think about the error of a classical method is

$$\|u - \widehat{u}\| \lesssim \underbrace{\text{approximation error}}_{\text{mesh or basis}} + \underbrace{\text{algebraic error}}_{\text{iterative solve}} + \underbrace{\text{roundoff error}}_{\text{floating point}}.$$

For a stable method and sufficiently smooth solution, refinement may produce a predictable rate such as

$$\|u - \widehat{u}\| \leq Ch^p \|u\|_{H^{p+1}},$$

where h is a mesh scale and p is related to the approximation order.

7.4.2 A useful PINN error picture

For PINNs, the sources of error are different. A useful informal decomposition is

$$\begin{aligned}\|u - u_\theta\| \lesssim\ & \underbrace{\text{approximation error}}_{\text{architecture}} + \underbrace{\text{optimization error}}_{\text{training}} \\ & + \underbrace{\text{generalization error}}_{\text{sampling}} + \underbrace{\text{modelling/data error}}_{\text{noise or mismatch}}.\end{aligned}$$

In practice, training failure and poor collocation coverage can be as important as the approximation capacity of the neural network. This is one reason why PINN performance can be less predictable than classical mesh-refinement studies [10, 11].

7.5 Computational cost: offline versus online

A useful comparison must distinguish *offline* and *online* cost.

Offline cost includes mesh generation, matrix assembly, solver setup, neural-network training, and hyperparameter tuning. Online cost is the cost of obtaining a prediction after the expensive setup phase has been completed.

For a single known forward PDE, a classical method is often far more economical because the PINN training cost is not amortized. The picture changes for repeated-query problems. If a trained model represents a parameterized family

$$(x, \mu) \longmapsto u(x; \mu),$$

then the training effort may be spread over many future evaluations.

Figure 7.2 illustrates the idea schematically. It is not a universal cost law; it only shows the possibility of a break-even point when a learned surrogate is reused sufficiently many times.

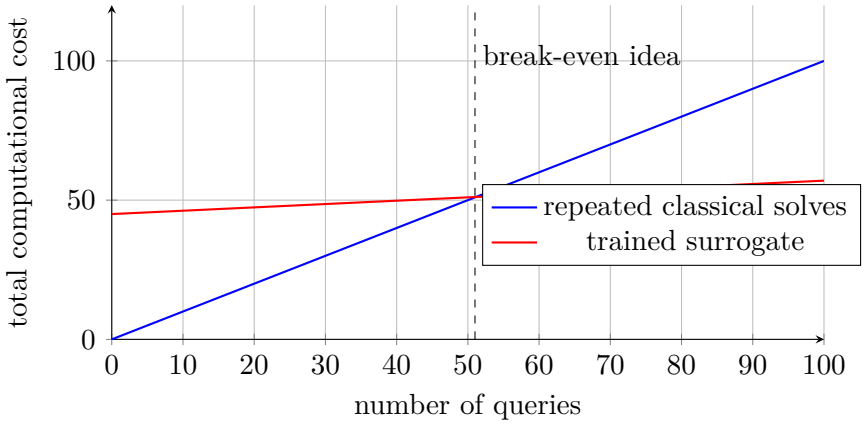


Figure 7.2: Conceptual offline/online cost trade-off.

This distinction is particularly important in design optimization, uncertainty quantification, parameter studies, and real-time decision support.

7.6 Mesh-based and mesh-free viewpoints

7.6.1 What a mesh provides

A mesh does more than place points in a domain. It represents local neighbourhood structure and supports discrete basis functions, fluxes, geometric approximation, and sparse coupling. A good mesh can provide

- sparse matrices;
- local support;
- natural boundary representation;
- local refinement;

- geometric accuracy;
- conservation and stability mechanisms.

Mesh generation can nevertheless be costly for complicated CAD geometry, moving interfaces, fracture, topology change, or repeated geometry optimization.

7.6.2 What "mesh-free" means for PINNs

PINNs do not require a connectivity graph. Instead, one samples interior and boundary points,

$$\{x_i\}_{i=1}^{N_f} \subset \Omega, \qquad \{x_j^b\}_{j=1}^{N_b} \subset \partial\Omega,$$

and evaluates the residual there.

This is convenient, but the phrase "mesh-free" should not be interpreted as "discretization-free." The continuous PDE constraint is still approximated by finitely many samples. Sampling quality therefore plays a role analogous to discretization quality: if important regions are undersampled, the training objective may not reveal large local errors.

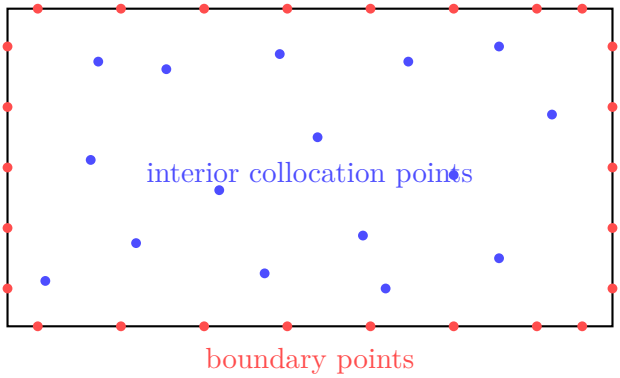


Figure 7.3: PINNs use sampled collocation points rather than mesh connectivity.

7.6.3 High-dimensional settings

A tensor-product grid with m points per coordinate in dimension d contains m^d points. For instance, $m = 10$ and $d = 20$ gives 10^{20} grid points. This motivates neural approximations in some high-dimensional PDE settings. However, avoiding a full grid does not remove the difficulties of high dimension: sampling, optimization, and representational complexity remain major challenges.

7.7 When PINNs are attractive

The strongest use cases are often those in which the task is more than a standard forward solve.

7.7.1 Inverse problems

Consider

$$-\nabla \cdot (k(x)\nabla u(x)) = f(x), \qquad x \in \Omega.$$

If k is known, this is a forward problem for u. If measurements of u are available while k is unknown, the problem becomes inverse. A PINN can represent both fields,

$$u \approx u_\theta, \qquad k \approx k_\eta,$$

and train them jointly using the PDE and the observations.

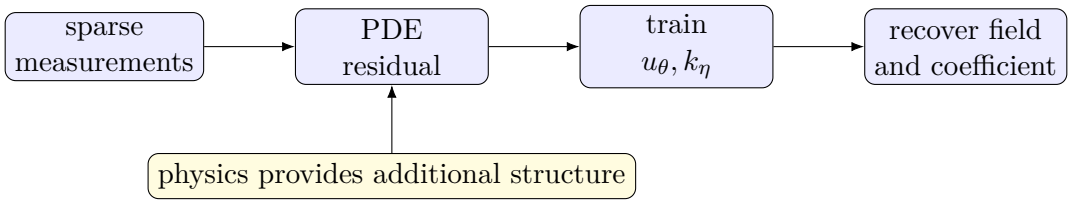


Figure 7.4: A PINN formulation for coefficient identification.

This formulation does not make an ill-posed inverse problem automatically well-posed; identifiability and data quality still matter. Its value is that physical and observational constraints can be optimized together.

7.7.2 Sparse measurements and data assimilation

If observations are available only at points (x_i, t_i),

$$\{(x_i, t_i, u_i)\}_{i=1}^{N_d},$$

a data-assisted PINN may use

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_b \mathcal{L}_{\text{BC}}(\theta) + \lambda_d \mathcal{L}_{\text{data}}(\theta),$$

with

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |u_\theta(x_i, t_i) - u_i|^2.$$

The physics term couples the sparse measurements through the governing PDE and helps constrain the solution away from sensor locations.

This viewpoint connects naturally with data assimilation: one combines a mechanistic model with measurements rather than relying exclusively on one or the other.

7.7.3 Digital twins and real-time use

A real-time system may tolerate significant offline training if the deployed model must answer queries very quickly. A common hybrid workflow is therefore

1. generate reliable information offline using experiments or classical solvers;
2. train a surrogate or physics-informed model;
3. use the learned model for fast online predictions.

7.7.4 Parameterized problems and uncertainty quantification

For a parameter-dependent PDE, the goal may be to approximate

$$(x, \mu) \mapsto u(x; \mu)$$

for many values of μ. PINNs can be extended to this setting, although operator-learning methods such as DeepONet and the Fourier Neural Operator are specifically designed for families of input functions or parameters [6, 7].

7.8 When PINNs struggle

PINNs are not equally effective for all PDEs. The lecture deck emphasizes several recurring challenges.

7.8.1 High-frequency solutions

For the Helmholtz equation

$$-\Delta u - k^2 u = f,$$

large k leads to rapidly oscillatory solutions. Standard neural networks often learn lower-frequency components first, a phenomenon associated with spectral bias [5]. Figure 7.5 gives a simple illustration.

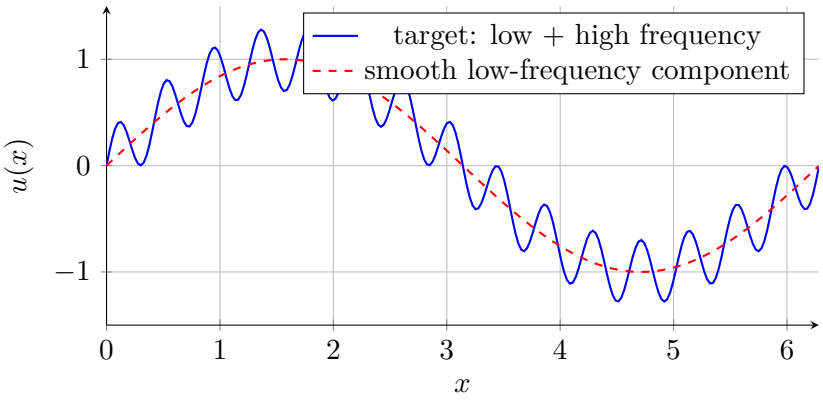


Figure 7.5: A schematic illustration of low-frequency learning preceding fine oscillations.

7.8.2 Multiscale problems

Consider

$$-\nabla \cdot (a(x/\varepsilon)\nabla u) = f, \qquad 0 < \varepsilon \ll 1.$$

The solution may contain both slow and fast spatial scales. Classical approaches include mesh refinement, homogenization, and multiscale basis construction. A single global network has to represent all scales simultaneously while the optimizer has to discover them from sampled residuals. This can be difficult without specialized architectures, features, or domain decomposition.

7.8.3 Sharp layers and shocks

Convection-dominated problems may contain thin boundary layers or shocks. Uniform collocation can undersample these localized features, and smooth neural approximations may smear steep transitions. Finite volume methods, shock-capturing discretizations, and adaptive mesh refinement were designed precisely for such behaviour and therefore remain strong baselines.

7.8.4 Stiffness and optimization

A PINN replaces a PDE solve by a nonlinear optimization problem. When the PDE is stiff, that optimization problem may also be badly conditioned. Different loss components can have strongly different scales, gradient directions may conflict, and the network can converge to a visually plausible but inaccurate solution [10, 11].

7.8.5 Very high accuracy requirements

If an engineering computation requires highly accurate quantities of interest, classical methods offer systematic refinement strategies, residual tolerances, error estimators, and in some settings stability or conservation guarantees. For such a standard forward problem, it is generally sensible to begin with a mature classical method.

7.8.6 Large industrial simulations

Large-scale industrial PDE solvers may involve millions or billions of unknowns, distributed sparse linear algebra, domain decomposition, multiphysics coupling, established meshing pipelines, and extensive validation. Current PINN practice is more naturally viewed as complementary to these pipelines than as a wholesale replacement.

7.9 Hybrid and advanced viewpoints

A productive direction is to avoid framing the subject as "PINNs versus numerical analysis." Several hybrid strategies are possible:

- generate training data with FEM or FVM and learn a surrogate;
- use a PINN only for inverse parameter estimation while a classical solver handles the forward model;
- use domain decomposition to localize neural training;
- combine neural approximations with conservation constraints;
- use variational or weak-form neural formulations.

7.9.1 Weak-form and variational PINNs

Strong-form PINNs evaluate a pointwise residual

$$\mathcal{N}[u_\theta](x) - f(x).$$

A weak formulation instead tests the residual against suitable functions v,

$$\int_\Omega (\mathcal{N}[u_\theta] - f) v \, dx.$$

This can reduce derivative order and connects naturally with finite element ideas. It can also be more appropriate when the desired solution is naturally understood in a weak sense.

7.9.2 Operator learning

A standard PINN often targets one solution field or a low-dimensional parameterized family. Operator-learning methods instead seek a map between functions,

$$f \mapsto u, \qquad \text{or} \qquad k(x) \mapsto u(x).$$

DeepONet and Fourier Neural Operator are two important examples [6, 7].

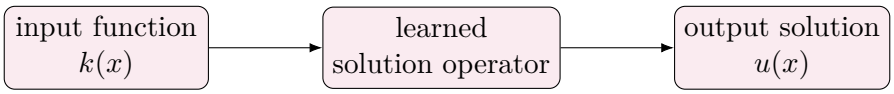


Figure 7.6: Operator learning targets a map between functions rather than only one solution field.

A simplified comparison is given in Table 7.1.

Table 7.1: PINNs and neural operators serve different but overlapping purposes.

PINN	**Neural operator**
Often learns one solution or a parameterized solution	Learns a solution operator
Can train without labelled solution data	Usually relies on training solution data
Uses physics residuals in the loss	Learns a function-to-function map
Natural for inverse and sparse-data problems	Natural for repeated-query surrogate problems

7.10 Benchmarking and reproducibility

Comparisons between PINNs and classical solvers are meaningful only when the experimental protocol is clear. Important questions include:

- Is wall-clock time, training time, or inference time being reported?
- Is the comparison for one solve or for many queries?
- Are the same accuracy metrics used?
- Are hyperparameters tuned with comparable effort?
- Is a trusted reference solution available?
- Are multiple random seeds reported for neural methods?

A reproducible PINN study should document the network architecture, activation, initialization, optimizer, learning-rate schedule, number of epochs, collocation strategy, loss weights, normalization, hardware, runtime, and random seeds. Code and data should be released when possible.

7.11 A practical decision framework

A compact decision guide is given in Table 7.2.

Table 7.2: Reasonable starting points for several problem classes.

Problem type	**Reasonable starting point**
2D Poisson problem with known coefficients	finite difference or finite element method
Heat equation with unknown diffusivity and few sensors	PINN or another inverse method
High-frequency Helmholtz problem	specialized finite element or spectral method
Parameterized Darcy flow with many samples	reduced-order model or neural operator
Fluid inverse problem from sparse velocity data	data-assisted PINN or hybrid approach
Shock-dominated conservation law	finite volume method, possibly with ML augmentation

Before choosing a PINN, ask:

1. What accuracy is required?
2. Is the PDE smooth, stiff, multiscale, or high-frequency?
3. Are there sparse measurements or unknown parameters?
4. Will the trained model be reused many times?
5. Is a strong classical baseline available?
6. Can the result be validated against exact, experimental, or high-fidelity data?

A useful decision tree is shown in Figure 7.7.

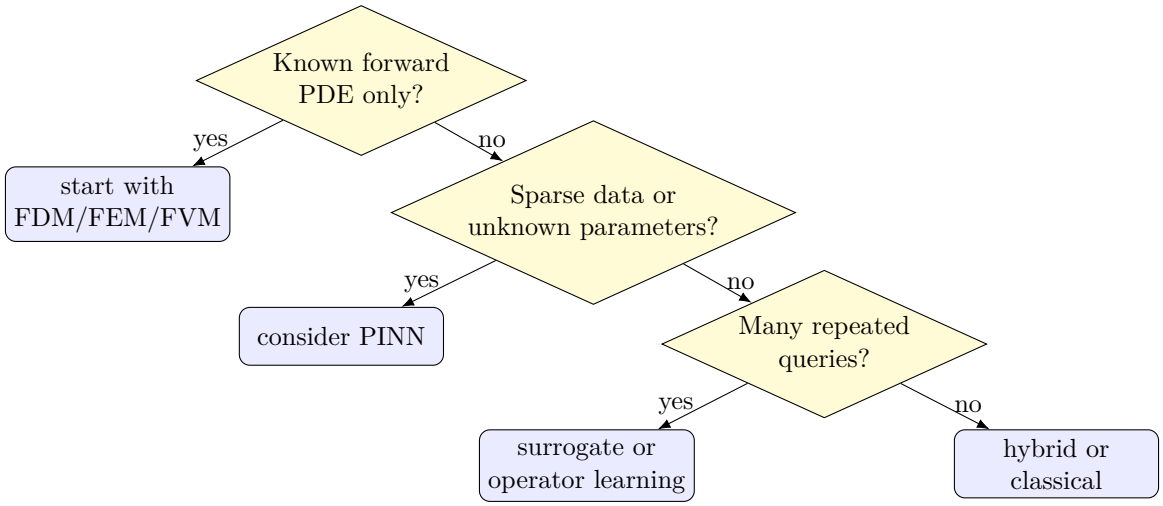


Figure 7.7: A simple starting-point decision tree.

7.12 Worked discussion examples

The following examples mirror the in-class discussion in the lecture.

Example 1: Poisson equation on a square

If the PDE, source term, and boundary conditions are all known and only one accurate solution is required, a classical FDM or FEM discretization is the natural starting point. The problem is smooth, low-dimensional, and supported by mature solvers and error theory.

Example 2: Unknown diffusivity from 20 sensors

Here the task is inverse. The field and the coefficient must be inferred from sparse measurements while respecting the PDE. A PINN is a plausible starting point because the physics and the sensor mismatch can be optimized simultaneously. It should still be compared against established inverse or Bayesian methods.

Example 3: High-frequency wave field

For a Helmholtz problem with large wave number, standard PINNs can be difficult to train because of oscillatory solutions and spectral bias. A specialized finite element or spectral solver is a stronger baseline.

Example 4: Ten thousand parameter queries

If many repeated evaluations are required, the key distinction is between solving each instance and learning a reusable surrogate. Reduced-order methods, operator learning, or a parameterized neural surrogate may be appropriate. The training cost is justified only if it is amortized over sufficiently many queries.

Example 5: Shock-dominated conservation law

Finite volume methods and shock-capturing schemes are designed for discontinuous or nearly discontinuous solutions and local conservation. A standard smooth PINN is therefore not the first choice, although hybrid neural-numerical approaches may still be useful.

7.13 Open research questions

The lecture closes with several research challenges that remain important:

- reliable PINN error estimators;
- adaptive collocation strategies with theoretical guarantees;
- robust loss balancing;
- treatment of shocks and weak solutions;
- multiscale and high-frequency architectures;
- fair benchmark suites;
- uncertainty quantification and reproducibility.

7.14 Summary

The principal conclusions are:

- Classical numerical methods remain the strongest default for many known forward PDEs.
- PINNs become especially attractive when physics must be combined with sparse data, unknown parameters, or incomplete information.
- "Mesh-free" does not mean that no discretization or sampling decisions are required.
- PINNs may be expensive to train, but a reusable model can offer low-cost evaluation after training.
- High-frequency, stiff, multiscale, shock-dominated, and very high-accuracy problems remain challenging for standard PINNs.
- Hybrid methods that combine numerical analysis and machine learning are a natural direction for future development.

Exercises

1. For the Poisson equation on $[0,1]^2$, compare what must be chosen in an FEM implementation and in a PINN implementation. Organize the comparison under geometry, approximation space, sampling/discretization, solver/optimizer, and error assessment.

2. Consider the inverse problem $-\nabla \cdot (k\nabla u) = f$ with sparse measurements of u. Write a PINN loss for jointly approximating u and k. Discuss which aspects of the inverse problem are not resolved merely by minimizing that loss.

3. Explain why a small PDE residual at the collocation points need not imply a small global solution error.

4. Give an example of a problem for which a high offline training cost could be justified by low online cost.

5. Compare a standard PINN and a neural operator in the setting of repeated solutions for many input coefficient fields $k(x)$.

Bibliography

[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[2] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.

[3] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems*, 2021.

[4] S. Wang, Y. Teng, and P. Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.

[5] N. Rahaman et al. On the spectral bias of neural networks. *Proceedings of the International Conference on Machine Learning*, 2019.

[6] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.

[7] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. *International Conference on Learning Representations*, 2021.

[8] A. Quarteroni and A. Valli. *Numerical Approximation of Partial Differential Equations*. Springer, 1994.

[9] S. C. Brenner and L. R. Scott. *The Mathematical Theory of Finite Element Methods*. Springer, 3rd edition, 2008.

[10] R. J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM, 2007.

[11] R. J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, 2002.

[12] L. N. Trefethen. *Spectral Methods in MATLAB*. SIAM, 2000.

Chapter 8

Applications and Outlook

Abstract

These notes expand Lecture 8 of the PINNs course. The focus is no longer only on how a physics-informed neural network is formulated, but on when such a formulation is scientifically useful. We study inverse problems, data-assisted PINNs, diffusion and fluid applications, parameter and coefficient identification, advanced PINN variants, neural operators, hybrid numerical–learning methods, uncertainty, reproducibility, and responsible scientific use. The central message is that PINNs are most compelling when physics, data, and unknown quantities must be handled in a single computational framework.

8.1 From PINN methodology to scientific applications

Earlier lectures developed the basic ingredients of a PINN: neural-network approximation, automatic differentiation, residual construction, loss functions, optimization, treatment of initial and boundary data, time-dependent problems, nonlinear PDEs, and comparisons with classical discretization methods. This lecture asks a different question:

Main question

For which scientific problems are PINNs genuinely useful, and what additional care is required when moving from benchmark PDEs to realistic scientific inference?

The standard PINN pipeline can be summarized as

$$\text{PDE} \longrightarrow \text{residual} \longrightarrow \text{loss} \longrightarrow \text{optimization}.$$

In an application, however, this pipeline sits inside a larger workflow involving measurements, unknown quantities, modelling assumptions, validation, and uncertainty.

8.1.1 Physics, data, and learning

Many scientific machine-learning applications combine three sources of information:

- **Physics**: differential equations, conservation laws, constitutive relations, symmetries, or other structural constraints.

- **Data**: observations from experiments, sensors, simulations, or historical records.
- **Learning**: a flexible parametrized function class, usually a neural network, together with an optimization procedure.

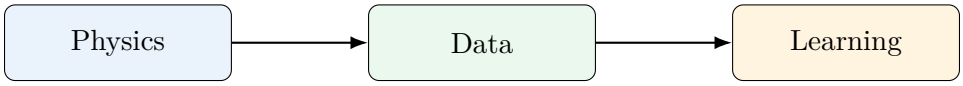


Figure 8.1: A useful way to view scientific machine learning: physics constrains the admissible behaviour, data connect the model to observations, and learning provides a flexible approximation mechanism.

For a fully specified forward PDE with accurate coefficients and boundary data, classical methods such as finite differences, finite elements, finite volumes, or spectral methods are often excellent choices. PINNs become more interesting when some part of the model is unknown, observations are incomplete, or the primary task is inference rather than a single forward solve.

8.1.2 What PINNs add beyond curve fitting

Suppose measurements are available at locations z_i:

$$\{(z_i, u_i)\}_{i=1}^{N_d}.$$

A purely data-driven neural model may minimize

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |u_\theta(z_i) - u_i|^2.$$

A PINN augments this with a physics term:

$$\mathcal{L}(\theta) = \lambda_d \mathcal{L}_{\text{data}}(\theta) + \lambda_f \mathcal{L}_{\text{PDE}}(\theta).$$

The PDE residual then provides information at collocation points where no measurements may be available. This is one reason physics can act as a form of regularization.

At the same time, the addition of a PDE residual does not guarantee success. The optimization problem can be difficult; the physical model can be misspecified; the unknown quantity can be non-identifiable; and poor sampling or loss weighting can dominate training.

Guiding principle

A good PINN application is not merely one in which a neural network can be trained. It is one in which incorporating physics into the learning problem contributes useful structure, information, or flexibility that is relevant to the scientific objective.

8.2 Forward and inverse problems

8.2.1 The distinction

In a forward problem, the model parameters are given and the solution is unknown. Schematically,

$$\text{known PDE, coefficients, forcing, and data} \quad \Longrightarrow \quad u.$$

In an inverse problem, observations of the state are used to infer an unknown parameter, coefficient, source, boundary condition, or constitutive relation:

$$\text{observations} + \text{physics} \quad \Longrightarrow \quad \text{unknown model quantity.}$$

Inverse problems are frequently ill-posed. There may be non-uniqueness, strong sensitivity to measurement noise, or insufficient information to distinguish between competing parameter values.

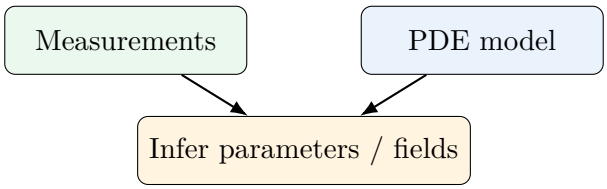


Figure 8.2: Inverse problems combine measurements and physical constraints to infer quantities that are not directly observed.

8.2.2 Example: unknown diffusion coefficient

Consider

$$-\nabla \cdot (k(x)\nabla u(x)) = f(x), \qquad x \in \Omega,$$

with suitable boundary conditions. In a forward problem, $k(x)$ is known and u is computed. In an inverse problem, we may observe only

$$\{(x_i^d, u_i)\}_{i=1}^{N_d}$$

and attempt to recover both the state u and the coefficient k.

A PINN formulation may use two networks,

$$u(x) \approx u_\theta(x), \qquad k(x) \approx k_\phi(x),$$

with residual

$$\mathcal{R}_f(x;\theta,\phi) = -\nabla \cdot (k_\phi(x)\nabla u_\theta(x)) - f(x).$$

The trainable parameters are now both θ and ϕ.

A typical loss is

$$\mathcal{L}(\theta,\phi) = \lambda_f \mathcal{L}_f + \lambda_d \mathcal{L}_d + \lambda_b \mathcal{L}_b,$$

where

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{R}_f(x_i^f;\theta,\phi)|^2,$$

and

$$\mathcal{L}_d = \frac{1}{N_d} \sum_{i=1}^{N_d} |u_\theta(x_i^d) - u_i|^2.$$

Boundary-condition terms are added according to the problem.

8.2.3 Physics as regularization

With sparse observations alone, many candidate functions may fit the measured data. Requiring the candidate solution to satisfy a PDE restricts the admissible class. This can reduce ambiguity, but it does not automatically remove ill-posedness.

Interpretation

Physics can act as a regularizer by shrinking the set of functions consistent with the observations. The effectiveness of this regularization depends on the correctness of the PDE, the informativeness of the data, and the identifiability of the unknown quantity.

8.2.4 What quantities may be inferred?

The inverse unknown can be

- a scalar parameter, such as viscosity or a reaction rate;
- a finite-dimensional parameter vector;
- a spatially varying coefficient, such as $k(x)$;
- a forcing or source term;
- an unknown boundary or initial condition;
- a constitutive or closure law.

8.2.5 Scalar parameter identification: Burgers' equation

For the viscous Burgers equation

$$u_t + uu_x = \nu u_{xx},$$

we may regard $\nu > 0$ as an unknown scalar. A PINN can treat ν as a trainable parameter together with the network weights. If u_θ approximates the state, then

$$\mathcal{R}(x, t; \theta, \nu) = \partial_t u_\theta + u_\theta \partial_x u_\theta - \nu \partial_{xx} u_\theta.$$

Measurements of u are combined with the residual loss and initial/boundary terms. This is conceptually simple, but whether ν can be recovered accurately depends on whether the data actually contain enough information about viscous effects.

8.2.6 Functional coefficient identification

If the unknown is a function, for example $k(x)$, the inverse problem is much more flexible and therefore potentially more ill-posed. A neural network $k_\phi(x)$ introduces many degrees of freedom. Additional structure may be needed, such as positivity constraints, smoothness assumptions, parametrizations, priors, or sufficiently informative observations.

8.2.7 Identifiability and sensor placement

An unknown parameter is identifiable when distinct parameter values lead to distinguishable observations under the experimental setup. This is not merely an optimization question. If the observations are insensitive to the unknown, no optimizer can reliably recover it.

Sensor placement therefore matters. Measurements located in regions where the solution is sensitive to the unknown parameter can be far more informative than the same number of measurements in insensitive regions.

Inverse-problem caution

A visually plausible reconstruction is not the same as a verified inverse solution. Non-uniqueness, noise sensitivity, poor sensor coverage, model mismatch, and optimization failure can all produce apparently reasonable but scientifically unreliable estimates.

8.3 Data-assisted PINNs

8.3.1 Combining physics and observations

In many experiments neither the PDE nor the observations alone are sufficient. A data-assisted PINN combines

$$\boxed{\text{PDE residual}} + \boxed{\text{boundary/initial conditions}} + \boxed{\text{observational data}}.$$

For a time-dependent approximation $u_\theta(x,t)$, a typical loss is

$$\mathcal{L}(\theta) = \lambda_f \mathcal{L}_f + \lambda_b \mathcal{L}_b + \lambda_0 \mathcal{L}_0 + \lambda_d \mathcal{L}_d.$$

The four components enforce the PDE, boundary conditions, initial conditions, and data fit, respectively.

8.3.2 Sparse measurements

Suppose measurements are available at only a small number of space-time locations,

$$\{(x_i, t_i, u_i)\}_{i=1}^{N_d}.$$

Then

$$\mathcal{L}_d(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |u_\theta(x_i, t_i) - u_i|^2.$$

The PDE residual supplies additional constraints between sensor locations. This is particularly attractive when dense data collection is expensive but a reasonably accurate physical model is known.

8.3.3 Noisy measurements

Real measurements often satisfy

$$u_i = u(x_i, t_i) + \varepsilon_i,$$

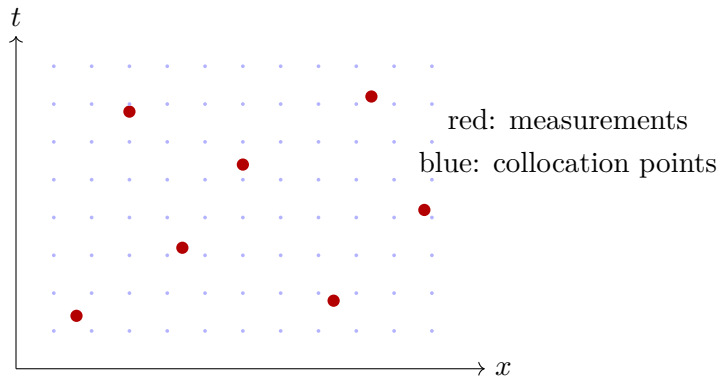


Figure 8.3: Sparse measurements can be supplemented by many physics collocation points.

where ε_i represents measurement error. Exact interpolation may therefore be undesirable. If the data loss is weighted too heavily, a network may overfit noise. If the physics loss is weighted too heavily, genuine information contained in the observations may be ignored.

This motivates regularization, robust data misfit functions, careful scaling, and uncertainty analysis.

8.3.4 Loss balancing

A multi-term PINN objective can contain components with very different magnitudes and gradient scales:

$$\mathcal{L} = \lambda_f \mathcal{L}_f + \lambda_b \mathcal{L}_b + \lambda_d \mathcal{L}_d.$$

If one term dominates the parameter gradients, the optimizer may make progress on that objective while neglecting the others.

Common responses include

- manual tuning of $\lambda_f, \lambda_b, \lambda_d$;
- nondimensionalization and rescaling;
- adaptive weighting strategies;
- curriculum or staged training.

8.3.5 Validation data

A useful workflow distinguishes among

- data used directly for training;
- collocation points used to enforce physics;
- validation observations used for model selection or stopping;
- independent test data used only for final assessment.

This separation is especially important when observational data are scarce.

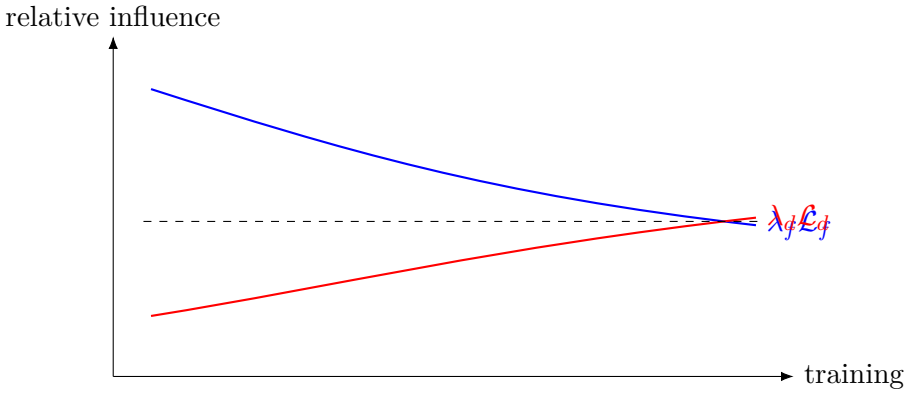


Figure 8.4: Loss balancing aims to prevent one objective from overwhelming the others throughout training.

8.3.6 Purely data-driven learning versus data-assisted PINNs

Feature	Pure ML	Data-assisted PINN
Uses observations	yes	yes
Uses PDE constraints	no	yes
Can exploit unlabelled collocation points	no	yes
Requires a physical model	no	yes
Training complexity	variable	often higher

Table 8.1: Conceptual comparison of purely data-driven and physics-assisted learning.

The physics-assisted approach is most attractive when the PDE model is trustworthy enough to be informative and the observations are limited enough that physical structure is valuable.

8.4 Representative applications

8.4.1 Diffusion and heat transfer

A generic diffusion model is

$$\frac{\partial u}{\partial t} - \nabla \cdot (k(x)\nabla u) = s(x,t).$$

Such equations arise in heat conduction, contaminant transport, groundwater flow, tissue modelling, battery models, and materials science.

Possible PINN tasks include

- reconstructing the field u from sparse measurements;
- identifying $k(x)$;
- estimating a source $s(x,t)$;
- inferring unknown boundary fluxes.

8.4.2 Inverse heat conduction

In inverse heat conduction, temperature sensors may be available at only a few locations. The heat equation couples temperatures throughout the domain and in time, allowing the residual to transmit information between observations.

A typical inverse problem might estimate thermal conductivity from measured temperature histories. If boundary heat flux is unknown, it may also be parameterized and inferred jointly with the state. These tasks remain sensitive to noise and identifiability, so independent validation is essential.

8.4.3 Pollutant transport

A simplified advection–diffusion equation is

$$\partial_t c + \boldsymbol{v} \cdot \nabla c - \nabla \cdot (D \nabla c) = s.$$

Here c is concentration, $\boldsymbol{v}$ is a velocity field, D is a diffusivity, and s is a source. Depending on the application, a PINN may estimate an unknown source, recover a transport coefficient, or reconstruct concentration away from sensor locations.

8.4.4 Fluid dynamics

For incompressible flow, the Navier–Stokes equations can be written as

$$\partial_t \boldsymbol{u} + (\boldsymbol{u} \cdot \nabla)\boldsymbol{u} + \nabla p - \nu \Delta \boldsymbol{u} = \boldsymbol{f}, \qquad \nabla \cdot \boldsymbol{u} = 0.$$

A PINN can include separate residuals for momentum and incompressibility, together with boundary conditions and measurements of velocity or pressure.

A representative loss is

$$\mathcal{L} = \lambda_m \mathcal{L}_{\text{momentum}} + \lambda_c \mathcal{L}_{\text{continuity}} + \lambda_d \mathcal{L}_{\text{data}}.$$

Possible inverse tasks include estimation of viscosity, Reynolds number, pressure, or forcing.

8.4.5 Pressure recovery from velocity measurements

Pressure can be difficult to measure directly. If velocity observations are available, the momentum equation links the velocity field to the pressure gradient. A PINN can therefore learn a joint representation $(\boldsymbol{u}_\theta, p_\theta)$ subject to the momentum and incompressibility equations.

This does not eliminate the need for appropriate pressure normalization or boundary information. It does, however, illustrate how an unobserved field can be inferred indirectly through a coupled physical law.

8.4.6 Reaction–diffusion systems

A generic reaction–diffusion model takes the form

$$\partial_t u - D \Delta u = R(u; \eta),$$

where D is a diffusion coefficient and η represents reaction parameters. Depending on the observations, a PINN can be used to identify reaction rates, diffusivities, or state variables. Multicomponent systems introduce additional coupled residuals and may present stiffness and scale-separation difficulties.

8.4.7 Digital-twin viewpoint

A digital twin is a computational representation of a physical system that is repeatedly updated using new observations. PINNs are relevant to this idea because they can fuse a differential-equation model with incoming data and unknown parameters. In practice, a digital-twin workflow also requires data pipelines, calibration, uncertainty estimates, computational efficiency, and robust validation.

8.5 Why realistic applications are difficult

Ideal benchmark examples often hide the complications encountered in practice:

- measurement noise;
- uncertain or time-varying boundary conditions;
- unknown sources or forcings;
- complex geometry;
- multiscale behaviour;
- incomplete physics or model mismatch;
- limited sensor coverage;
- expensive optimization.

Model mismatch

The PDE used in training may be only an approximation of the real process. A small residual with respect to an imperfect PDE does not imply that the learned solution accurately represents reality.

A practical application workflow therefore includes more than training:

1. define the physical model and unknown quantities;
2. collect and preprocess observations;
3. nondimensionalize and scale variables;
4. select architecture, collocation strategy, and loss terms;
5. train while monitoring all loss components;
6. validate on independent observations or trusted numerical results;
7. compare with credible baselines;

8. quantify uncertainty where possible;
9. document the full computational procedure.

8.6 Advanced PINN methods

The basic PINN framework is elegant but can suffer from slow convergence, spectral bias, loss imbalance, multiscale dynamics, long-time error accumulation, and geometry-related difficulties. Several extensions are designed to address these issues.

8.6.1 Residual-based adaptive sampling

Rather than keeping collocation points fixed, one can use the current residual to decide where additional points are needed. A simple strategy is:

1. train an initial network;
2. evaluate $|\mathcal{R}|$ on a large candidate set;
3. identify high-residual regions;
4. add new collocation points there;
5. continue training.

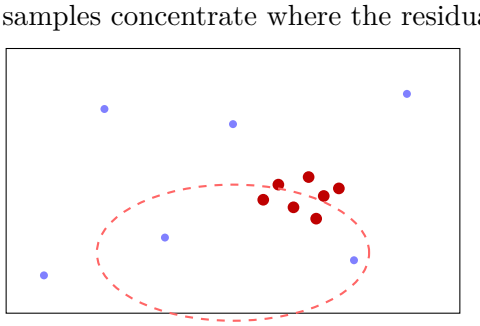


Figure 8.5: Schematic of residual-based adaptive sampling.

This strategy is especially relevant near shocks, boundary layers, localized sources, and sharp transitions.

8.6.2 Improved representations and Fourier features

Standard multilayer perceptrons can have difficulty representing oscillatory or high-frequency components. One common idea is to transform an input x into oscillatory features such as

$$\gamma(x) = (\sin(2\pi Bx), \cos(2\pi Bx)),$$

where B is a matrix or collection of frequencies. The network is then applied to $\gamma(x)$ rather than directly to x.

Other representation strategies include sinusoidal activations, adaptive activation functions, and multiscale architectures.

8.6.3 Domain decomposition

Instead of representing the entire solution with one network, the physical domain can be split into subdomains. Separate subnetworks are trained locally and coupled through interface conditions.

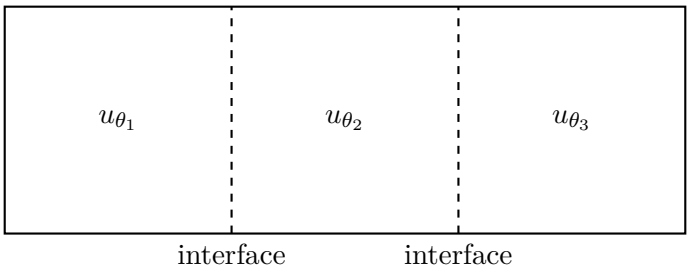


Figure 8.6: Domain decomposition replaces one global network by local networks coupled at interfaces.

Potential advantages include smaller local problems, parallelization, and improved treatment of localized features. The price is the need to enforce interface continuity, flux balance, or other coupling conditions. XPINNs and cPINNs are examples of domain-decomposition PINN strategies.

8.6.4 Neural operators

A standard PINN usually learns one solution function,

$$(x, t) \mapsto u(x, t).$$

A neural operator instead aims to learn a map between functions,

$$\mathcal{G} : a(\cdot) \mapsto u(\cdot),$$

where a might represent a coefficient field, forcing function, initial condition, or other functional input. This is attractive when many related PDE solves are required.

DeepONet and Fourier neural operators are prominent examples of operator-learning approaches. Conceptually, this shifts the objective from approximating one solution to approximating a family of solution maps.

8.6.5 Hybrid classical–learning methods

Learning and classical numerics need not be competitors. Hybrid strategies include

- generating high-quality FEM/FDM data for neural surrogates;
- learning unknown closures or constitutive laws while retaining a classical PDE solver;
- using PINNs for parameter estimation and a classical solver for subsequent forward prediction;
- embedding learned corrections in numerical time-stepping or discretization schemes.

Outlook

A major direction in scientific machine learning is the careful integration of numerical analysis, physical modelling, and learned components rather than the wholesale replacement of one methodology by another.

8.6.6 Software ecosystems

Typical workflows use general automatic-differentiation libraries such as PyTorch, TensorFlow, or JAX. DeepXDE provides PINN and differential-equation examples. Credible evaluation often also requires classical software such as FEniCS, PETSc, Firedrake, or deal.II to produce numerical baselines or reference solutions.

8.7 Uncertainty, validation, and reproducibility

8.7.1 Why uncertainty matters

A deterministic PINN prediction is only a point estimate. Scientific conclusions may depend on uncertainty from

- measurement noise;
- uncertain parameters;
- model discrepancy;
- optimization variability;
- finite and imperfect sensor placement.

Uncertainty-aware PINN methods remain an active area of research.

8.7.2 Scientific reproducibility

PINN results can depend strongly on implementation choices. At minimum, one should report

- random seeds;
- architecture and activation functions;
- optimizer and learning-rate schedule;
- all loss weights;
- collocation and observation sampling;
- data preprocessing and nondimensionalization;
- precision, hardware, and software versions;
- stopping criteria;
- validation protocol and error metrics.

8.7.3 Fair benchmarking

A scientifically meaningful comparison should report accuracy together with computational cost and problem setup. Important quantities include training time, inference time, hardware, number of collocation points, number of trainable parameters, numerical resolution of baseline methods, and the error metric used.

Benchmarking warning

A PINN result is not convincing if it is compared only with a weak, mismatched, or poorly tuned baseline. The classical comparison should be appropriate for the PDE, geometry, regularity, and accuracy regime under study.

8.7.4 A validation hierarchy

A practical validation hierarchy can be viewed as

training diagnostics $\longrightarrow$ PDE residual checks $\longrightarrow$ comparison with independent data $\longrightarrow$ comparison with tr

A small training loss alone is not sufficient evidence that the learned state or inferred parameter is correct.

8.7.5 Responsible scientific claims

PINNs may be used in engineering, environmental monitoring, materials, medicine, and other settings where model outputs can influence decisions. Responsible use requires transparent assumptions, careful validation, reproducibility, and uncertainty reporting.

Claims such as “the PINN discovered the true physics” or “PINNs replace numerical solvers” should be avoided unless supported by appropriately designed evidence. A more defensible statement identifies exactly what was inferred, under what assumptions, with what data, against which baselines, and with what uncertainty.

8.8 Summary

The main application-level conclusions are:

- PINNs are especially natural for inverse problems and data–physics fusion.
- Unknown scalar parameters, spatial coefficients, sources, boundary data, and latent fields can all be incorporated as trainable quantities.
- Identifiability and sensor placement are fundamental; they cannot be repaired merely by changing the optimizer.
- Sparse or noisy data can benefit from physical constraints, but loss balancing and model mismatch require care.
- Representative application areas include diffusion, heat conduction, pollutant transport, fluid dynamics, reaction–diffusion systems, and digital-twin style inference.

- Advanced methods address sampling, representation, decomposition, and repeated-query settings.
- Neural operators target families of PDE solution maps rather than a single solution.
- Hybrid numerical–learning approaches are often more realistic than treating PINNs and classical solvers as mutually exclusive.
- Reproducibility, benchmarking, validation, and uncertainty are part of the scientific method, not optional extras.

Course takeaway

PINNs are most powerful when physics, data, and unknown quantities meet. A strong scientific machine-learning practitioner should understand both classical numerical methods and learning-based approaches and select the computational strategy according to the scientific task.

Exercises

1. Explain the difference between a forward and an inverse PDE problem.
2. For $-\nabla \cdot (k\nabla u) = f$, write a PINN loss for recovering both u and an unknown $k(x)$ from sparse data.
3. Why can the PDE residual be interpreted as a regularizer in a data-assisted problem?
4. Why is parameter identifiability conceptually different from optimization difficulty?
5. Give two reasons why sensor placement can affect inverse-problem accuracy.
6. Explain why a small training loss does not guarantee a correct scientific solution.
7. Describe one situation in which residual-based adaptive sampling may help.
8. Contrast a PINN with a neural operator.
9. Give two examples of hybrid classical–learning workflows.
10. List the information that should be reported for a reproducible PINN experiment.

Bibliography

[1] I. E. Lagaris, A. Likas, and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," *IEEE Transactions on Neural Networks*, 1998.

[2] J. Sirignano and K. Spiliopoulos, "DGM: A deep learning algorithm for solving partial differential equations," *Journal of Computational Physics*, 2018.

[3] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs," *Journal of Computational Physics*, 2019.

[4] G. E. Karniadakis et al., "Physics-informed machine learning," *Nature Reviews Physics*, 2021.

[5] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, "DeepXDE: A deep learning library for solving differential equations," *SIAM Review*, 2021.

[6] A. D. Jagtap, K. Kawaguchi, and G. E. Karniadakis, "Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks," *Proceedings of the Royal Society A*, 2020.

[7] A. D. Jagtap and G. E. Karniadakis, "Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear PDEs," *Communications in Computational Physics*, 2020.

[8] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, 2021.

[9] Z. Li et al., "Fourier neural operator for parametric partial differential equations," *International Conference on Learning Representations*, 2021.

[10] A. Krishnapriyan et al., "Characterizing possible failure modes in physics-informed neural networks," *NeurIPS*, 2021.

[11] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, 2021.

[12] S. Wang, H. Wang, and P. Perdikaris, "On the eigenvector bias of Fourier feature networks," *Computer Methods in Applied Mechanics and Engineering*, 2021/2022.

[13] L. McClenny and U. Braga-Neto, "Self-adaptive physics-informed neural networks," arXiv preprint, 2020.

[14] N. Kovachki et al., "Neural operator: Learning maps between function spaces with applications to PDEs," *Journal of Machine Learning Research*, 2023.

[15] S. Cuomo et al., "Scientific machine learning through physics-informed neural networks: Where we are and what's next," *Journal of Scientific Computing*, 2022.

Concluding remarks

These notes form a self-contained introduction to physics-informed neural networks, from foundations and automatic differentiation through formulation, time dependence, nonlinear training, classical comparisons, and applications.